USC Viterbi
School of Engineering

Voronoi-Based K Nearest Neighbor Search for Spatial Network Databases

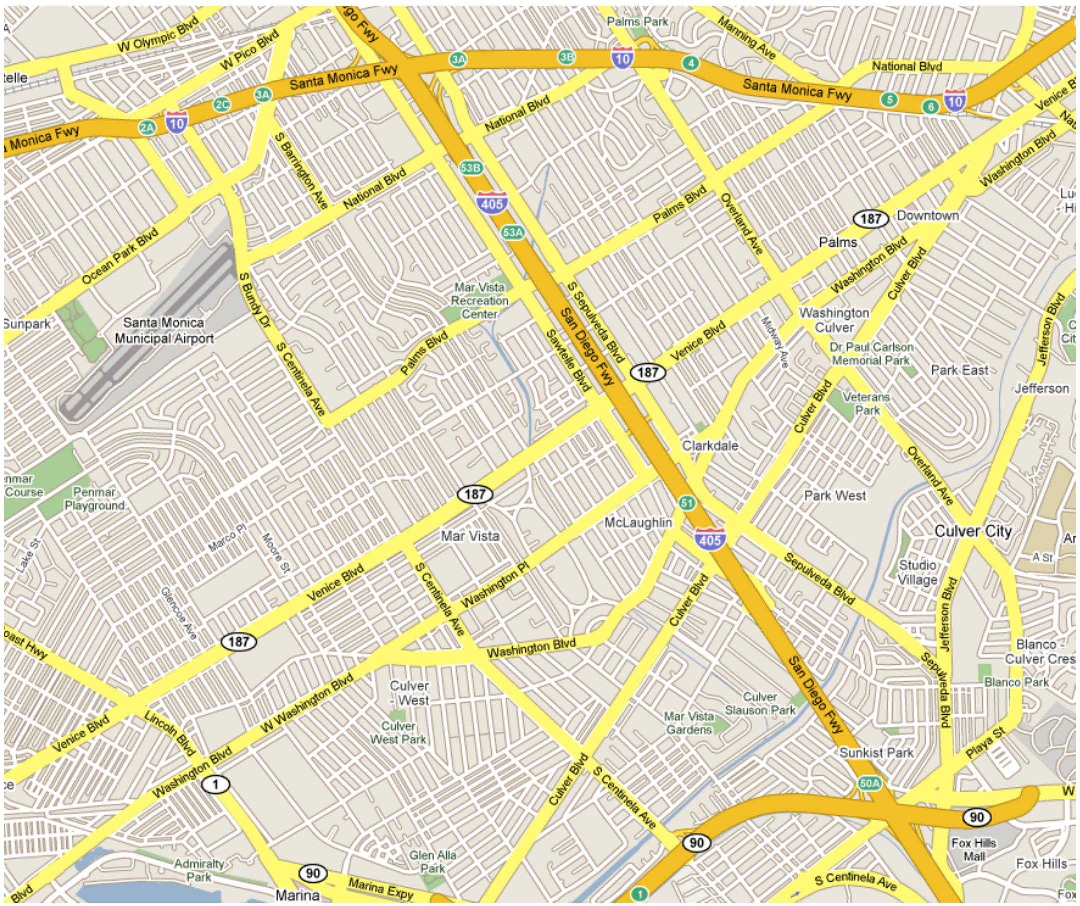
Instructor: Cyrus Shahabi

USC Viterbi School of Engineering

Agenda

- Problem Definition
- Related Work
- Voronoi Diagrams
- Voronoi Based Network Nearest Neighbor (VN^3)
- Experiments
- Discussion

Problem Definition

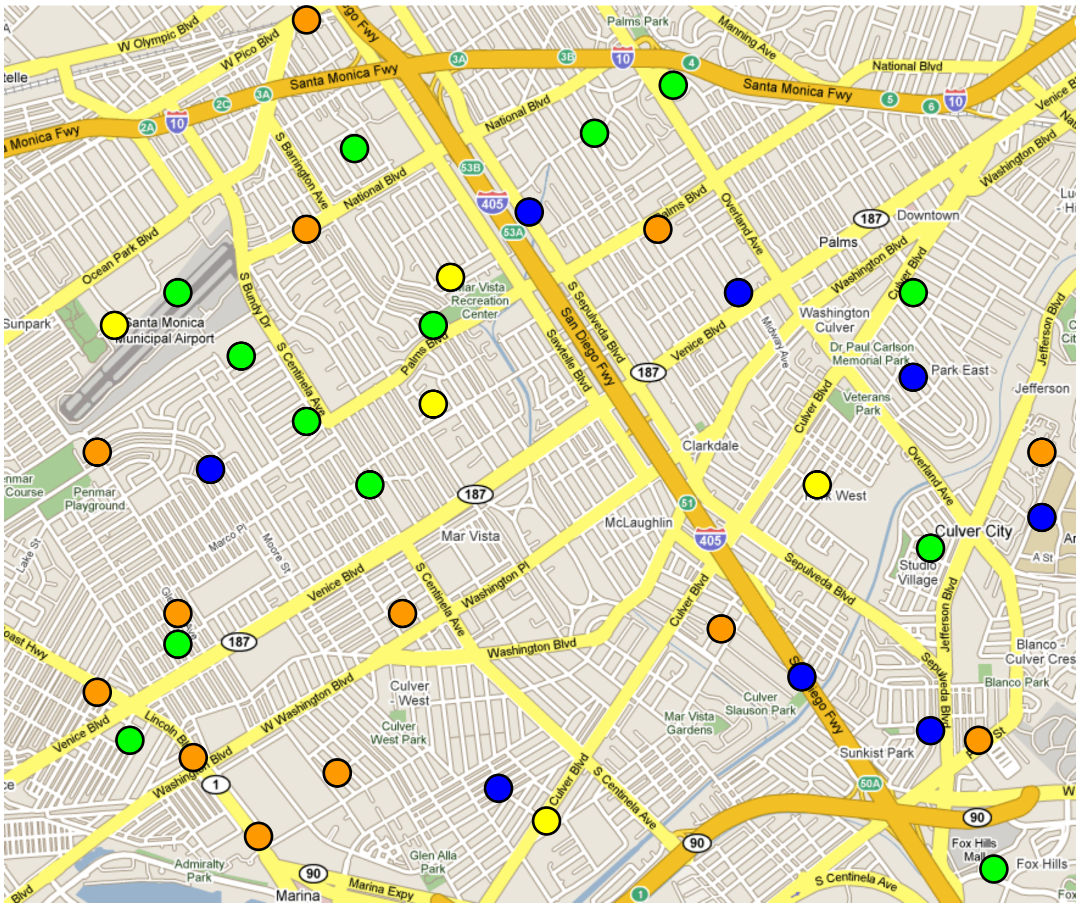


kNN Problem:
Given a set of objects *P* and query point *q*, find the ***k*** objects in *P* that are closest to *q*

3

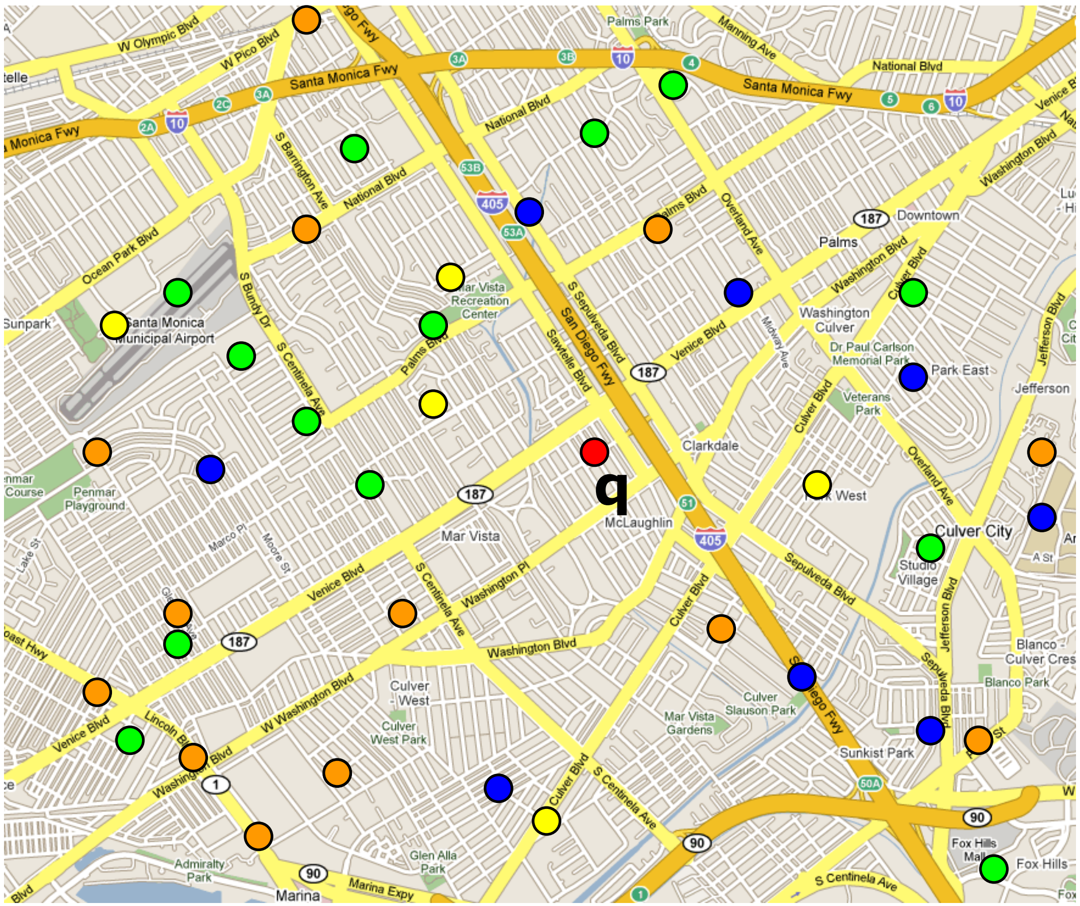
USC Viterbi
School of Engineering

Problem Definition



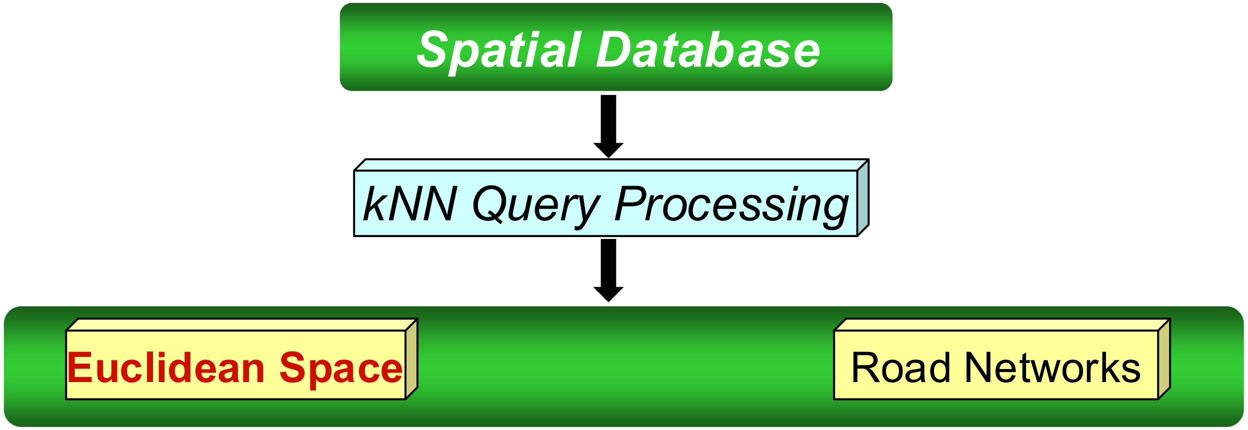
kNN Problem:
Given a set of objects *P* and query point *q*, find the ***k*** objects in *P* that are closest to *q*

Problem Definition



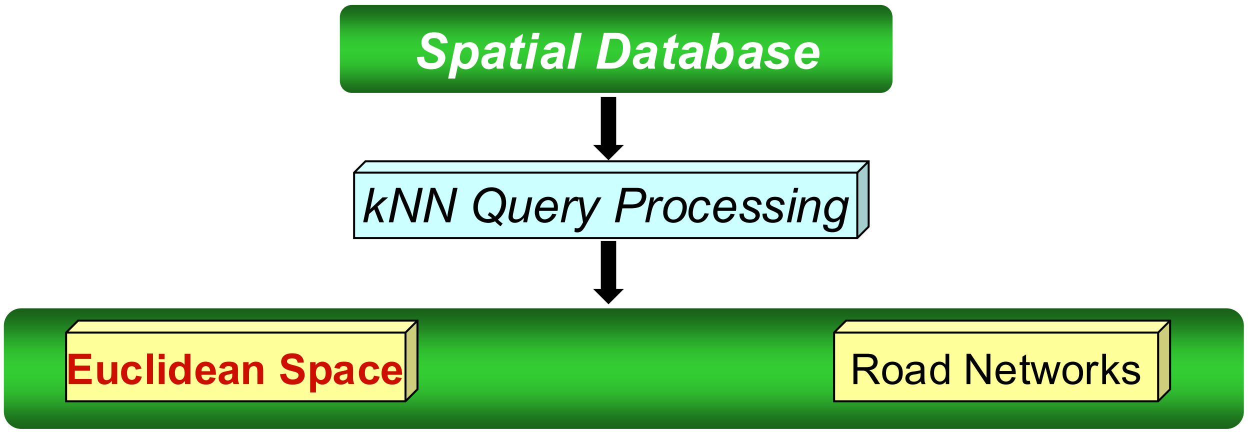
kNN Problem:
Given a set of objects *P* and query point *q*, find the ***k*** objects in *P* that are closest to *q*

Related Work



USC Viterbi School of Engineering

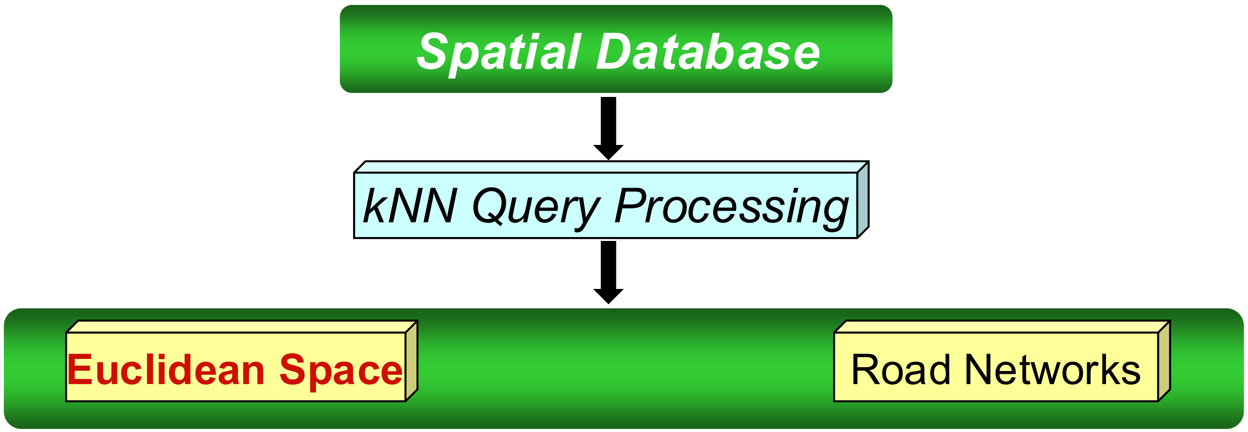
Related Work



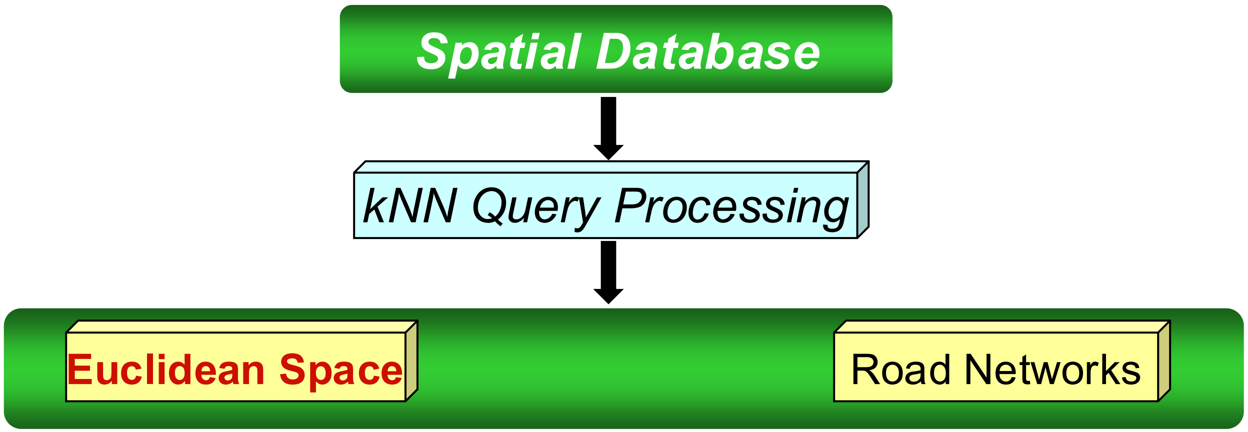
- ✓ **NN Query: Roussopoulos et al., SIMGOD 1995**
- ✓ **K-NN for moving query point: Zong et al., SSTD 2001**
- ✓ **Time-parameterized queries : Tao et al., SIMGOD 2002**
- ✓ **Continuous NN Search: Tao et al,. VLDB 2002**

Object Based

Related Work



Related Work

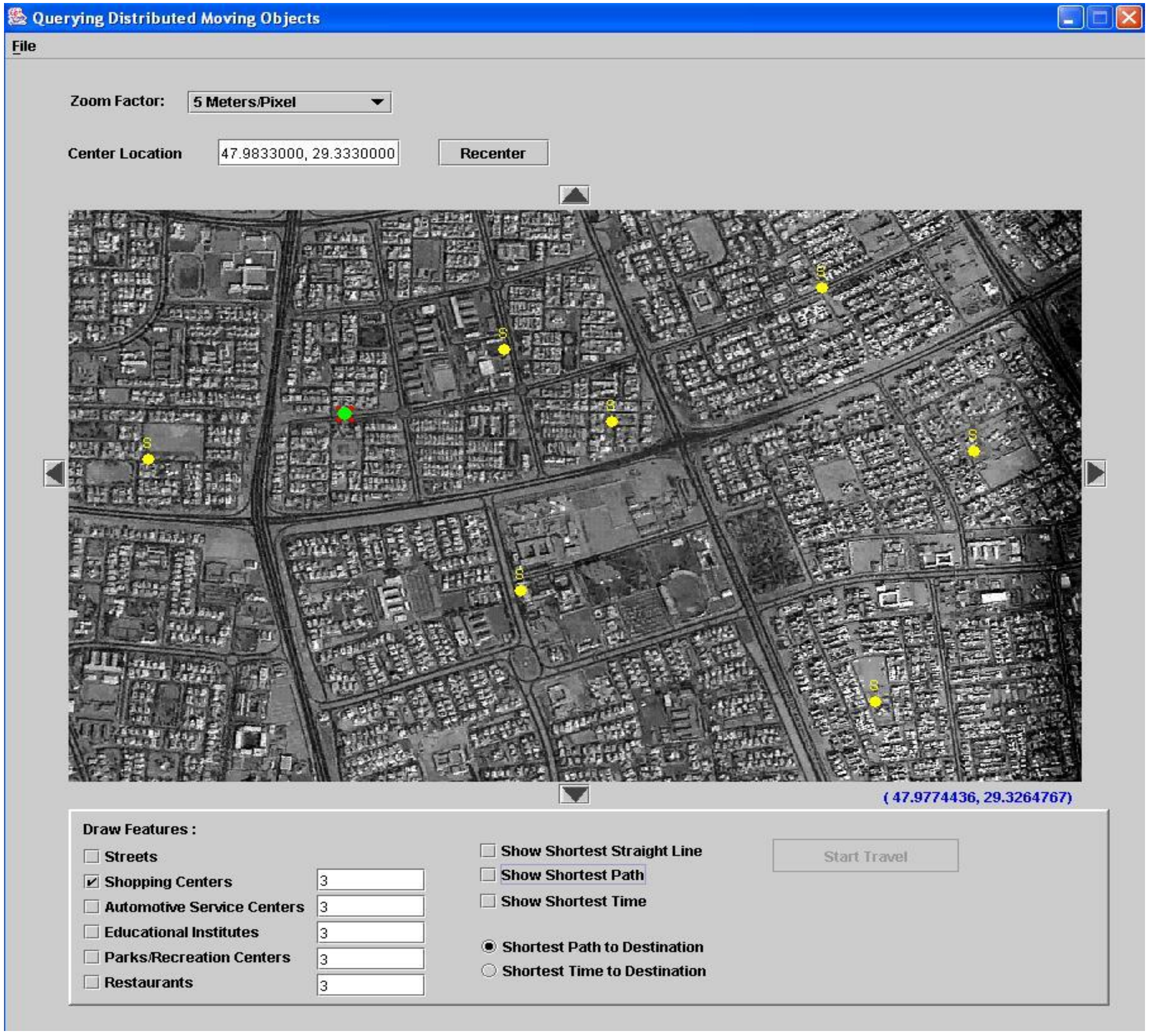


- ✓ **Sina-Continuous querying: Mokbel et al., SIGMOD 2004**
- ✓ **Sea-CNN queries: Xiong et al., ICDE 2005**
- ✓ **Monitoring kNN on moving objects: Yu et al., ICDE 2005**
- ✓ **Conceptual partitioning: Mouratidis et al., SIGMOD 2005**

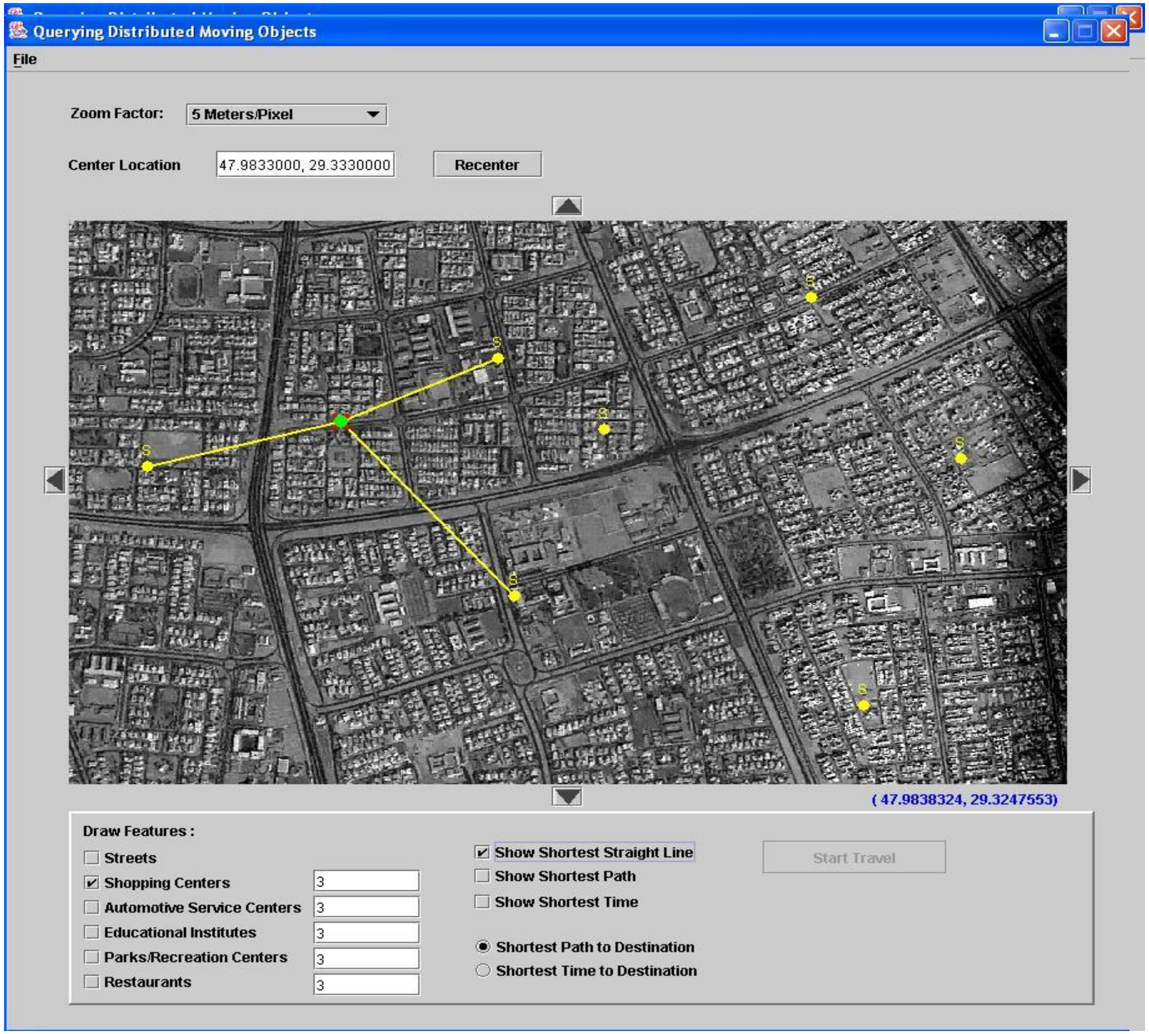
Space Based

USC Viterbi School of Engineering

Example 1:
Finding the 3 closest shopping centers

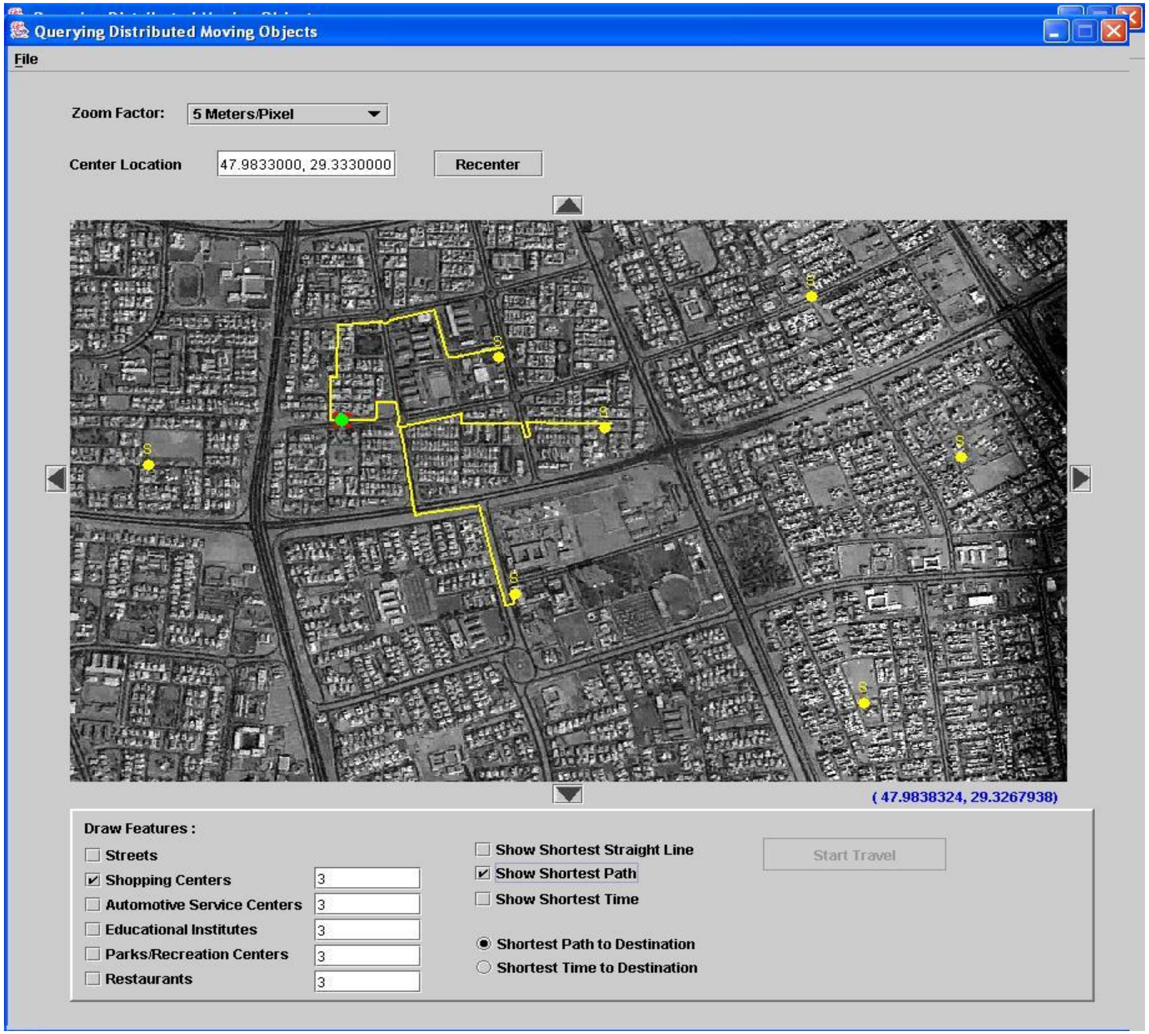


Example 1:
Finding the 3 closest shopping centers



USC Viterbi School of Engineering

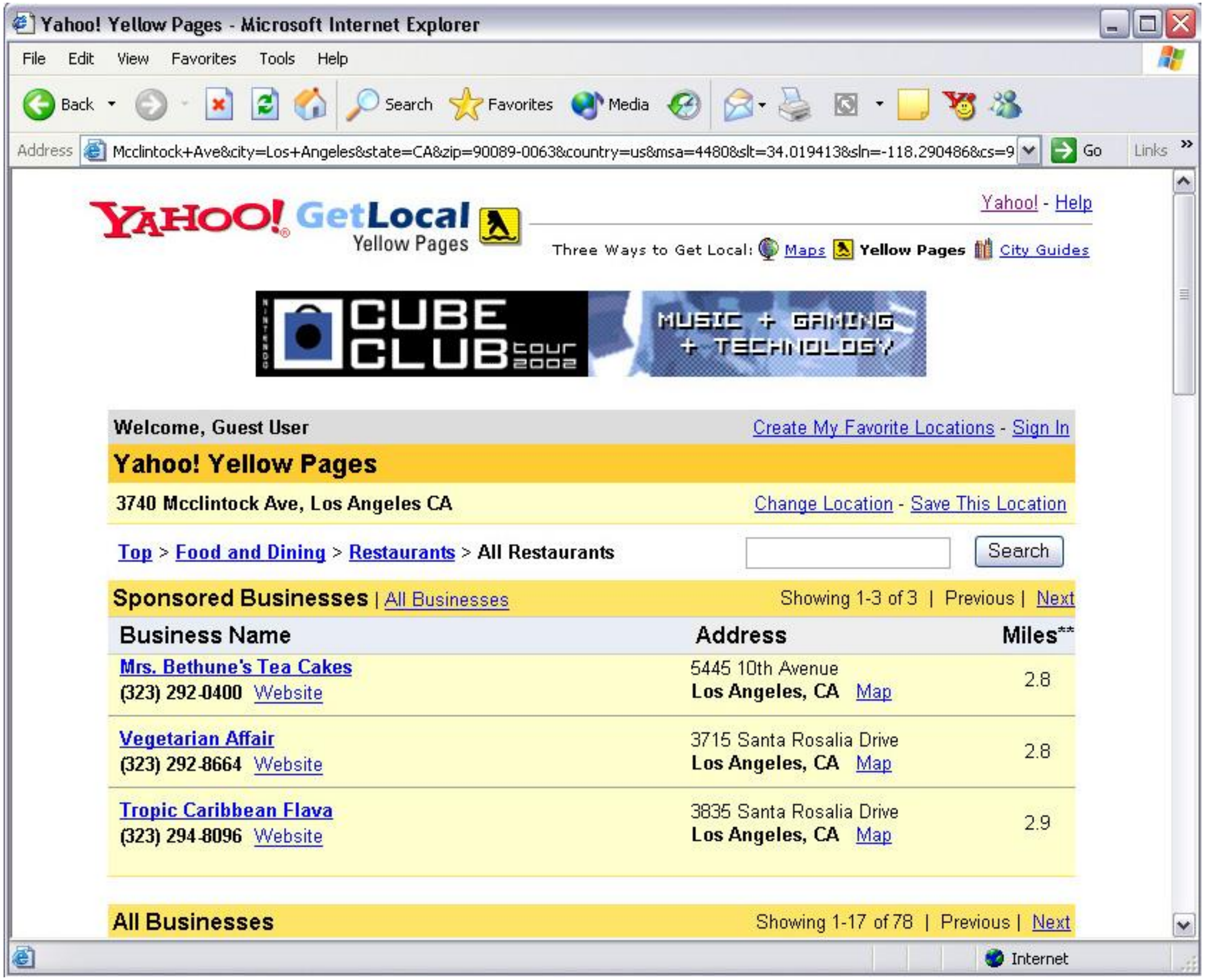
Example 1:
Finding the 3 closest shopping centers



USC Viterbi
School of Engineering

Example 2:

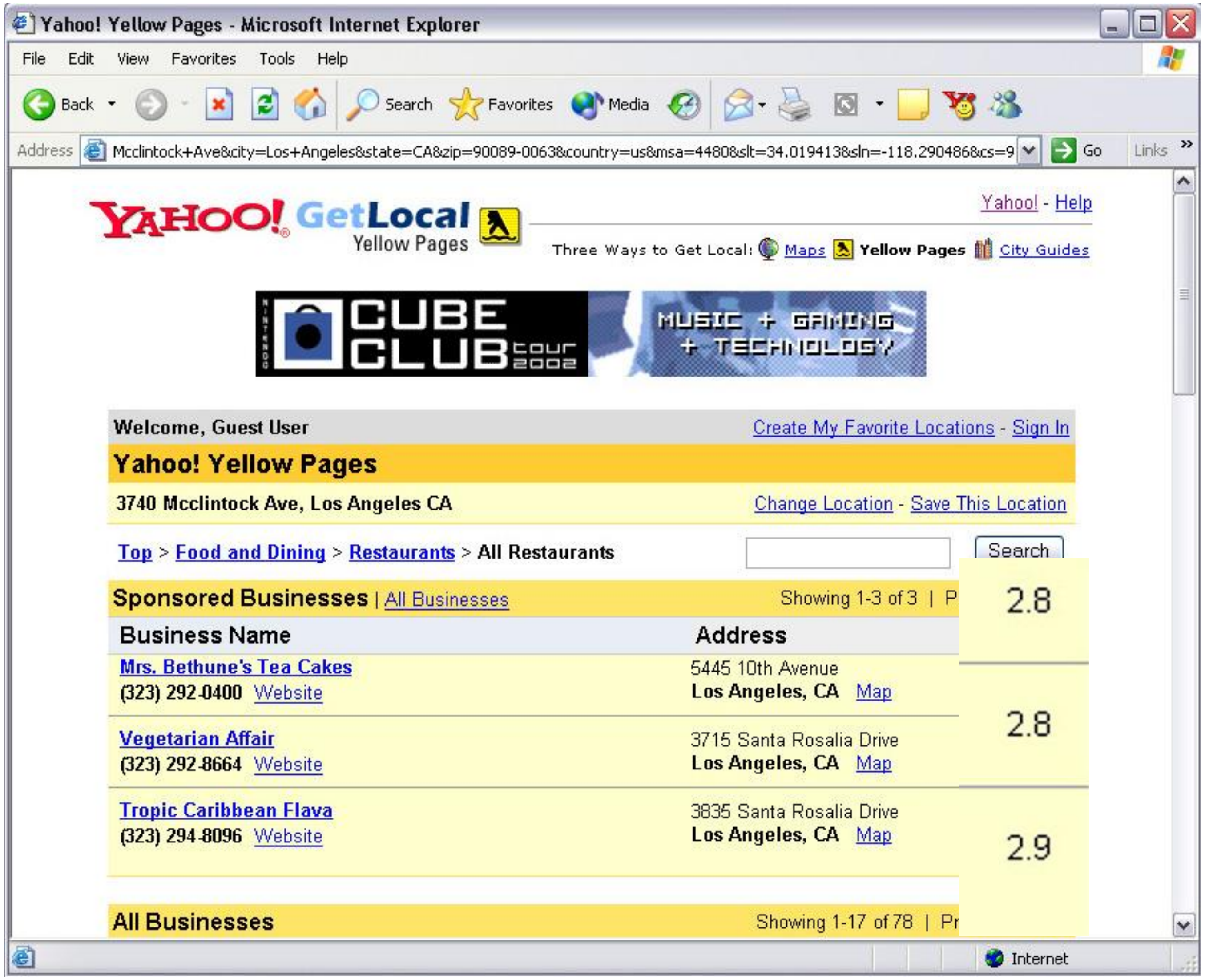
Finding the 3 closest restaurants to USC with Yahoo



USC Viterbi School of Engineering

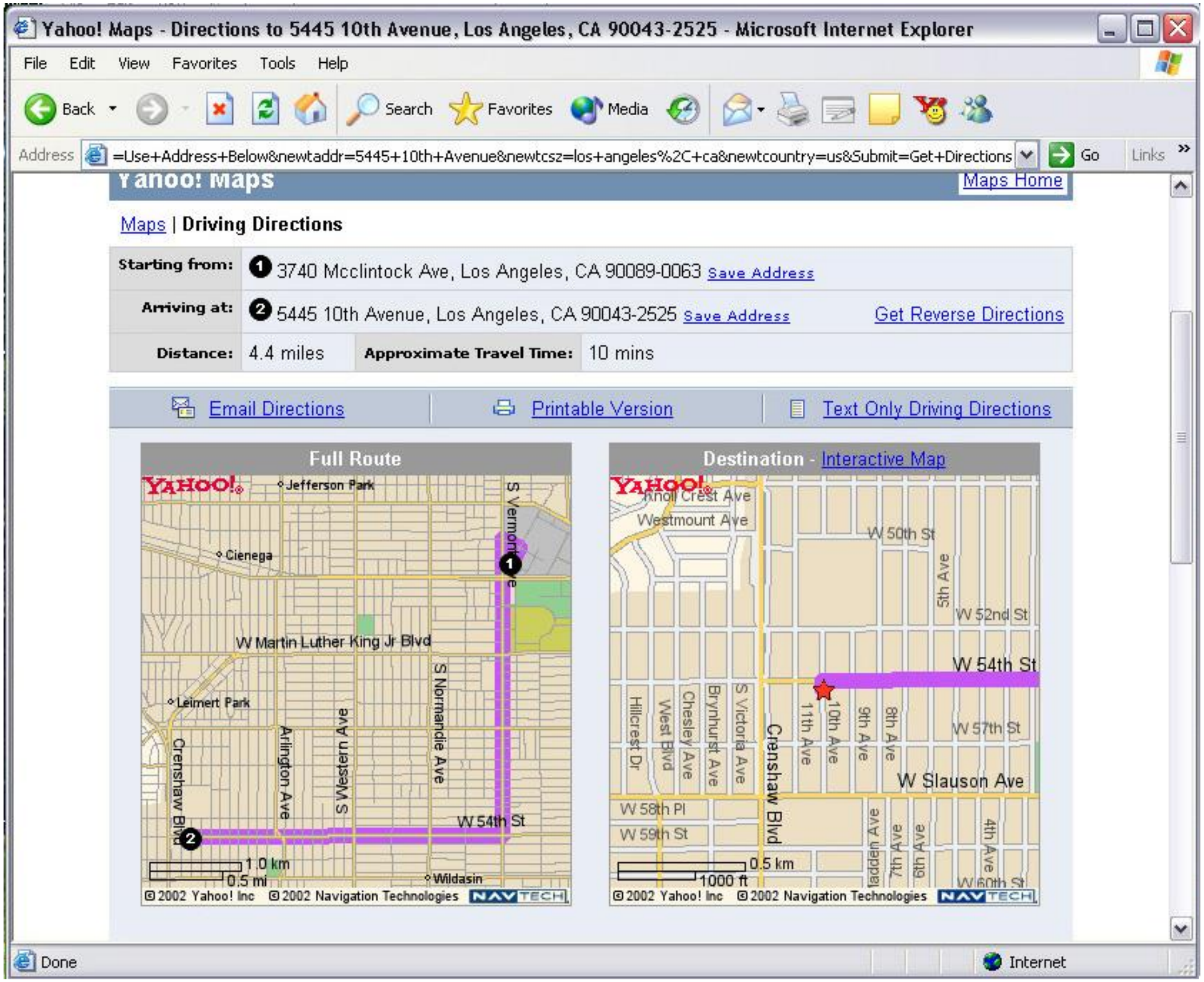
Example 2:

Finding the 3 closest restaurants to USC with Yahoo



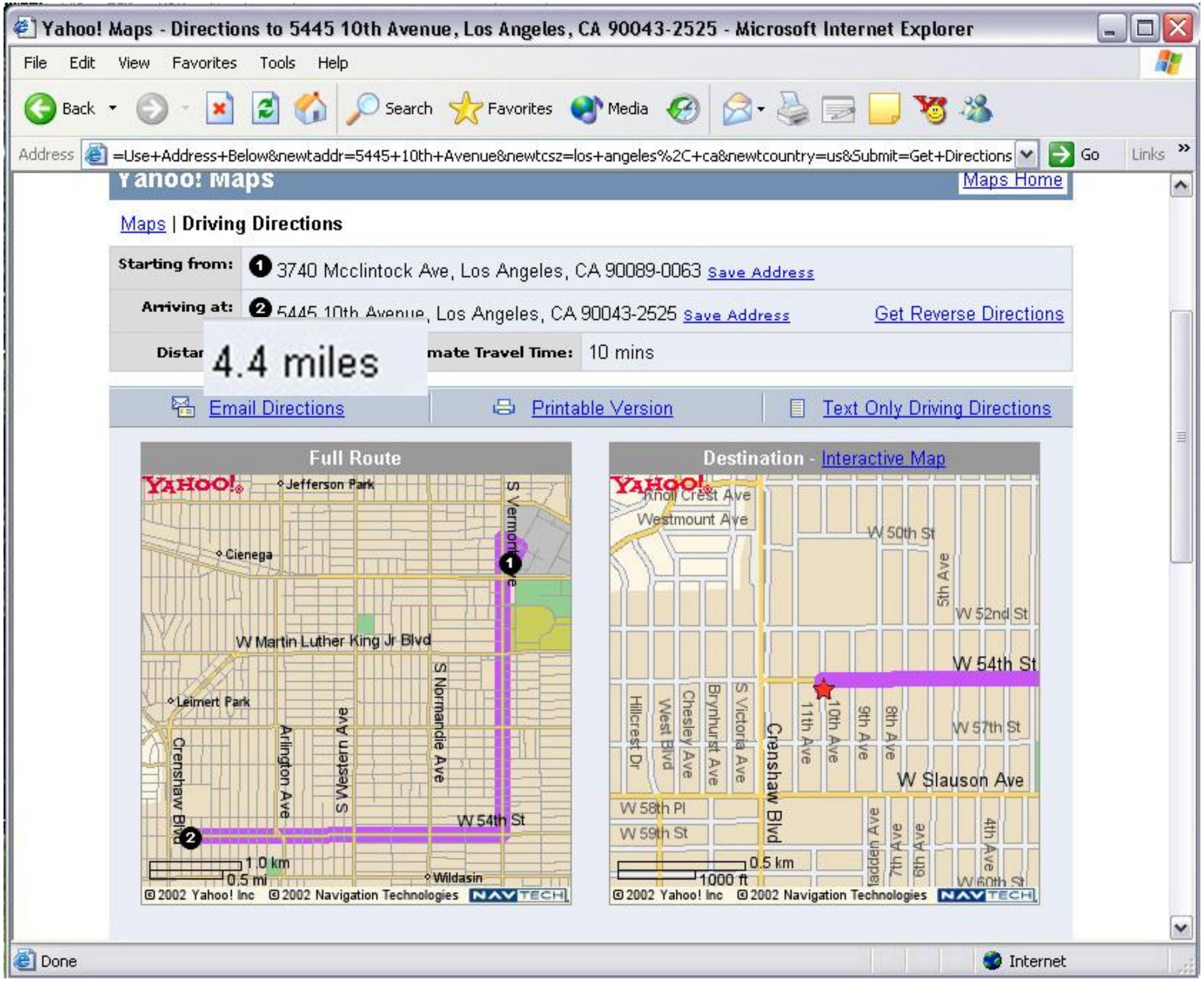
Example 2:

Finding the 3 closest restaurants to USC with Yahoo



USC Viterbi School of Engineering

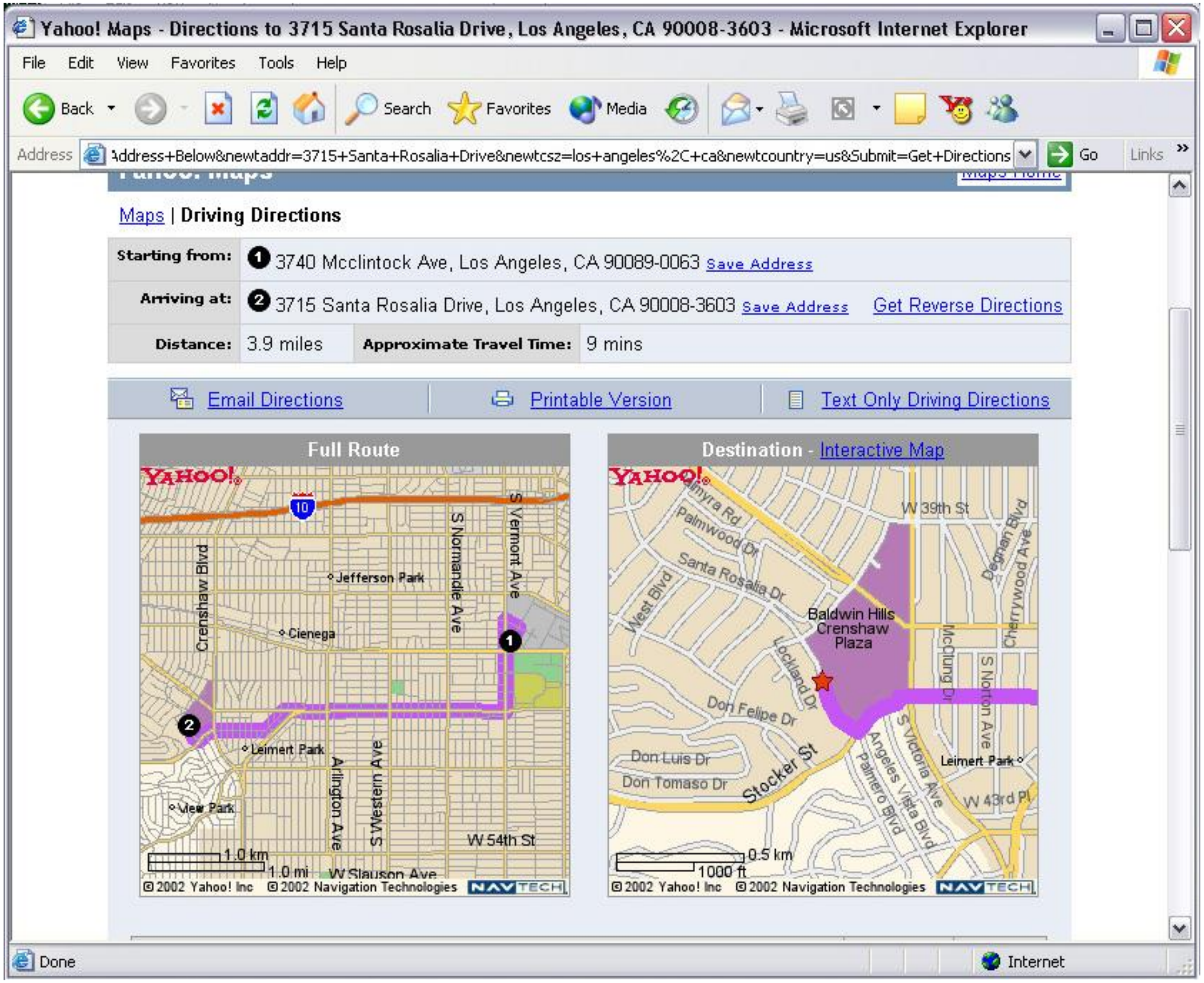
Example 2:
Finding the 3 closest restaurants to USC with Yahoo



USC Viterbi School of Engineering

Example 2:

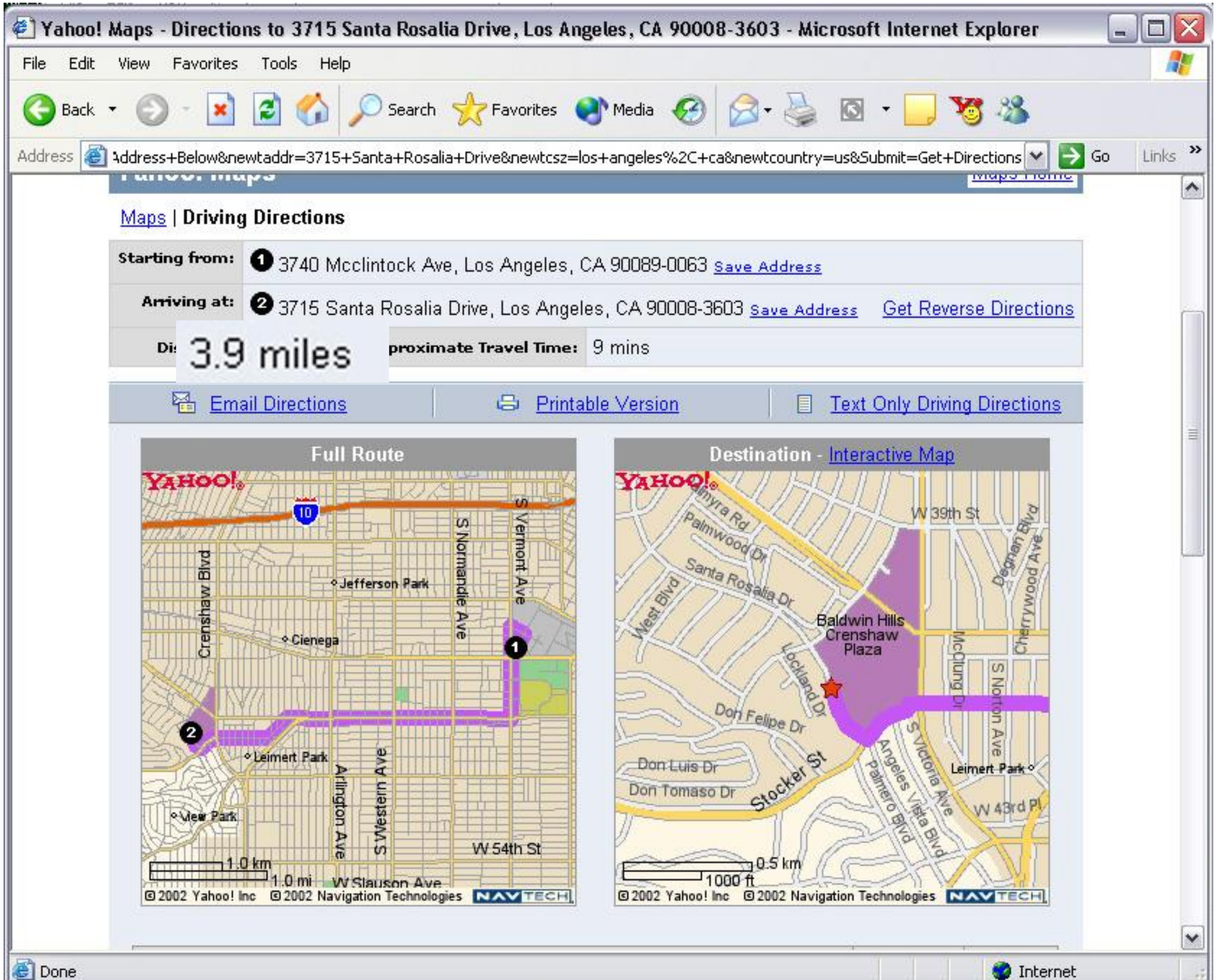
Finding the 3 closest restaurants to USC with Yahoo



USC Viterbi School of Engineering

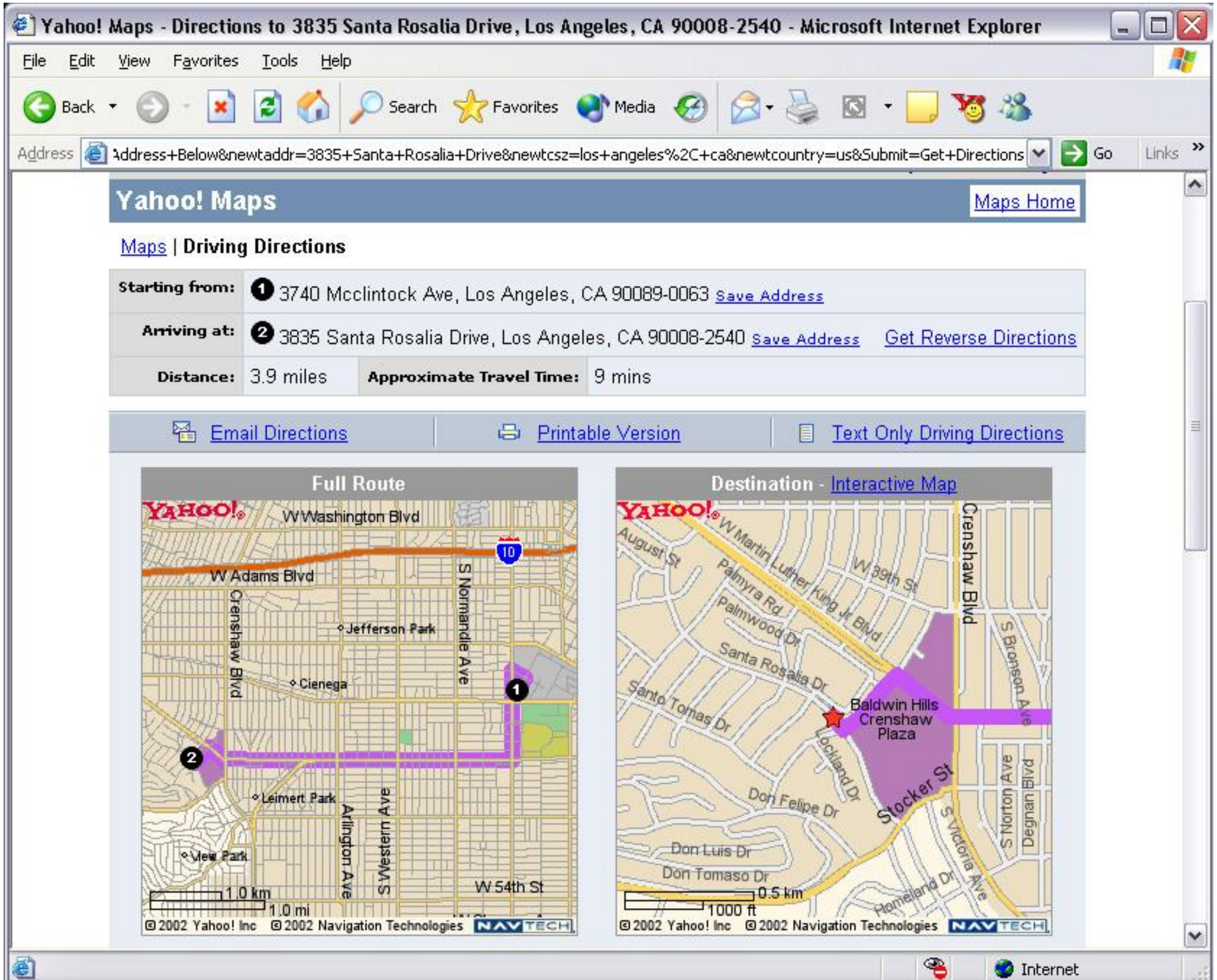
Example 2:

Finding the 3 closest restaurants to USC with Yahoo



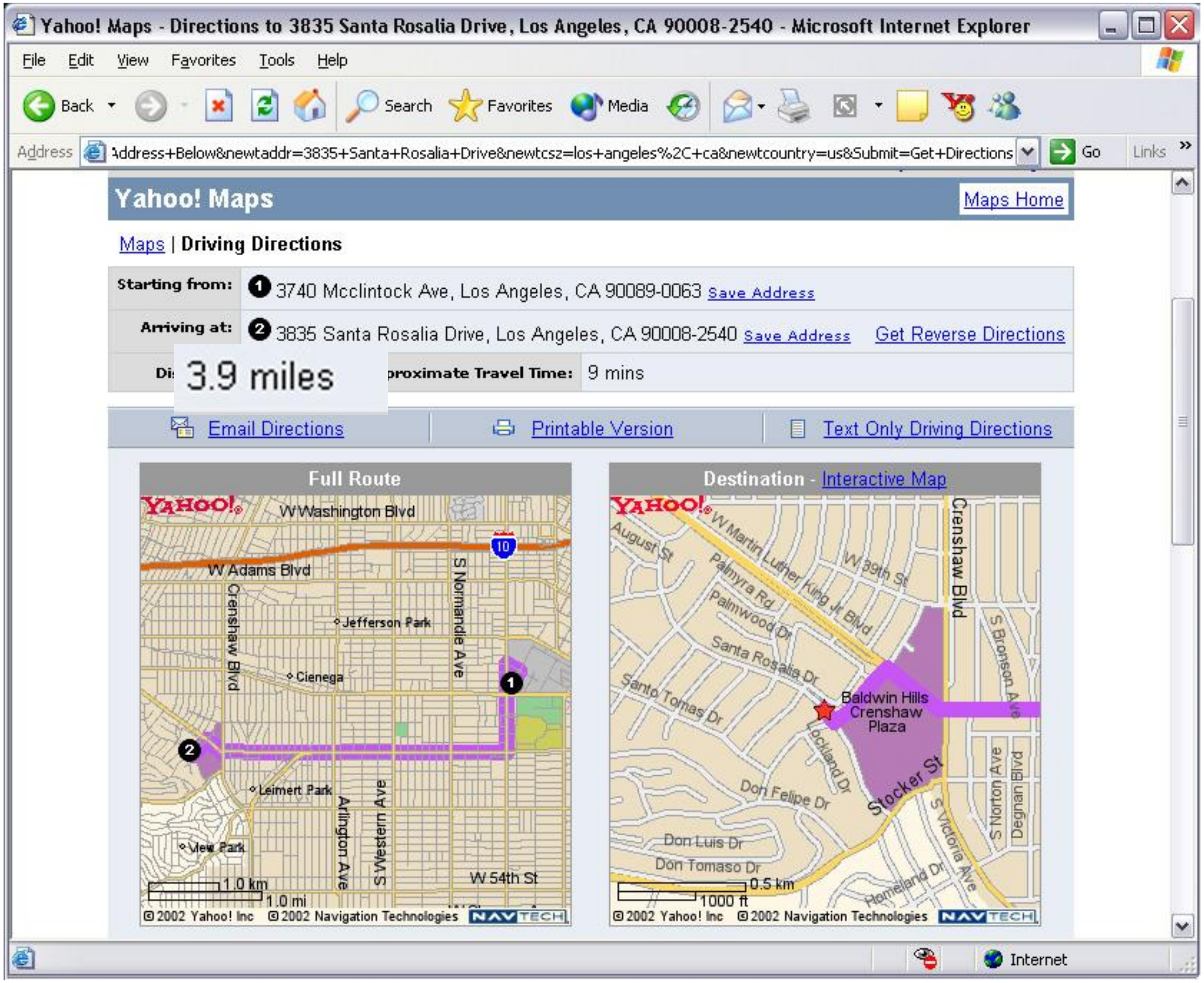
USC Viterbi School of Engineering

Example 2:
Finding the 3 closest restaurants to USC with Yahoo



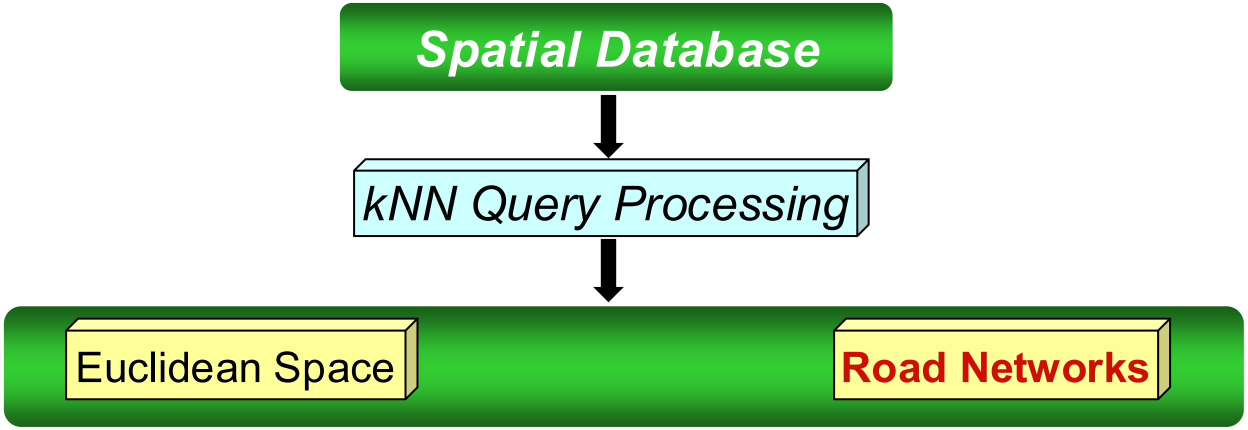
USC Viterbi School of Engineering

Example 2:
Finding the 3 closest restaurants to USC with Yahoo



USC Viterbi School of Engineering

Related Work



- ✓ **Road Network Embedding for K-NN Search: Shahabi et al., ACM-GIS 2002**
- ✓ **Query processing in SNDB : Papadias et al., VLDB 2003**
- ✓ **Voronoi-based kNN in SNDB: Kolahdouzan et al., VLDB 2004**

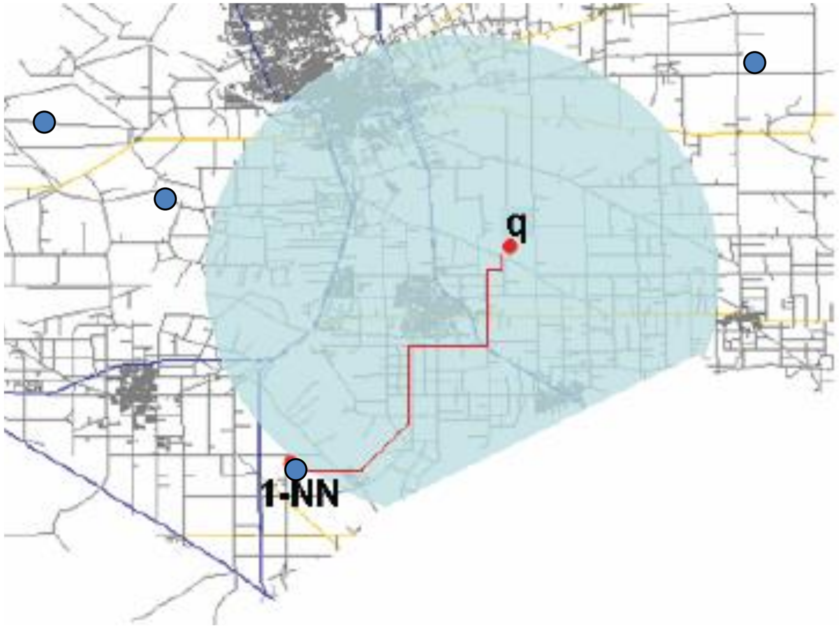
USC Viterbi School of Engineering

Related Work

Query processing in SNDB : Papadias et al., VLDB 2003

Incremental Network Expension (INE)

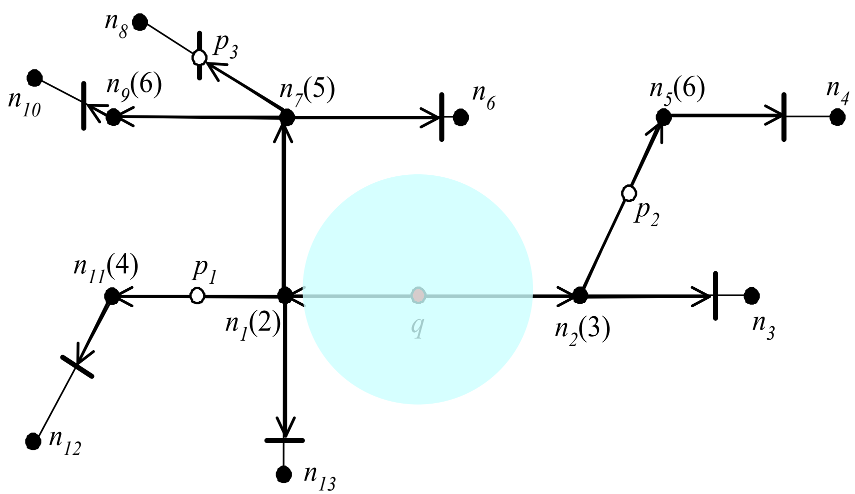
- Blind expansion hence redundant node access



Related Work

Query processing in SNDB : Papadias et al., VLDB 2003

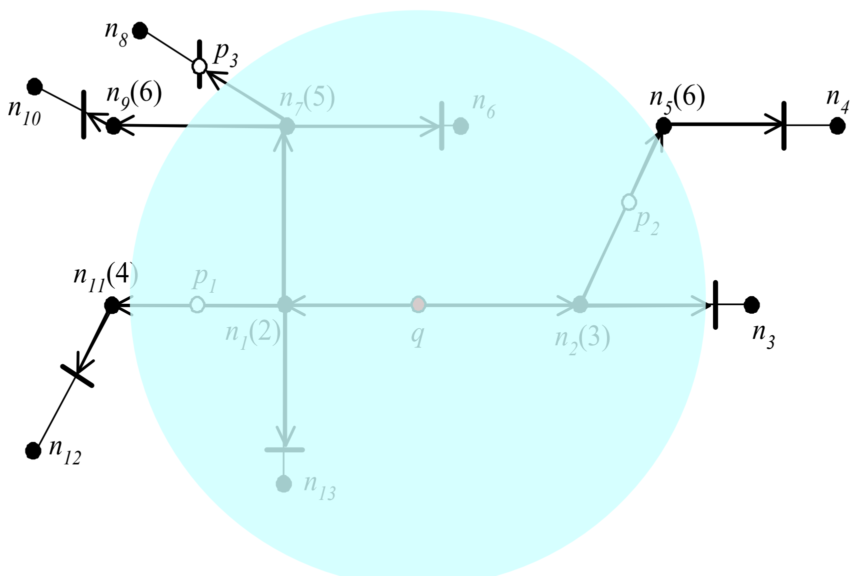
– Incremental Network Expansion



Related Work

Query processing in SNDB : Papadias et al., VLDB 2003

– Incremental Network Expansion

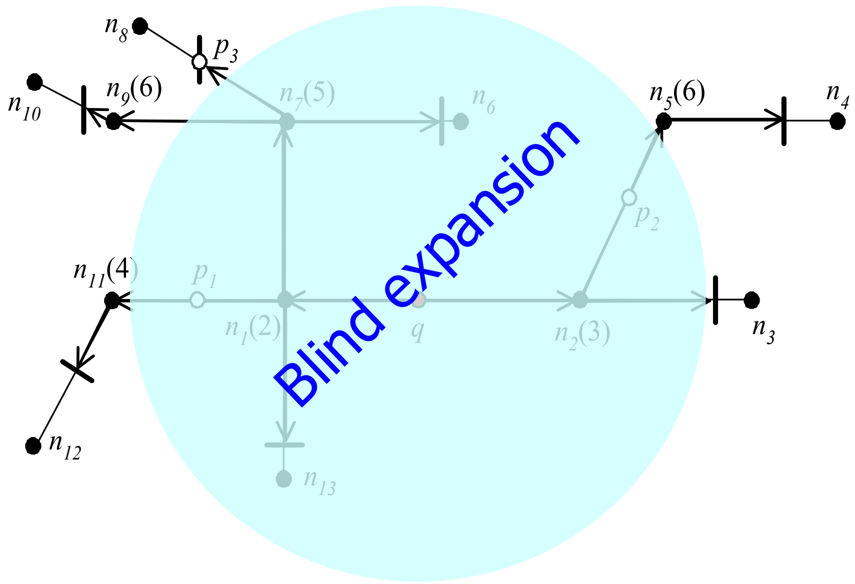


USC Viterbi School of Engineering

Related Work

Query processing in SNDB : Papadias et al., VLDB 2003

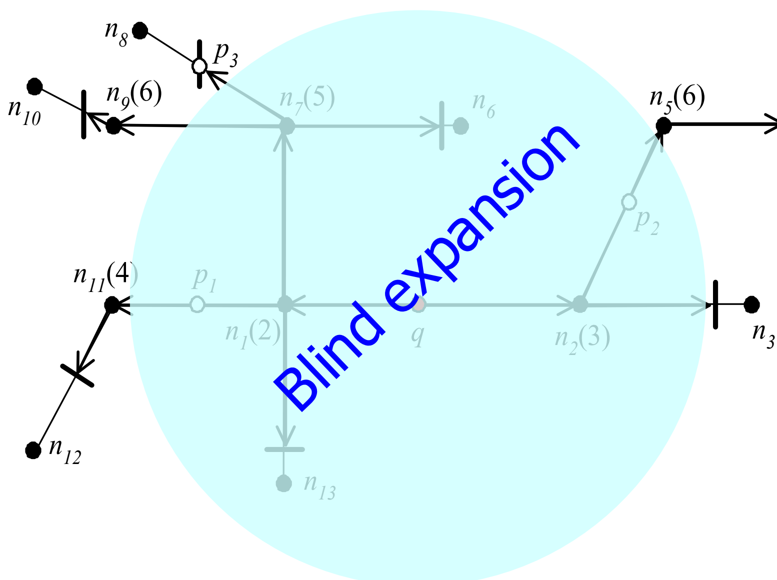
– Incremental Network Expansion

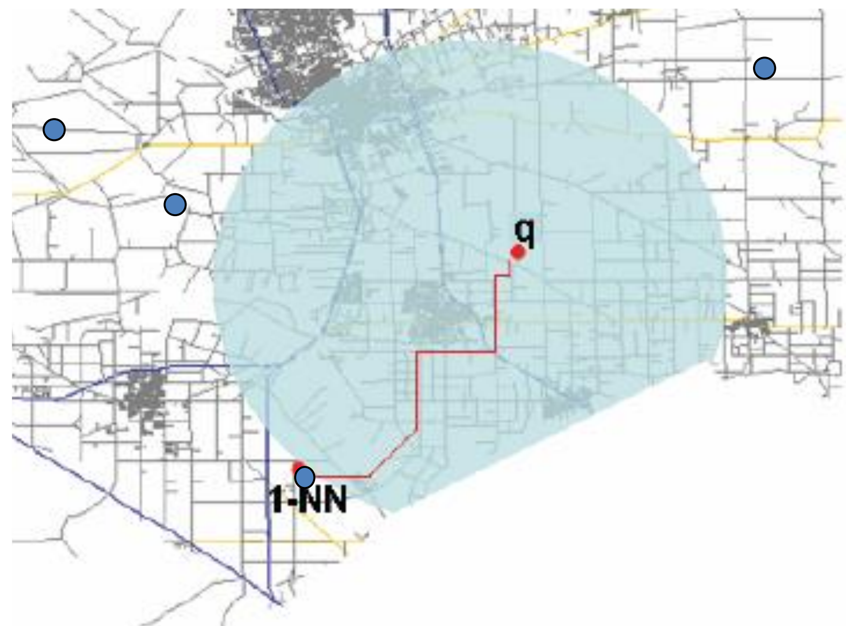


Related Work

Query processing in SNDB : Papadias et al., VLDB 2003

– Incremental Network Expansion



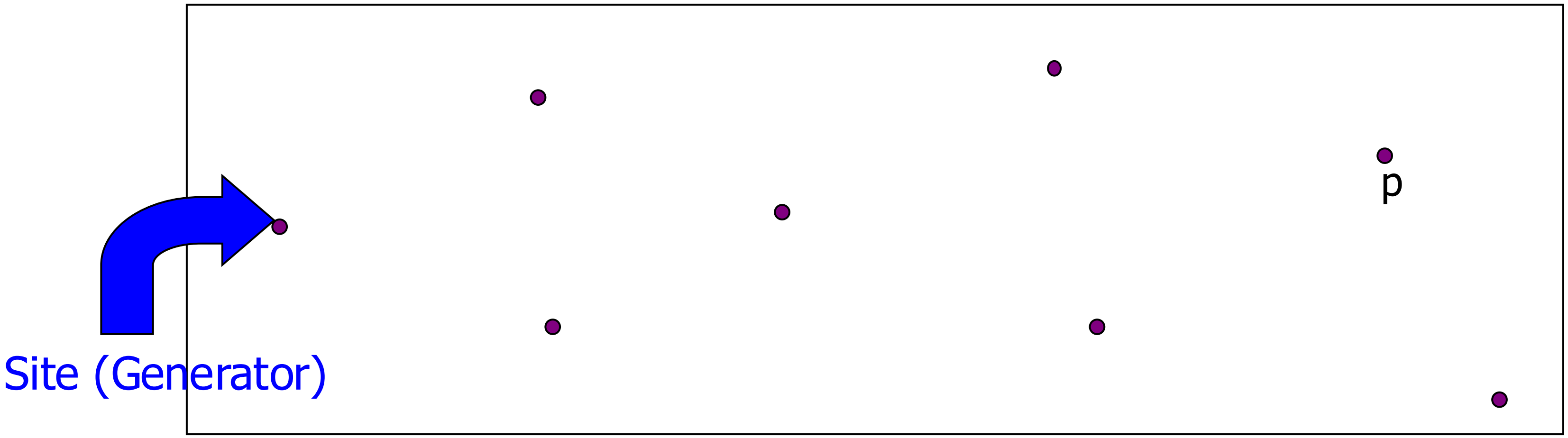


Preliminaries: Voronoi Diagram

- Given set of sites (POI), a Voronoi diagram partitions the plane into disjoint Voronoi polygons (cells), one for each site

Preliminaries: Voronoi Diagram

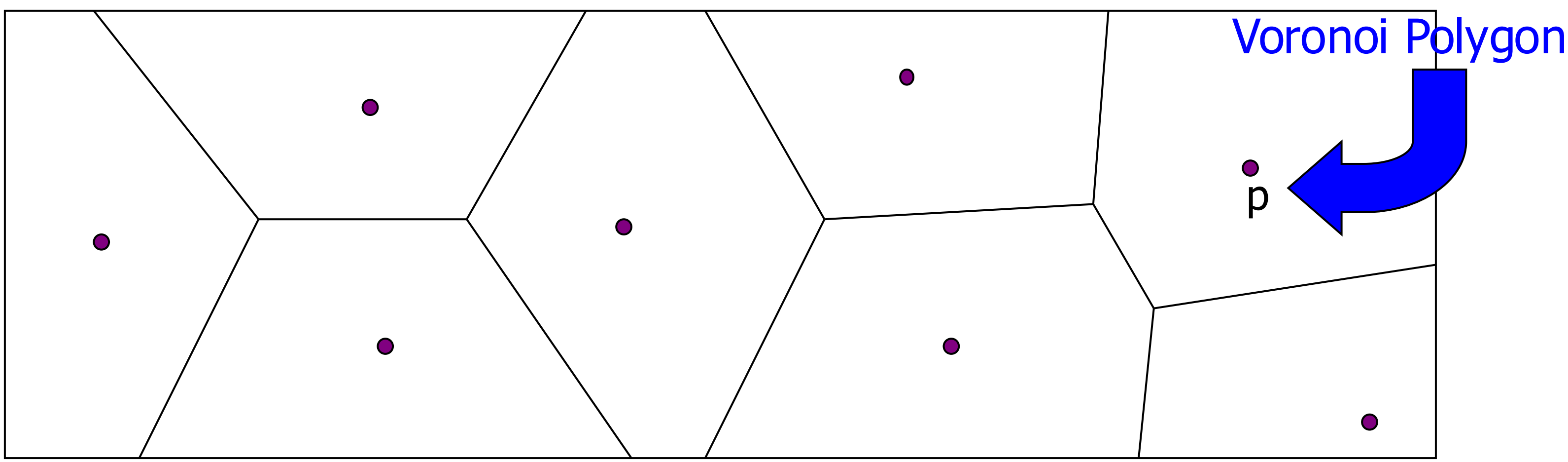
- Given set of sites (POI), a Voronoi diagram partitions the plane into disjoint Voronoi polygons (cells), one for each site



USC Viterbi
School of Engineering

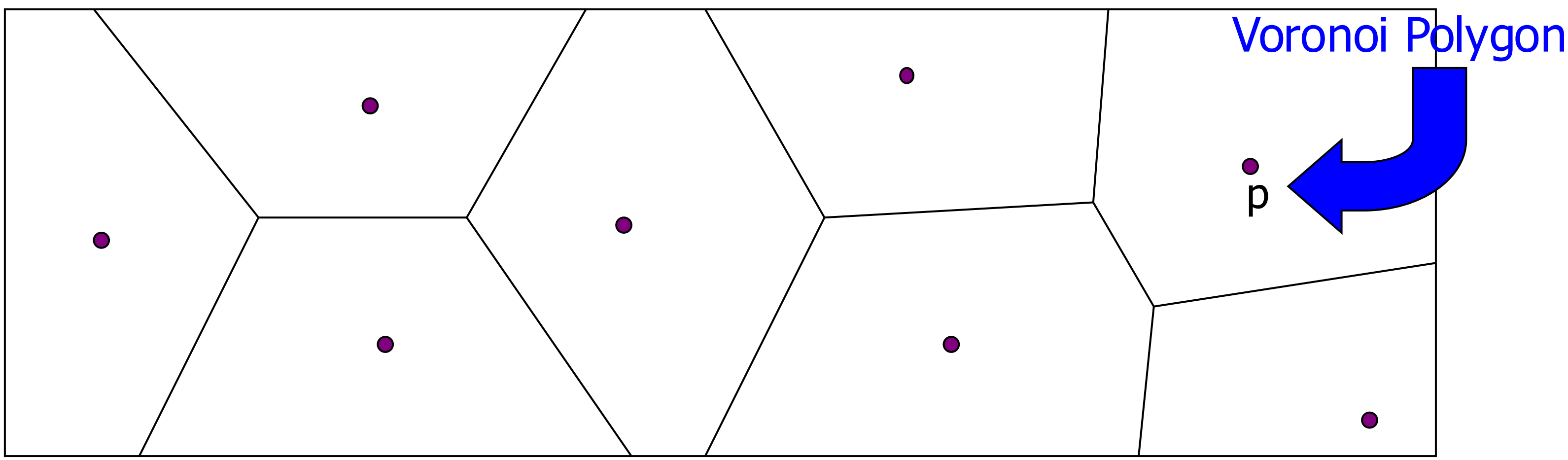
Preliminaries: Voronoi Diagram

- Given set of sites (POI), a Voronoi diagram partitions the plane into disjoint Voronoi polygons (cells), one for each site



Preliminaries: Voronoi Diagram

- Given set of sites (POI), a Voronoi diagram partitions the plane into disjoint Voronoi polygons (cells), one for each site
- The region including a site p includes all locations which are closer to p than to any other object p'.

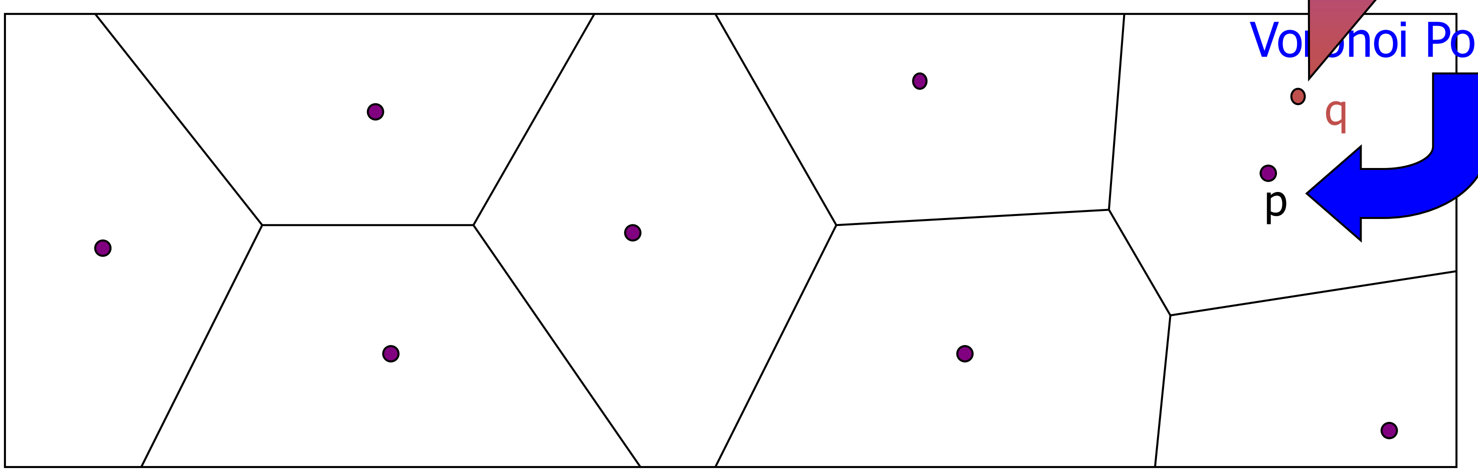


USC Viterbi
School of Engineering

Preliminaries: Voronoi Diagram

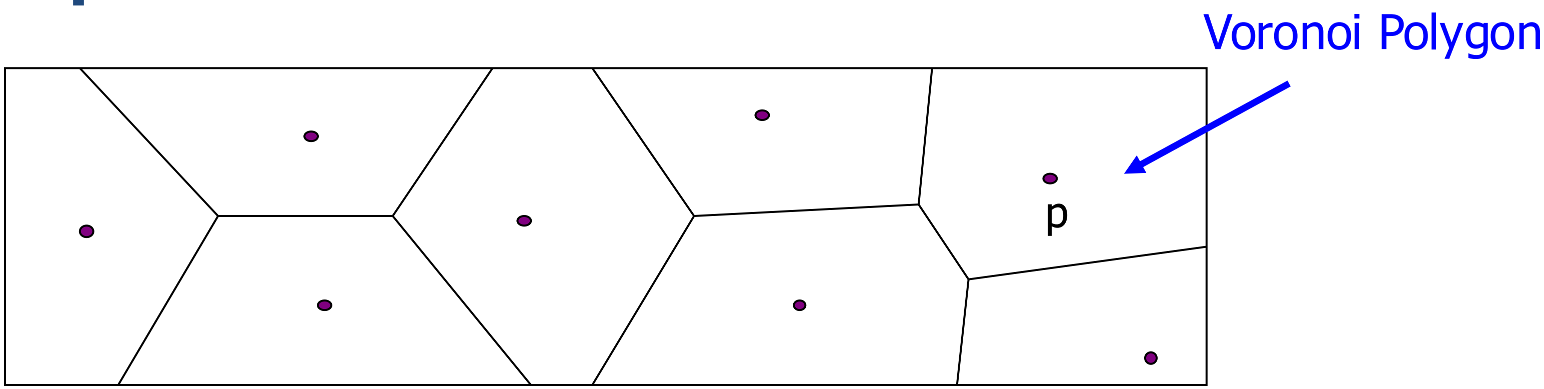
- Given set of sites (POI), a Voronoi diagram partitions the plane into disjoint Voronoi polygons (
site
- The region including a site p includes
are closer to p than to any other object p’.

q is closer to the generator of the Voronoi Polygon than any other generator



USC Viterbi School of Engineering

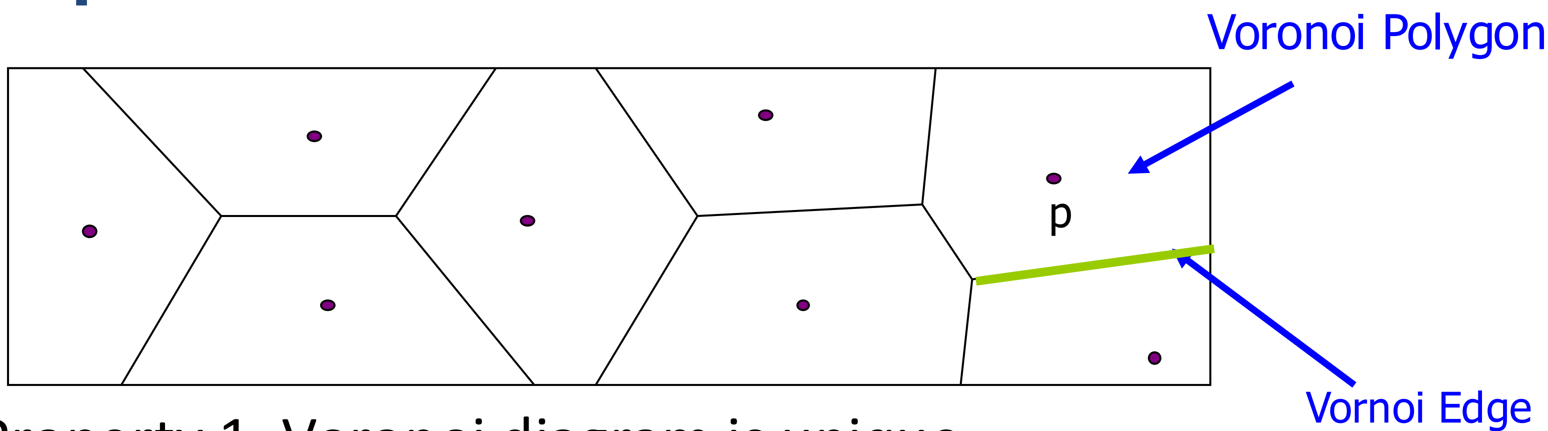
Preliminaries: Voronoi Properties



- Property 1: Voronoi diagram is unique

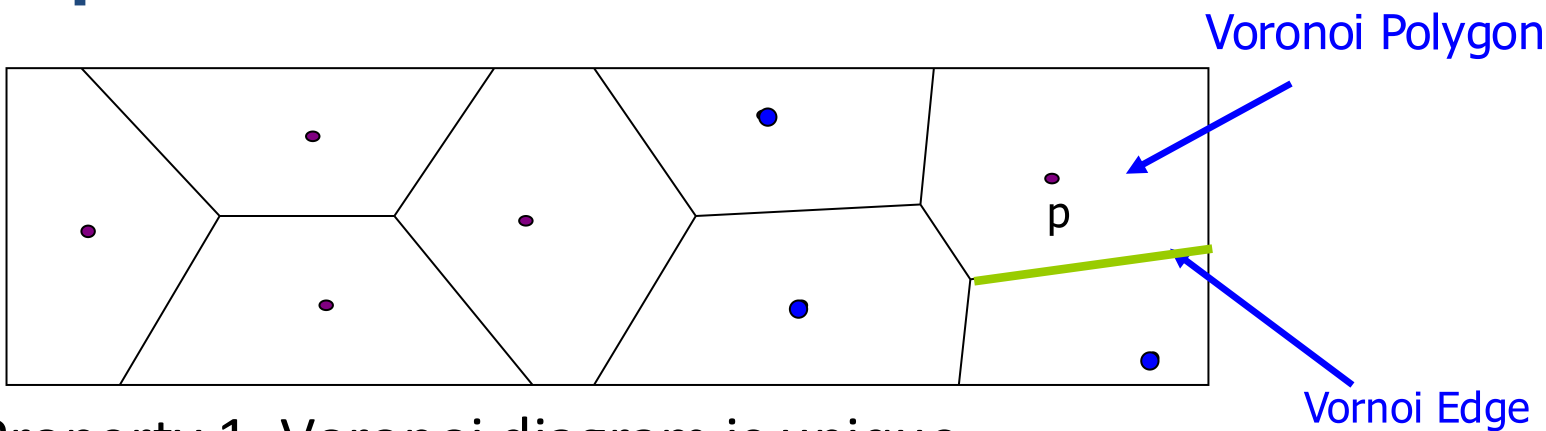
USC Viterbi
School of Engineering

Preliminaries: Voronoi Properties



- Property 1: Voronoi diagram is unique
- Property 2: Voronoi edges are
  - shared by two generators
  - in equal distance to neighboring generators

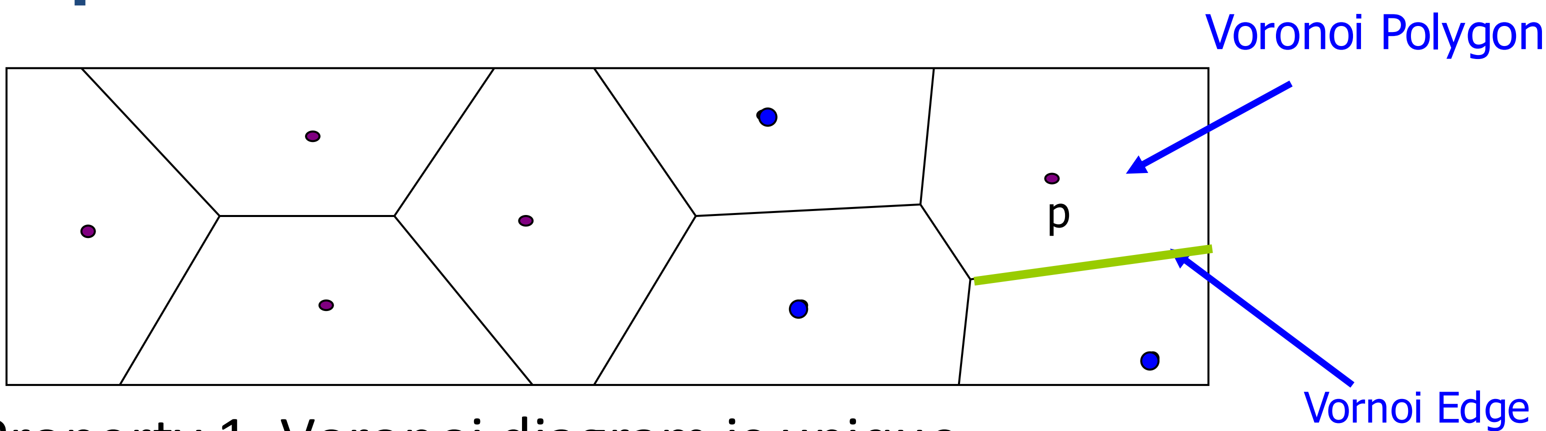
Preliminaries: Voronoi Properties



- Property 1: Voronoi diagram is unique
- Property 2: Voronoi edges are
  - shared by two generators
  - in equal distance to neighboring generators
- Property 3: Nearest generator of p is among the generators whose VP shares similar edges with p

USC Viterbi
School of Engineering

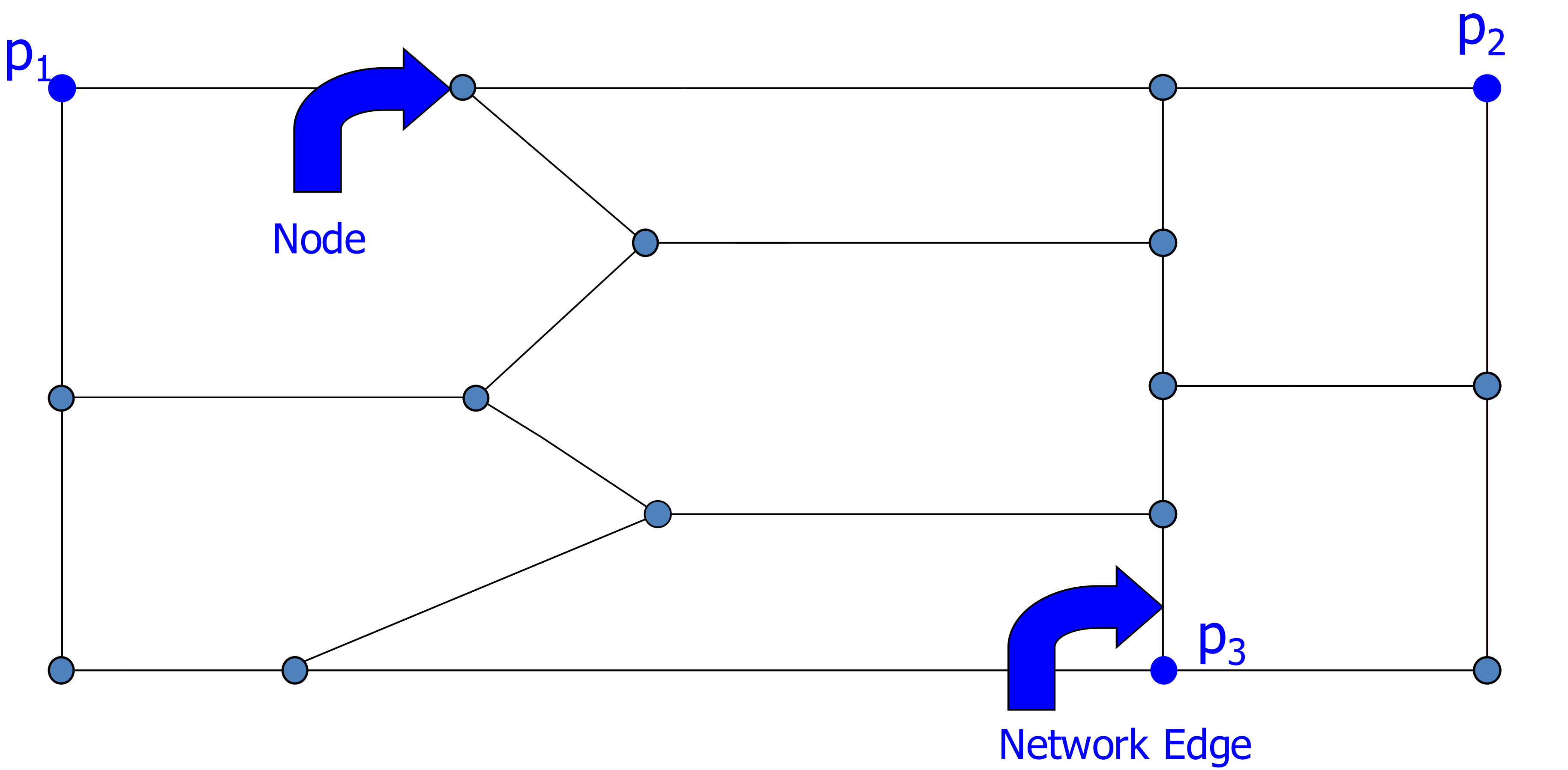
Preliminaries: Voronoi Properties



- Property 1: Voronoi diagram is unique
- Property 2: Voronoi edges are
  - shared by two generators
  - in equal distance to neighboring generators
- Property 3: Nearest generator of p is among the generators whose VP shares similar edges with p
- Property 4: Average number of Voronoi edges per VP is at most 6

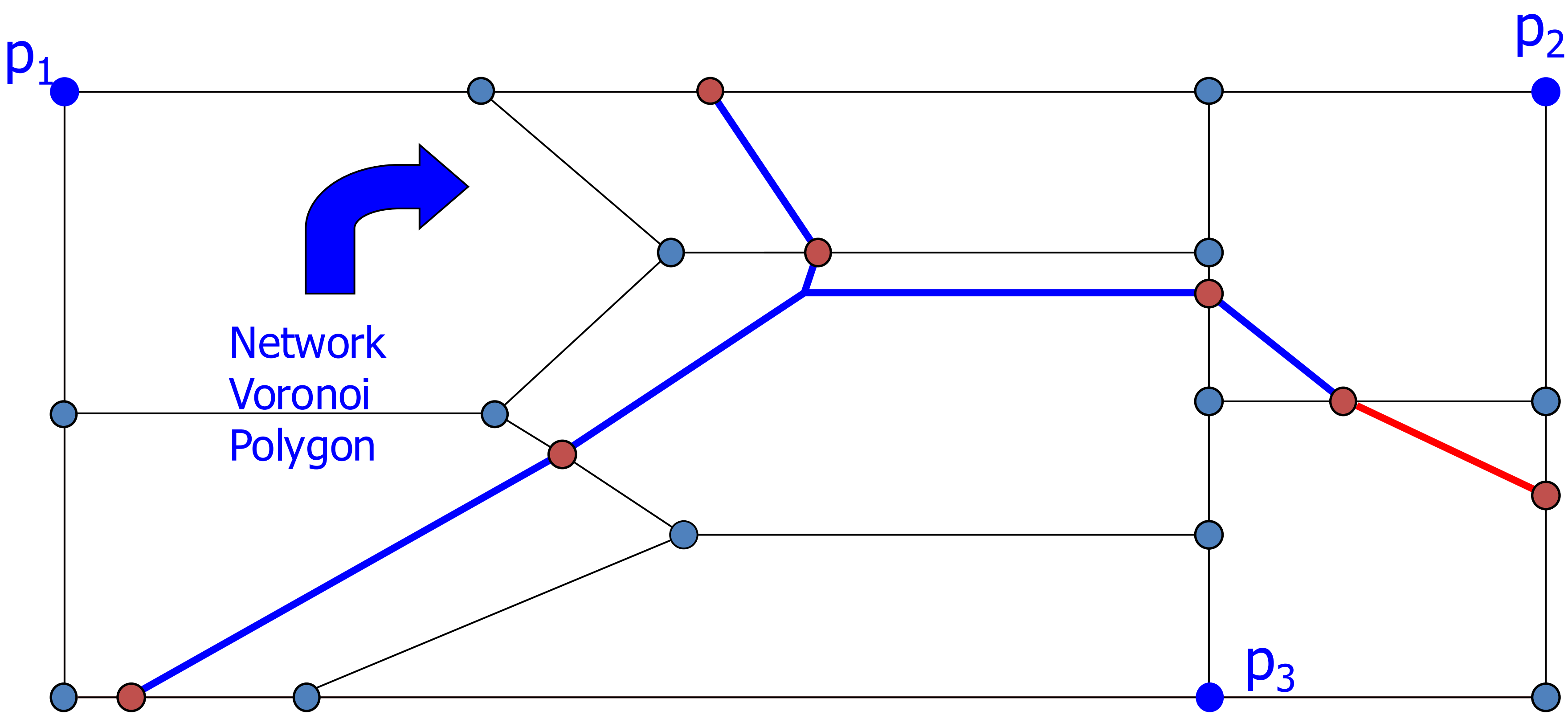
USC Viterbi School of Engineering

Network Voronoi Diagram



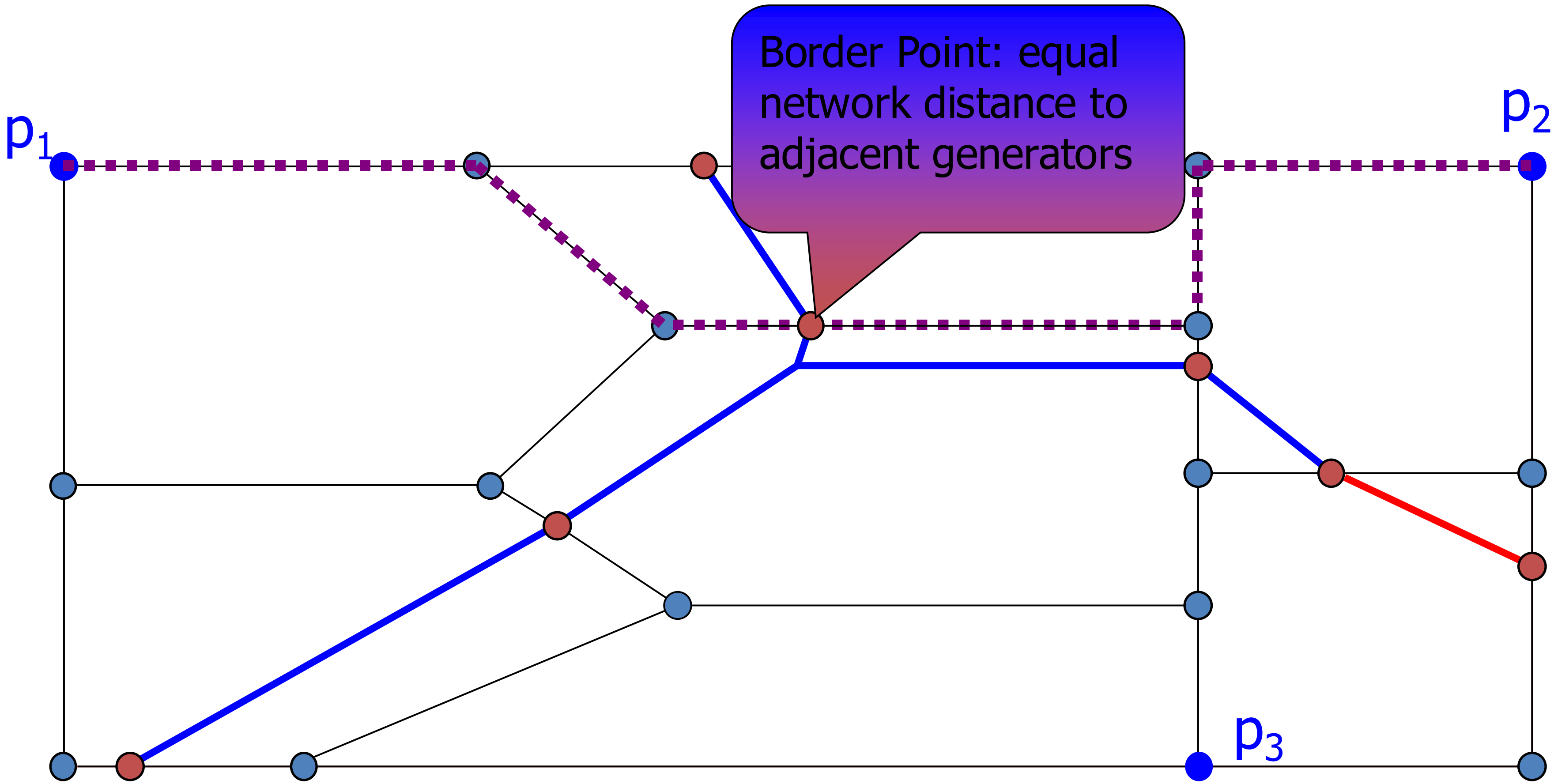
USC Viterbi School of Engineering

Network Voronoi Diagram



USC Viterbi School of Engineering

Network Voronoi Diagram



USC Viterbi School of Engineering

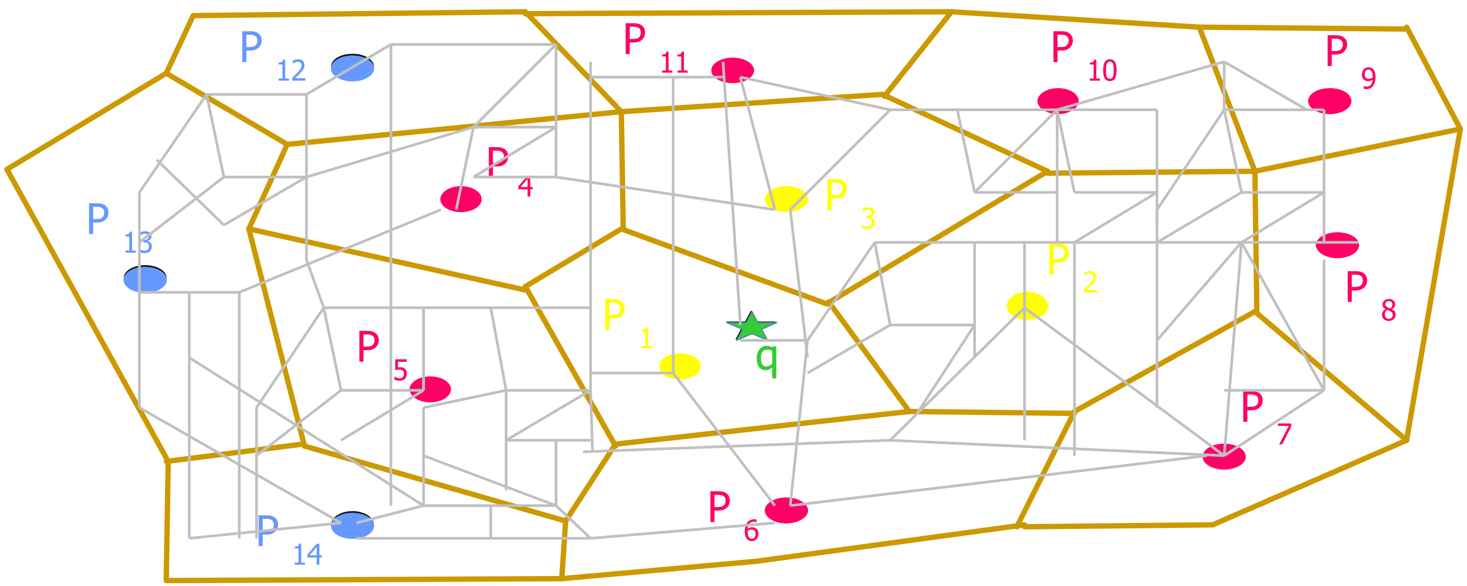
VN^3 Approach

- Offline Index Generation
  1. Network Voronoi Construction
  2. Index Generation (R-tree)
  3. Distance Precomputation
- Online Query Processing
  1. Find 1^{st} NN
  2. Find k NN-> Filter & Refine

Overview

USC Viterbi
School of Engineering

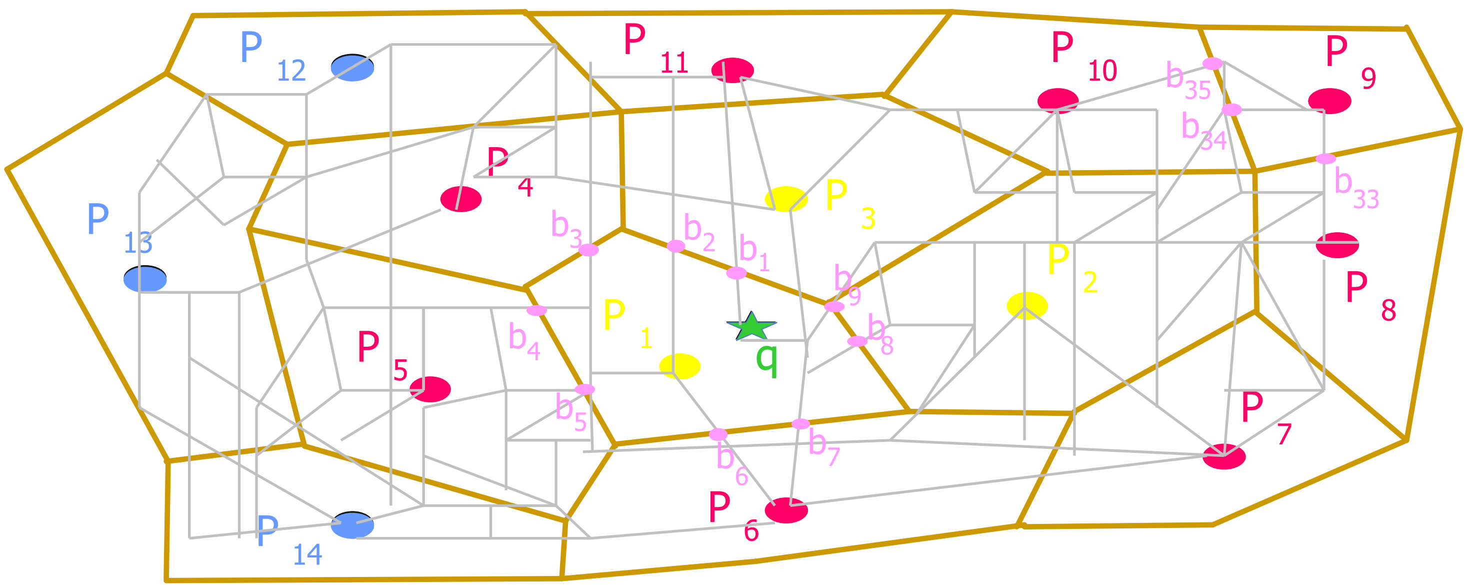
- How to compute distance from q to candidates?



USC Viterbi School of Engineering

Overview

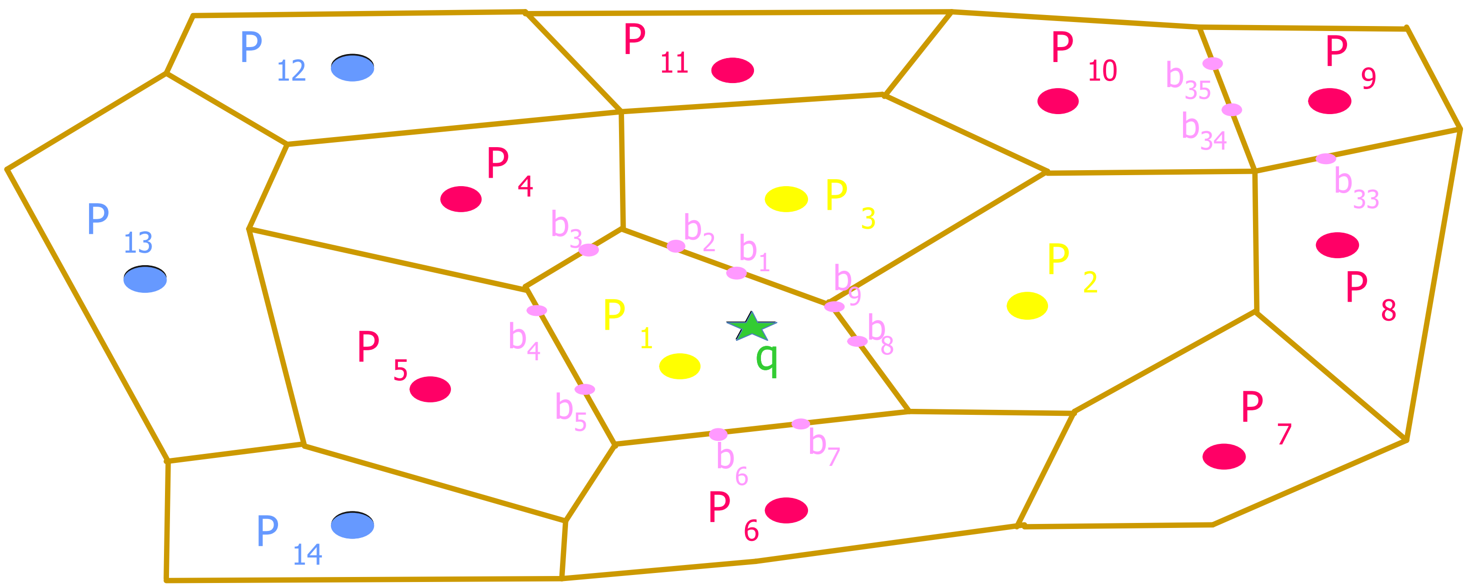
- How to compute distance from q to candidates?



USC Viterbi School of Engineering

Overview

- How to compute distance from q to candidates?

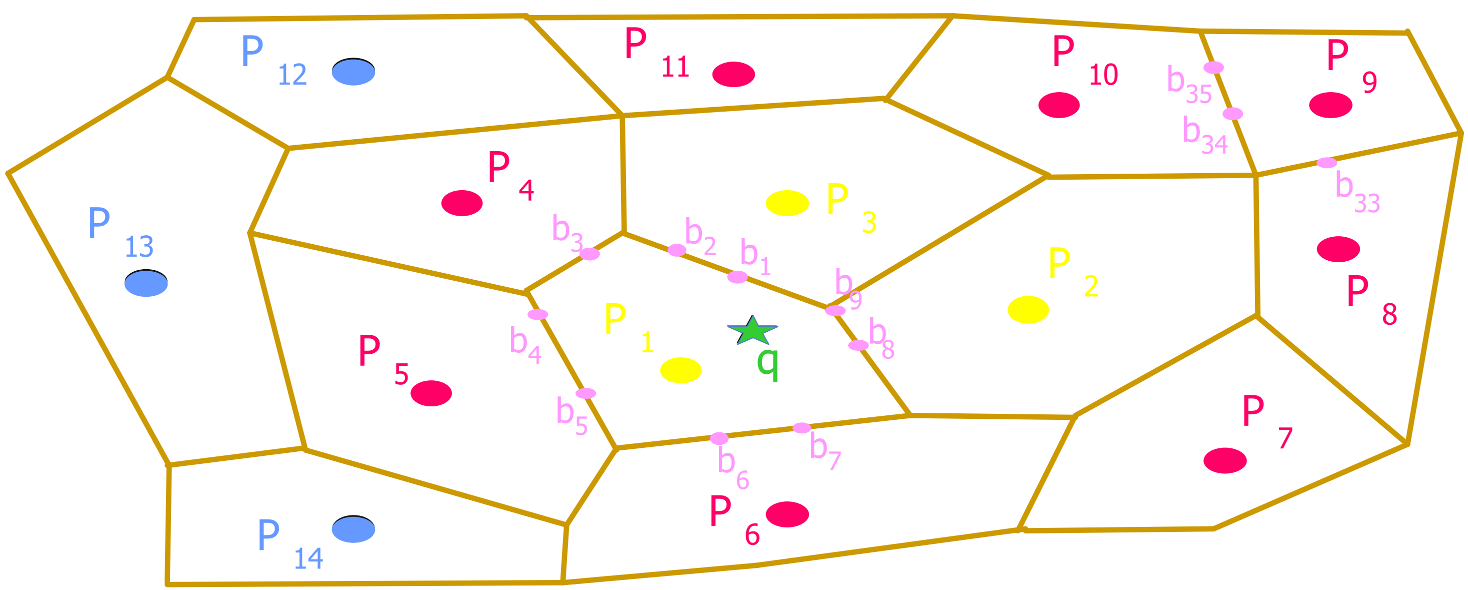


USC Viterbi School of Engineering

Overview

- How to compute distance from q to candidates?
- Break distance from q to any P_i to:

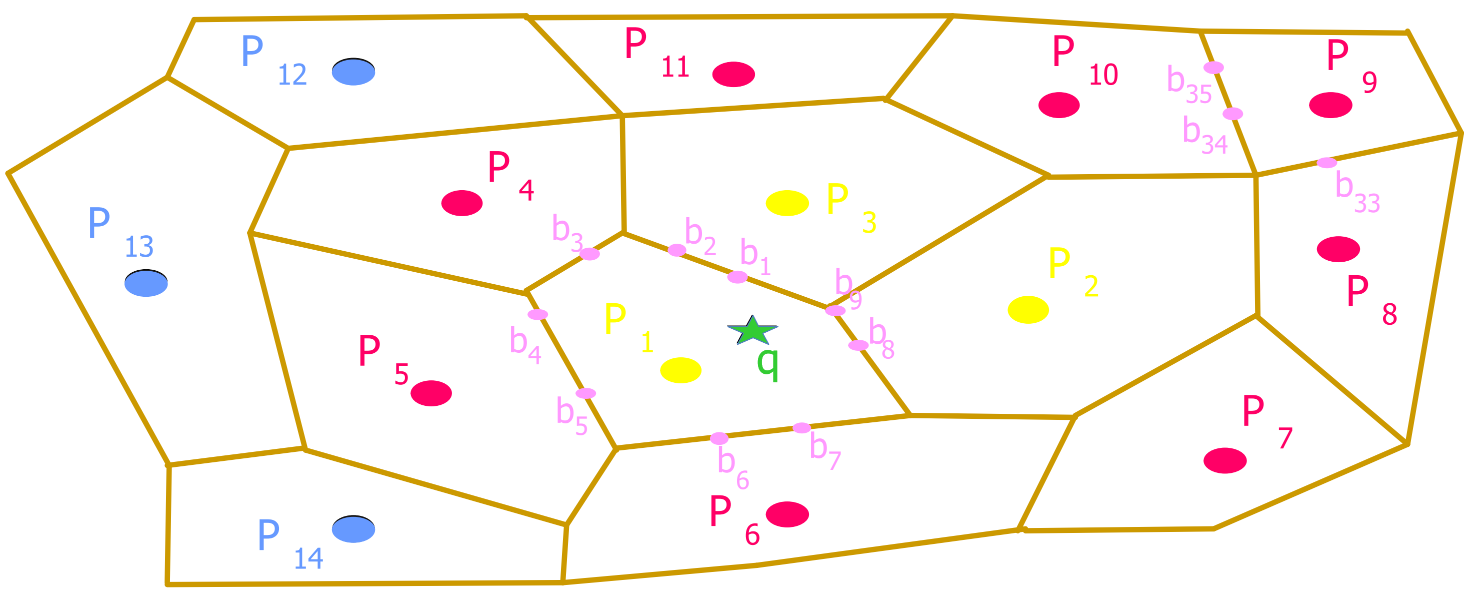
1. Distance from q to its borders
   I. Online Progressive Expansion
   II. Offline Pre-calculation



Overview

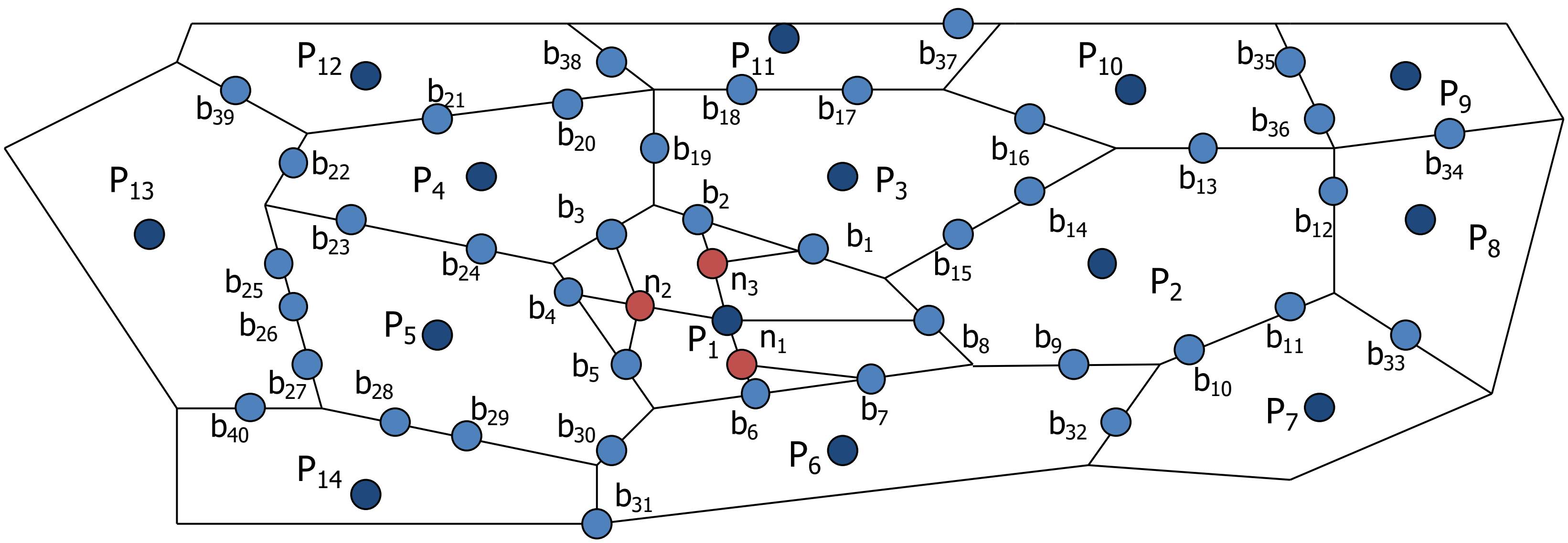
- How to compute distance from q to candidates?
- Break distance from q to any P_i to:

1. Distance from q to its borders
   I. Online Progressive Expansion
   II. Offline Pre-calculation
2. Distance from borders of q to borders of P_i
   I. NVP Expansion
   II. Distance Computing Optimization



Offline Step

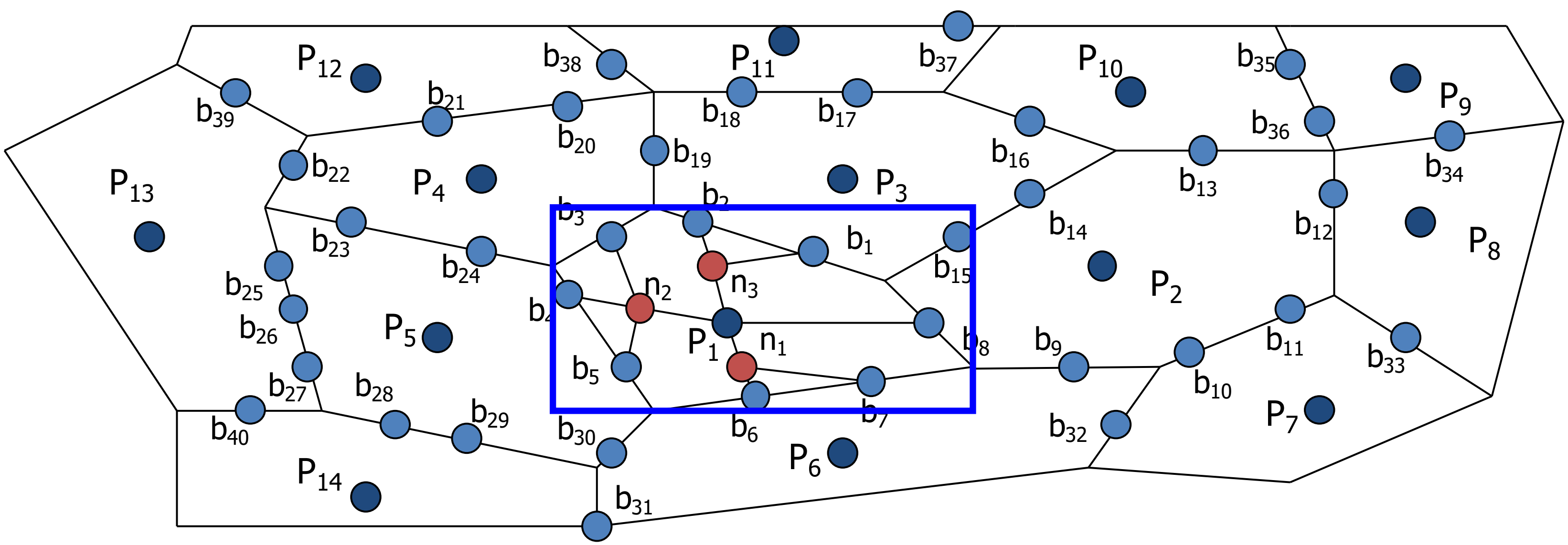
- Compute Network Voronoi Polygons (NVP)



USC Viterbi School of Engineering

Offline Step

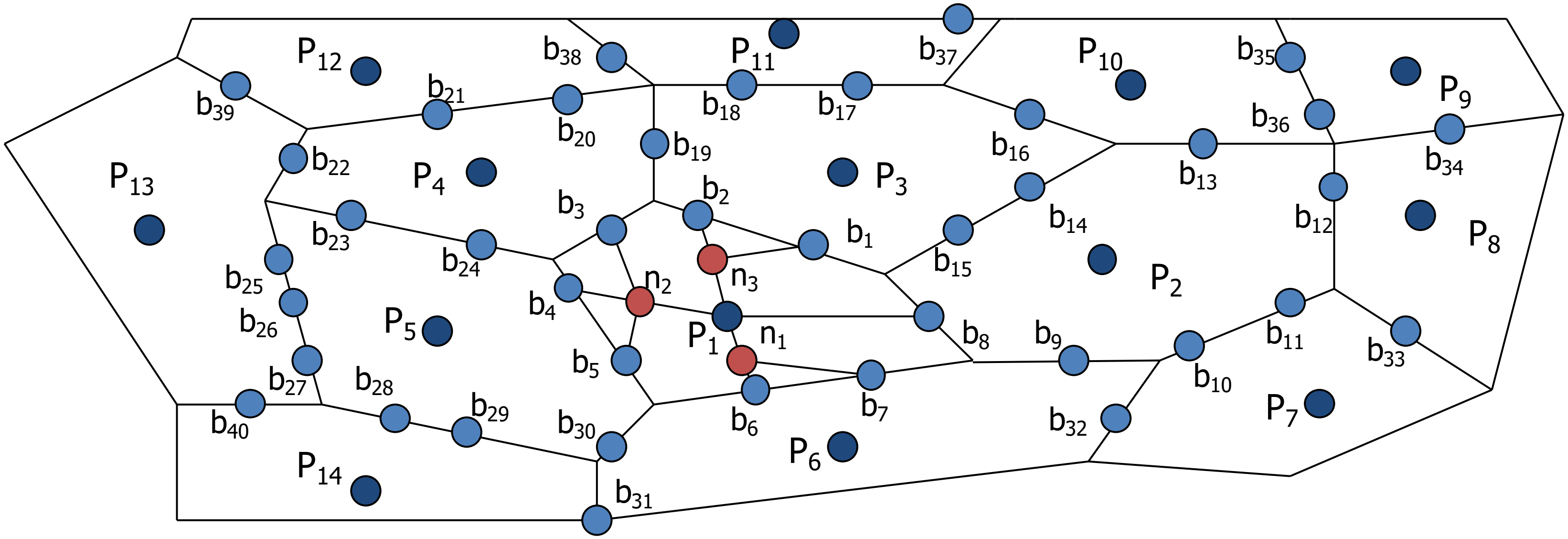
- Compute Network Voronoi Polygons (NVP)
- Index NVPs with R-Tree



USC Viterbi School of Engineering

Offline Step

- Compute Network Voronoi Polygons (NVP)
- Index NVPs with R-Tree
- Precompute the inter (accross polygons) and intra (within polygons) network distances for each NVP



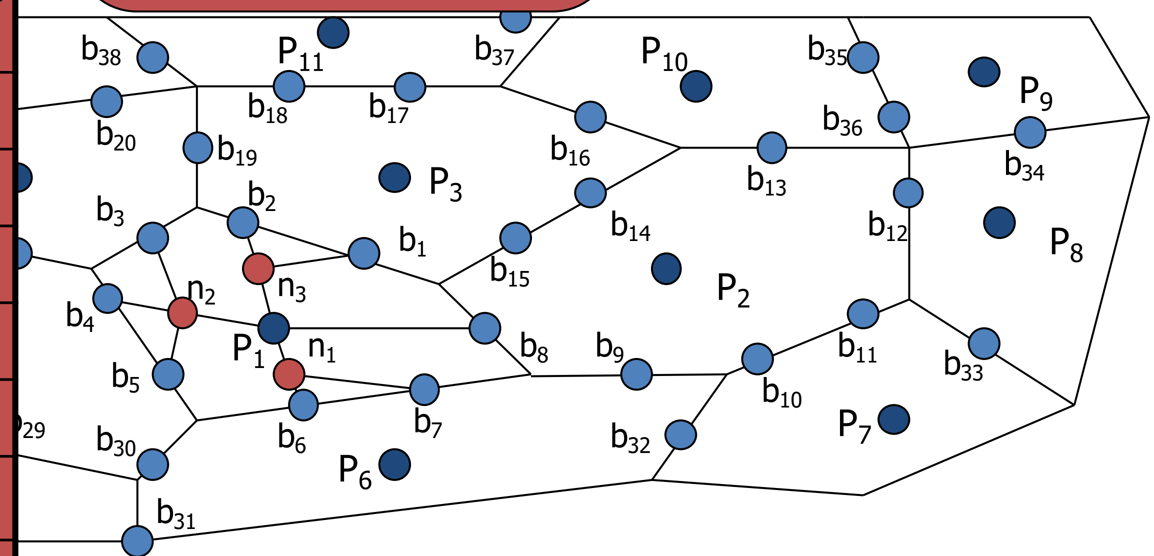
USC Viterbi
School of Engineering

Offline Step

- Compute Network Voronoi Polygons (NVP)
- Index NVPs with
- Precompute the intra (within polygons) network distance

1. Precompute the Shortest Path distance from each generator to its border point
2. Precompute the Shortest Path between border points

b_{34}	P_9	$d_n (b_{34}, P_9)$
b_{35}	P_9	$d_n (b_{35}, P_9)$
b_{36}	P_9	$d_n (b_{36}, P_9)$
…	…	……
b_1	b_2	$d_n (b_1, b_2)$
b_1	b_{15}	$d_n (b_1, b_{15})$
b_1	b_{13}	$d_n (b_1, b_{13})$
…	…	……



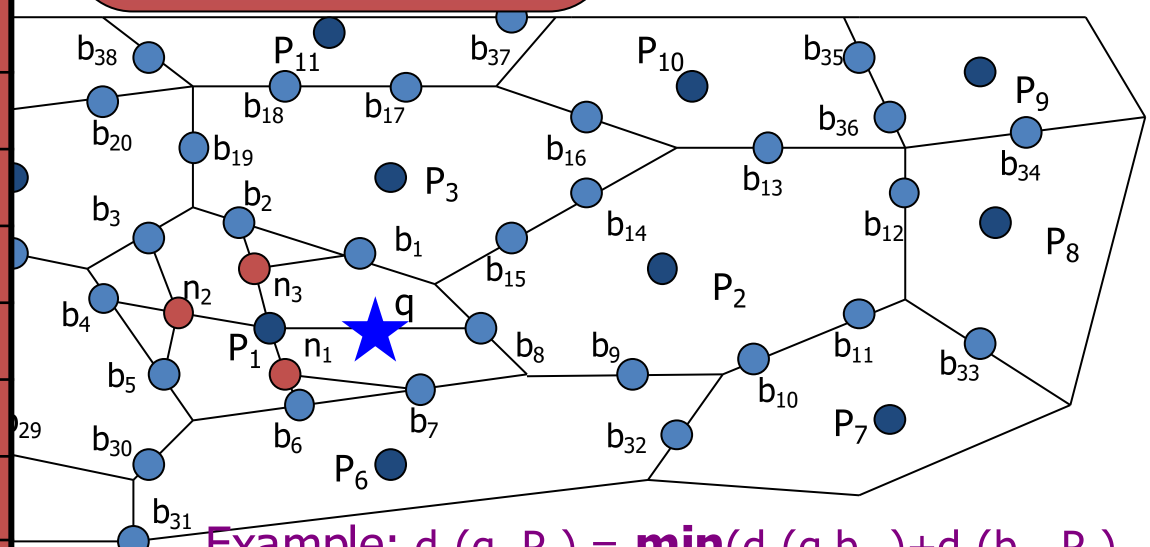
USC Viterbi School of Engineering

Offline Step

- Compute Network Voronoi Polygons (NVP)
- Index NVPs with
- Precompute the … intra (within polygons) network distance

1. Precompute the Shortest Path distance from each generator to its border point
2. Precompute the Shortest Path between border points

b_{34}	P_9	$d_n(b_{34}, P_9)$
b_{35}	P_9	$d_n(b_{35}, P_9)$
b_{36}	P_9	$d_n(b_{36}, P_9)$
…	…	……
b_1	b_2	$d_n(b_1, b_2)$
b_1	b_{15}	$d_n(b_1, b_{15})$
b_1	b_{13}	$d_n(b_1, b_{13})$
…	…	……



Example: $d_n(q, P_9) = \mathbf{min}(d_n(q,b_{34})+d_n(b_{34},P_9), d_n(q,b_{35})+d_n(b_{35},P_9), d_n(q,b_{36})+dn\ (b_{36},P_9))$

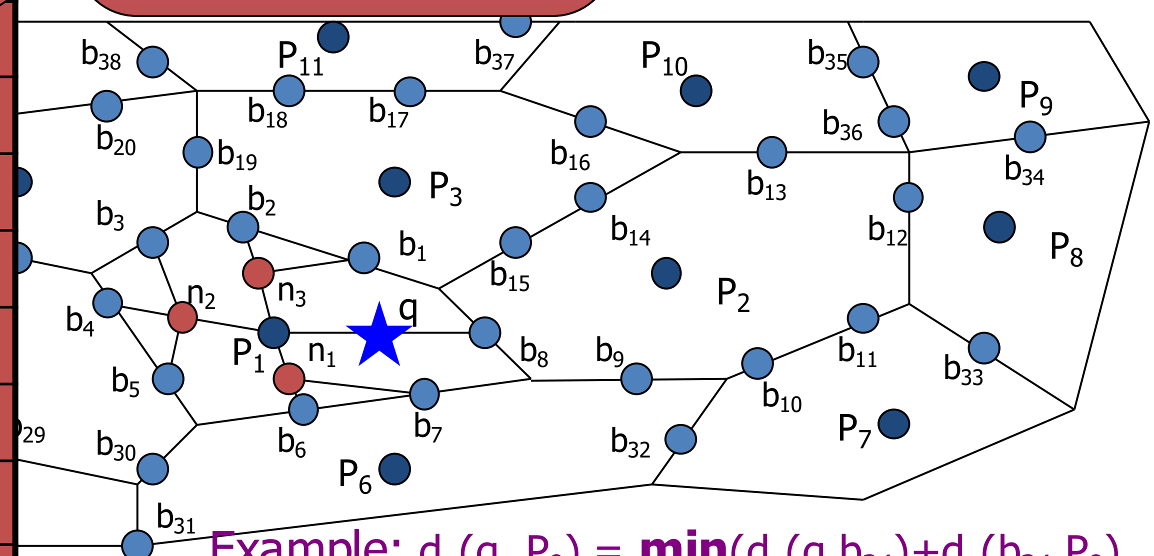
USC Viterbi School of Engineering

Offline Step

- Compute Network Voronoi Polygons (NVP)
- Index NVPs with
- Precompute the ... intra (within polygons) network distance

1. Precompute the Shortest Path distance from each generator to its border point
2. Precompute the Shortest Path between border points

b_{34}	P_9	$d_n(b_{34}, P_9)$
b_{35}	P_9	$d_n(b_{35}, P_9)$
b_{36}	P_9	$d_n(b_{36}, P_9)$
…	…	……
b_1	b_2	$d_n(b_1, b_2)$
b_1	b_{15}	$d_n(b_1, b_{15})$
b_1	b_{13}	$d_n(b_1, b_{13})$
…	…	……



Example: $d_n(q, P_9) = \textbf{min}(d_n(q,b_{34})+d_n(b_{34},P_9), d_n(q,b_{35})+d_n(b_{35},P_9), d_n(q,b_{36})+dn\ (b_{36},P_9))$

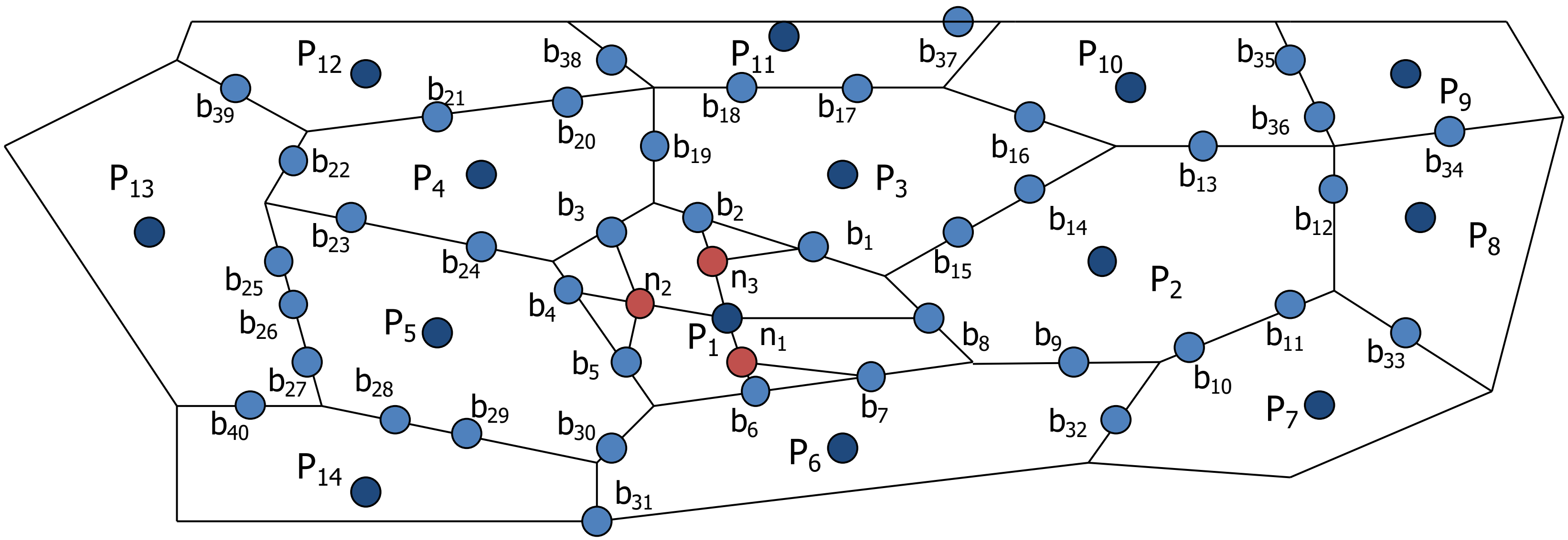
$d_n(q,b_{34}) = \min\ (d_n(q,b_1)+d_n(b_1,b_{34}), d_n(q,b_2)+ d_n(b_2,b_{34}),\ldots\ ,d_n(q,b_8)+d_n(b_8,b_{34}))$

USC Viterbi School of Engineering

Online Step

- Find 1st NN
  - Search R-Tree to find the NVP that overlaps q
  - Report the generator of the NVP as the 1st NN;

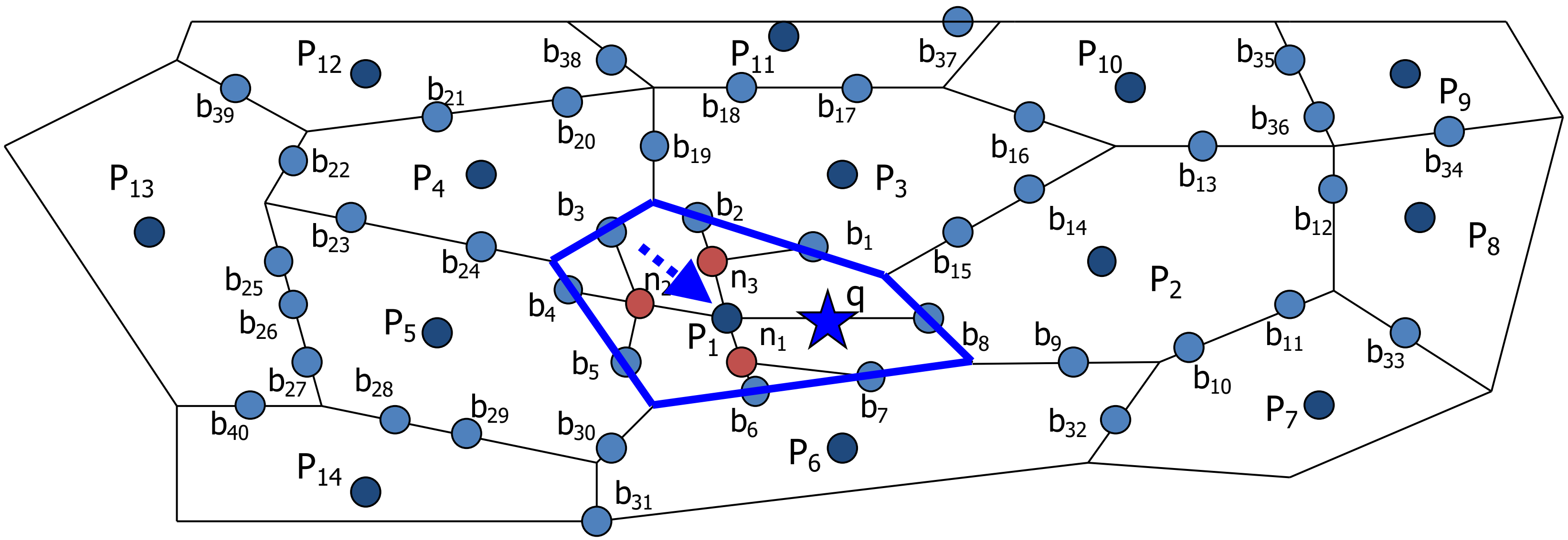
    Cost: **O(logn)**



Online Step

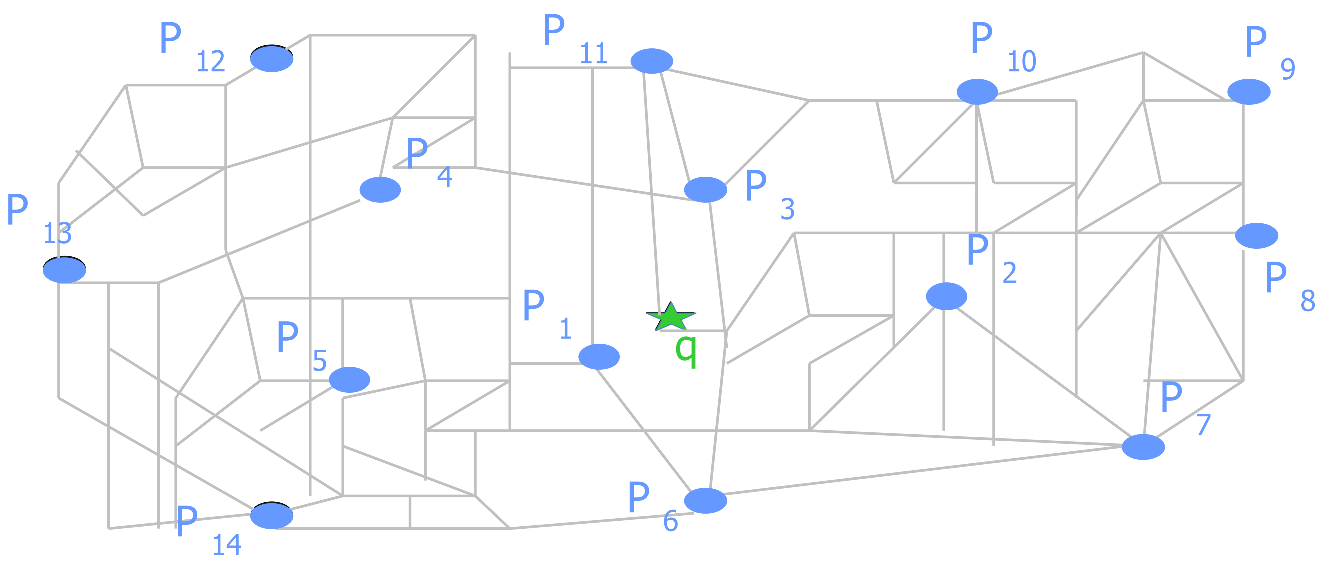
- Find 1st NN
  - Search R-Tree to find the NVP that overlaps q
  - Report the generator of the NVP as the 1st NN;

    Cost: **O(logn)**

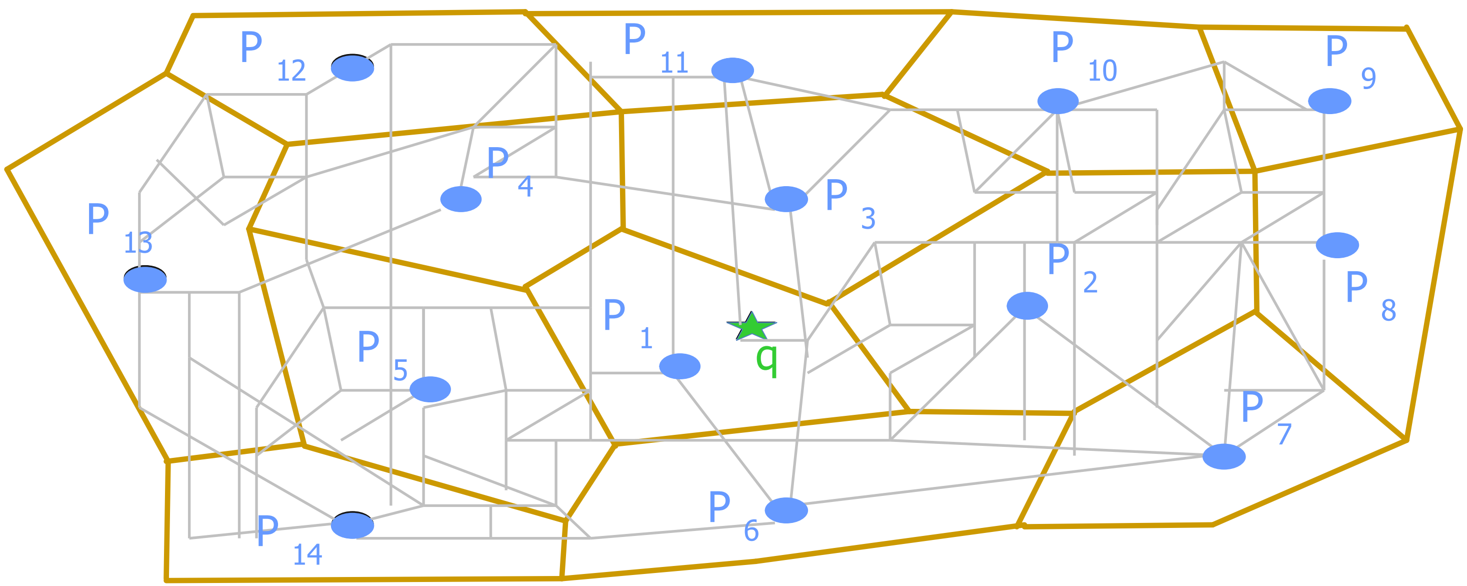


USC Viterbi School of Engineering

Online Step

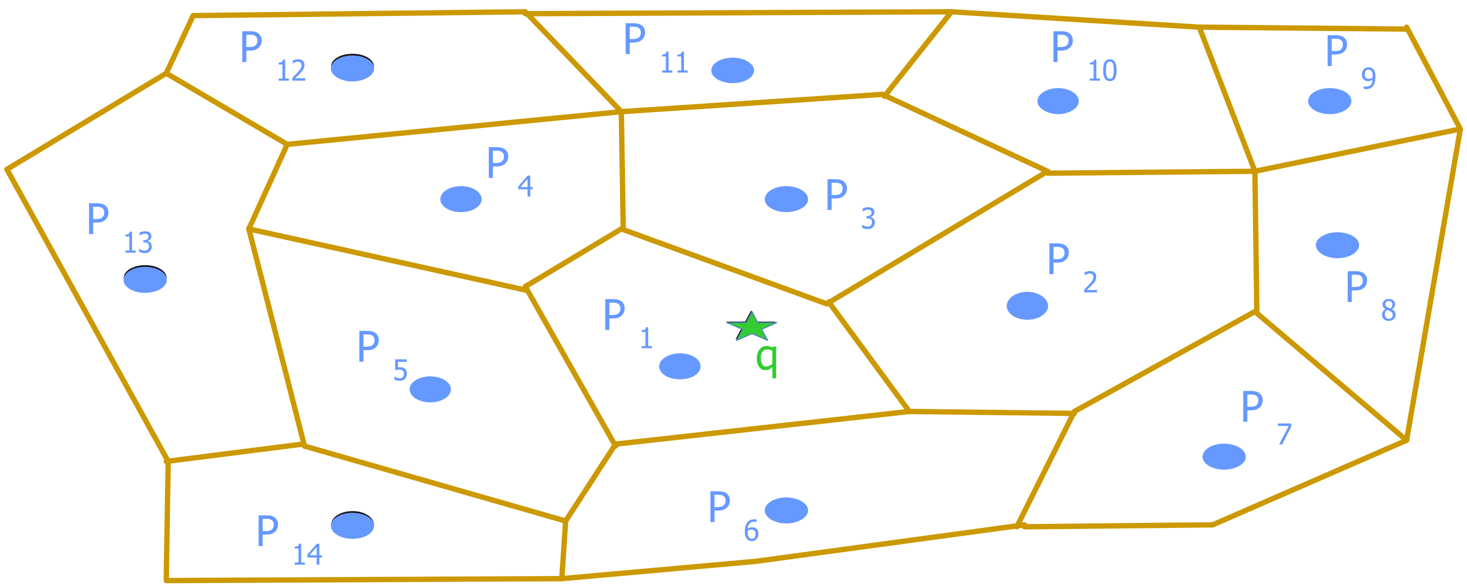


Online Step



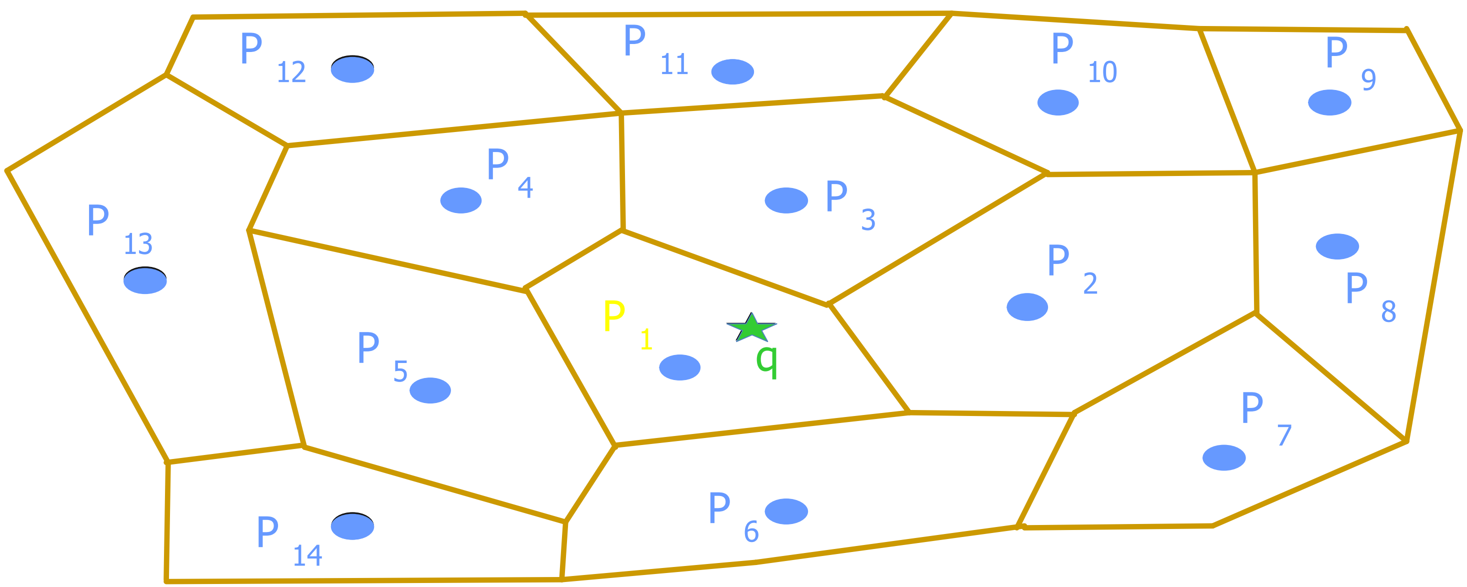
Online Step

USC Viterbi
School of Engineering



Online Step

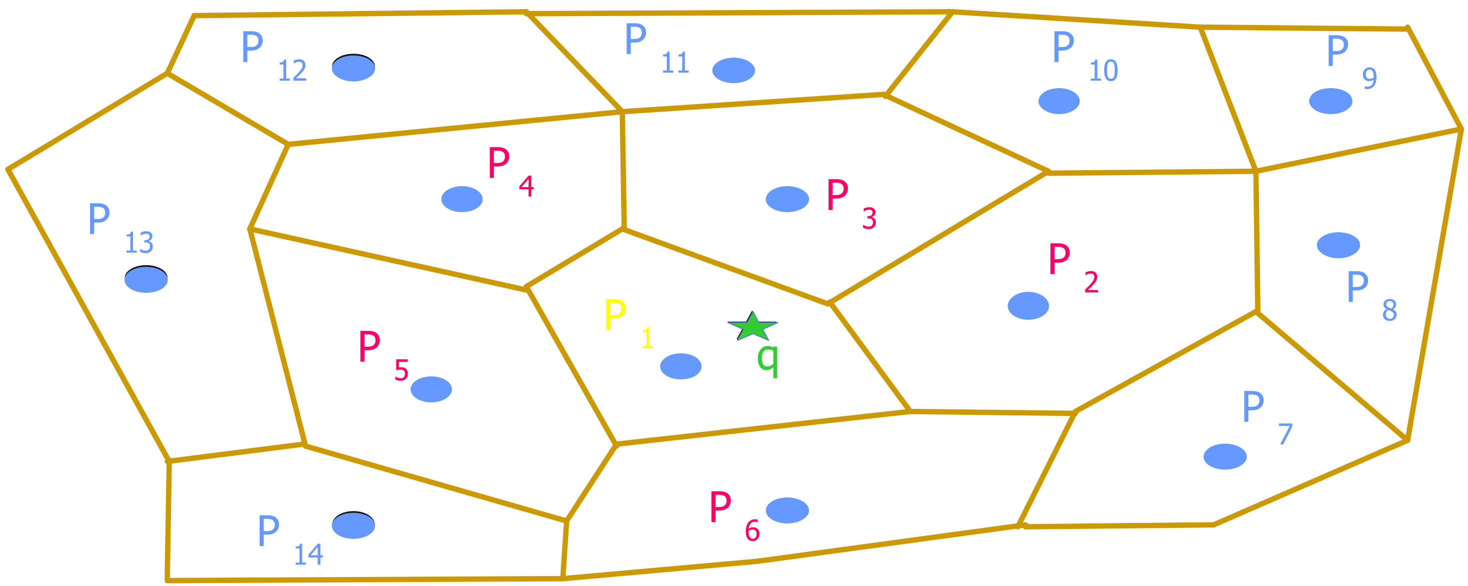
- 1^{st} NN = P_1



USC Viterbi School of Engineering

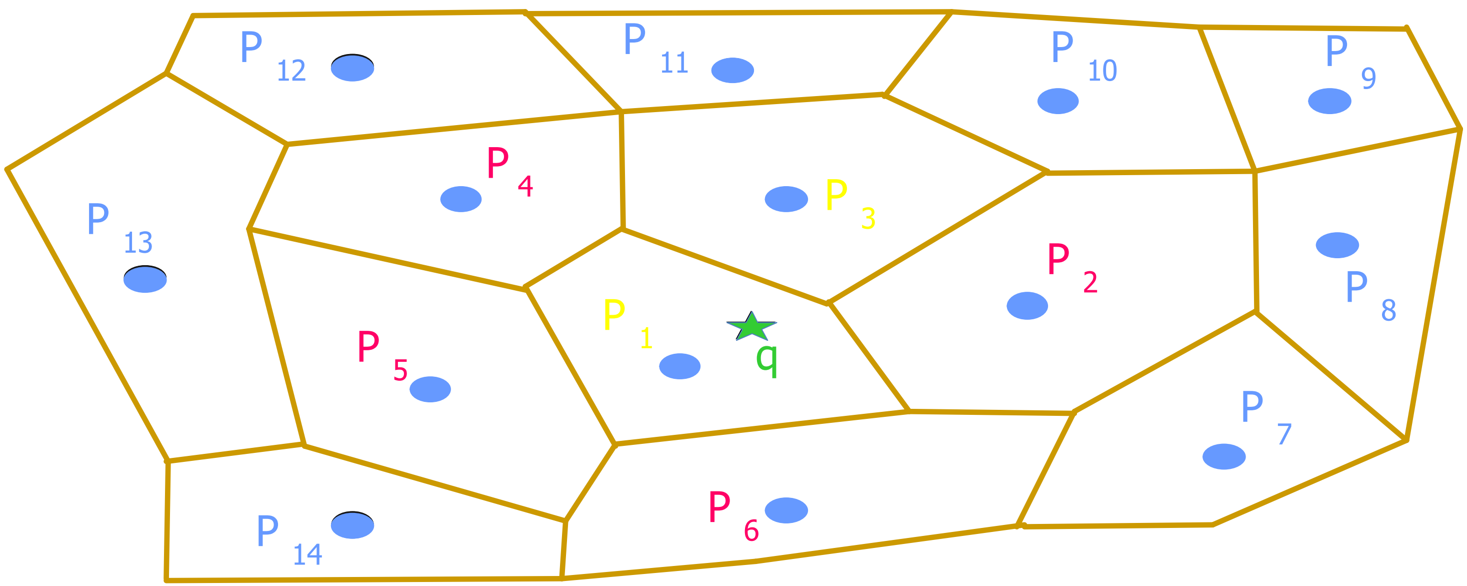
Online Step

- 1^{st} NN = P_1
- 2^{nd} NN $\in$ { Neighbors(P_1) }



Online Step

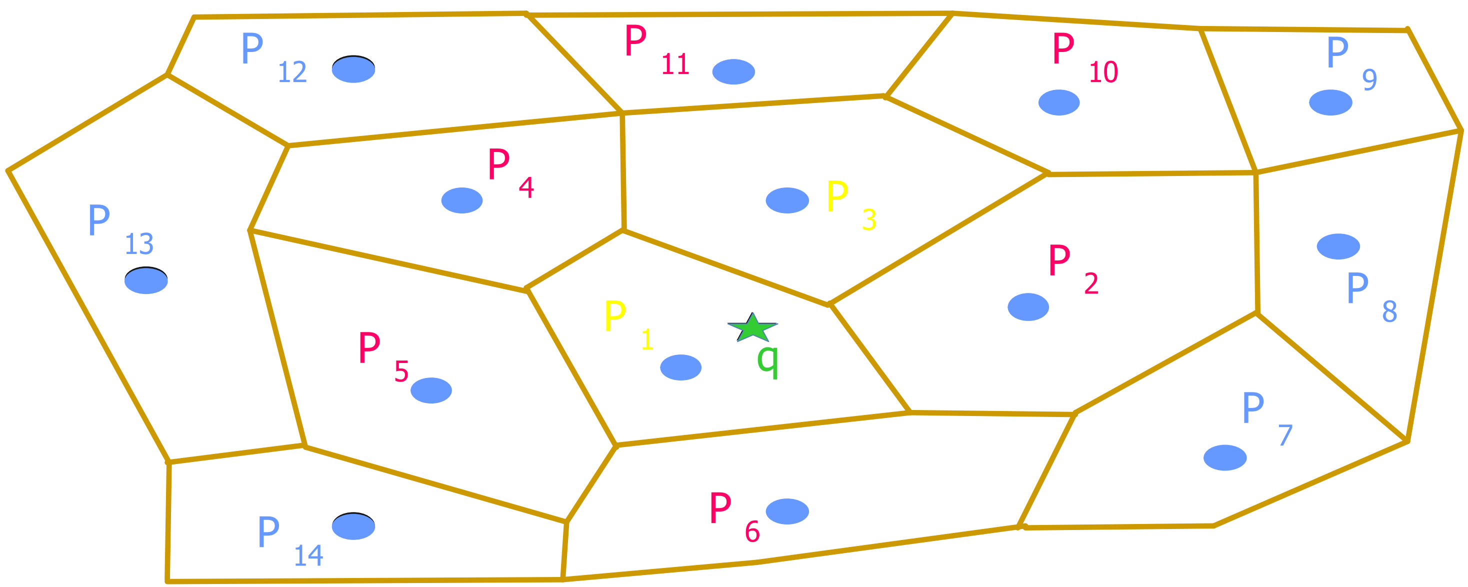
- 1^{st} NN = P_1
- 2^{nd} NN $\in$ { Neighbors(P_1) }
  (e.g., P_3)



Online Step

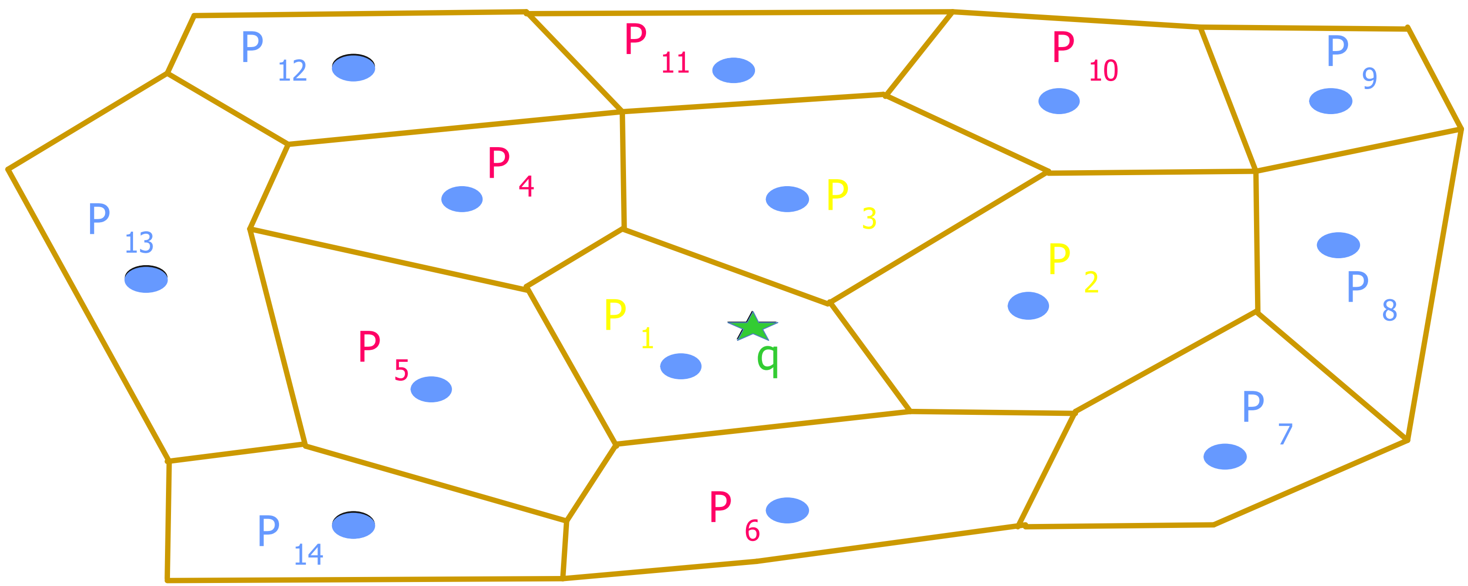
USC Viterbi School of Engineering

- 1^{st} NN = P_1
- 2^{nd} NN $\in$ { Neighbors(P_1) }
  (e.g., P_3)
- 3^{rd} NN $\in$ { Neighbors(P_1) $\cup$ Neighbors(P_3) }



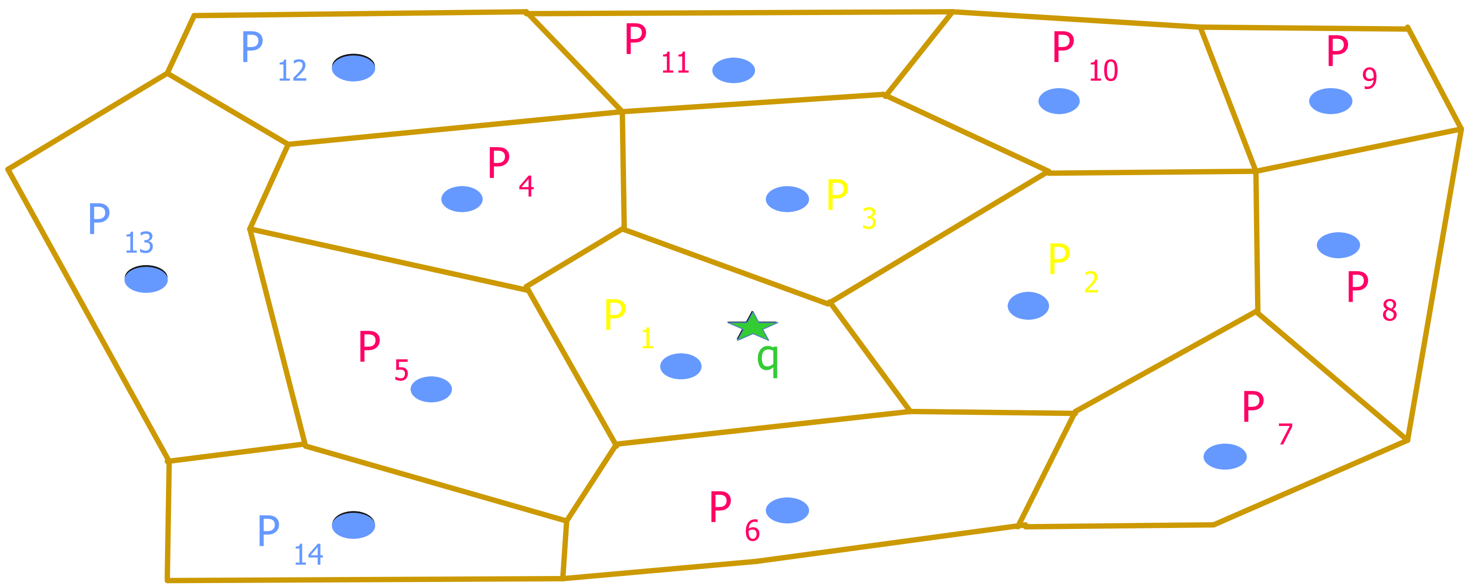
Online Step

- 1^{st} NN = P_1
- 2^{nd} NN $\in$ { Neighbors(P_1) }
  (e.g., P_3)
- 3^{rd} NN $\in$ { Neighbors(P_1) $\cup$ Neighbors(P_3) }
  (e.g., P_2)



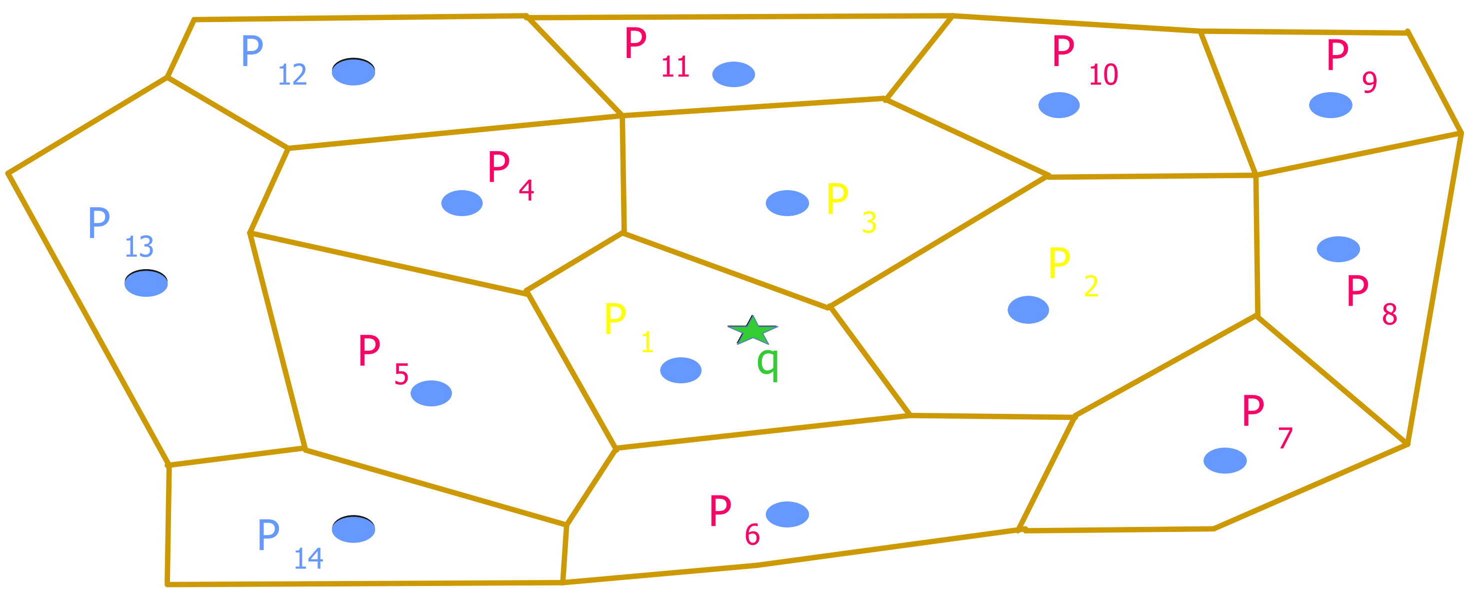
Online Step

- 1^{st} NN = P_1
- 2^{nd} NN $\in$ { Neighbors(P_1) }
  (e.g., P_3)
- 3^{rd} NN $\in$ { Neighbors(P_1) $\cup$ Neighbors(P_3) }
  (e.g., P_2)
- 4^{th} NN $\in$ { Neighbors(P_1) $\cup$ Neighbors(P_3) $\cup$ Neighbors(P_2) }
- …..



Online Step

- 1^{st} NN = P_1
- 2^{nd} NN $\in$ { Neighbors(P_1) }
  (e.g., P_3)
- 3^{rd} NN $\in$ { Neighbors(P_1) $\cup$ Neighbors(P_3) }
  (e.g., P_2)
- 4^{th} NN $\in$ { Neighbors(P_1) $\cup$ Neighbor
- …..

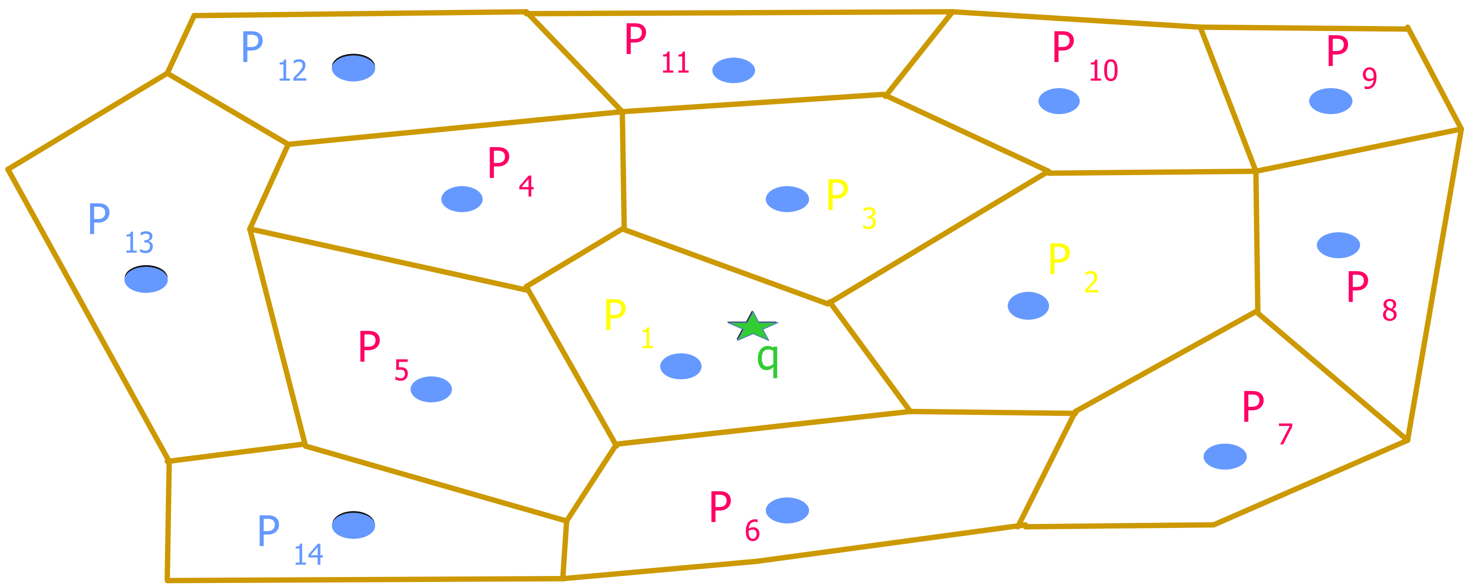


USC Viterbi School of Engineering

Online Step

- 1^{st} NN = P_1
- 2^{nd} NN $\in$ { Neighbors(P_1) }
  (e.g., P_3)
- 3^{rd} NN $\in$ { Neighbors(P_1) $\cup$ Neighbors(P_3) }
  (e.g., P_2)
- 4^{th} NN $\in$ { Neighbors(P_1) $\cup$ Neighbor
- .....

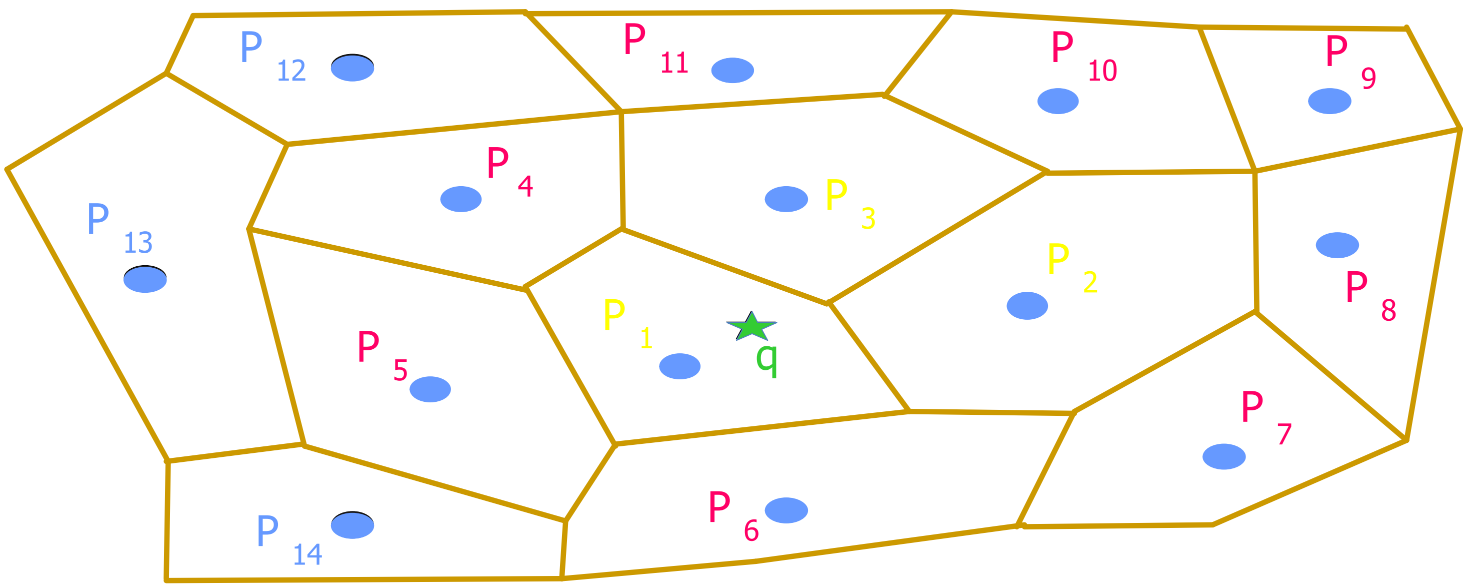
GOOD NEWS! In theory average # of neighbors= O(5k+1)=O(k)



Online Step

- 1^{st} NN = P_1
- 2^{nd} NN $\in$ { Neighbors(P_1) }
  (e.g., P_3)
- 3^{rd} NN $\in$ { Neighbors(P_1) $\cup$ Neighbors(P_3) }
  (e.g., P_2)
- 4^{th} NN $\in$ { Neighbors(P_1) $\cup$ Neighbor
- .....

GOOD NEWS! In theory average # of neighbors= O(5k+1)=O(k)
In Practice: << 5 x K



USC Viterbi School of Engineering

Performance of VN^3

- Data set:
  - Road network in Los Angeles (from NavTeq)
    - 110,000 streets, 79,800 intersections
    - Different points of interest:
      - restaurants, auto services, schools, parks, shopping centers, hospitals
- Measured:
  - Query response time and CPU
    - Comparison with INE [Papadias et al. (vldb03)]
  - Size of candidate set of VN^3 filter
    - Comparison with R-tree-based KNN [Seidl et al. (sigmod98) and Hjaltason et al. (tods99)]

USC Viterbi School of Engineering

Pre-computation Overhead of VN^3

Entities	Cardinality	Points Inside each NVP	Average BPs per NVP	No. of Pre-comp. Border to Border	No. of Pre-comp. Border to inside
Hospitals	0.04%	1698	52	232,000	8,781,000
Shopping	0.16%	458	25	225,600	4,653,000
Parks	0.53%	142	14	239,500	2,630,000
Schools	1.15%	64	10	246,000	1,787,000
Auto. Svc.	3.26%	38	7	239,900	1,611,000
Restaurants	5.80%	27	6	243,600	1,348,000

Pre-computation Overhead of VN^3

Entities	Cardinality	Points Inside each NVP	Average BPs per NVP	No. of Pre-comp. Border to Border	No. of Pre-comp. Border to inside
Hospitals	0.04%	1698	52	232,000	8,781,000
Shopping	0.16%	458	25	225,600	4,653,000
Parks	0.53%	142	14	239,500	2,630,000
Schools	1.15%	64	10	246,000	1,787,000
Auto. Svc.	3.26%	38	7	239,900	1,611,000
Restaurants	5.80%	27	6	243,600	1,348,000

Bigger/less NVPs when sparse

USC Viterbi School of Engineering

Pre-computation Overhead of VN^3

Entities	Cardinality	Points Inside each NVP	Average BPs per NVP	No. of Pre-comp. Border to Border	No. of Pre-comp. Border to inside
Hospitals	0.04%	1698	52	232,000	8,781,000
Shopping	0.16%	458	25	225,600	4,653,000
Parks	0.53%	142	14	239,500	2,630,000
Schools	1.15%	64	10	246,000	1,787,000
Auto. Svc.	3.26%	38	7	239,900	1,611,000
Restaurants	5.80%	27	6	243,600	1,348,000

Smaller/more NVPs when dense

USC Viterbi School of Engineering

Pre-computation Overhead of VN^3

Entities	Cardinality	Points Inside each NVP	Average BPs per NVP	No. of Pre-comp. Border to Border	No. of Pre-comp. Border to inside
Hospitals	0.04%	1698	52	232,000	8,781,000
Shopping	0.16%	458	25	225,600	4,653,000
Parks	0.53%	142	14	239,500	2,630,000
Schools	1.15%	64	10	246,000	1,787,000
Auto. Svc.	3.26%	38	7	239,900	1,611,000
Restaurants	5.80%	27	6	243,600	1,348,000

More Border point

Pre-computation Overhead of VN^3

Entities	Cardinality	Points Inside each NVP	Average BPs per NVP	No. of Pre-comp. Border to Border	No. of Pre-comp. Border to inside
Hospitals	0.04%	1698	52	232,000	8,781,000
Shopping	0.16%	458	25	225,600	4,653,000
Parks	0.53%	142	14	239,500	2,630,000
Schools	1.15%	64	10	246,000	1,787,000
Auto. Svc.	3.26%	38	7	239,900	1,611,000
Restaurants	5.80%	27	6	243,600	1,348,000

Almost same

USC Viterbi School of Engineering

Pre-computation Overhead of VN^3

Entities	Cardinality	Points Inside each NVP	Average BPs per NVP	No. of Pre-comp. Border to Border	No. of Pre-comp. Border to inside
Hospitals	0.04%	1698	52	232,000	8,781,000
Shopping	0.16%	458	25	225,600	4,653,000
Parks	0.53%	142	14	239,500	2,630,000
Schools	1.15%	64	10	246,000	1,787,000
Auto. Svc.	3.26%	38	7	239,900	1,611,000
Restaurants	5.80%	27	6	243,600	1,348,000

Increases when sparse

USC Viterbi School of Engineering

Pre-computation Overhead of VN^3

Entities	Cardinality	Points Inside each NVP	Average BPs per NVP	No. of Pre-comp. Border to Border	No. of Pre-comp. Border to inside
Hospitals	0.04%	1698	52	232,000	8,781,000
Shopping	0.16%	458	25	225,600	4,653,000
Parks	0.53%	142	14	239,500	2,630,000
Schools	1.15%	64	10	246,000	1,787,000
Auto. Svc.	3.26%	38	7	239,900	1,611,000
Restaurants	5.80%	27	6	243,600	1,348,000

Naive approach: 3.2 billion pre-computations

USC Viterbi School of Engineering

Comparison of VN^3 and INE

- K = 1

NavTeq Entities	Density, Cardinality	VN^3 (cpu) , db sec	INE (cpu) , disk sec
Hospitals	0.04% , 46	(0) , 0.018	(0.30) , 12.4
Shopping	0.16% , 173	(0) , 0.020	(0.090) , 3.6
Parks	0.53% , 561	(0) , 0.021	(0.030) , 1.4
Schools	1.15% , 1230	(0) , 0.027	(0.015) , 0.6
Auto. Svc.	3.26% , 2093	(0) , 0.030	(0.013) , 0.67
Restaurants	5.80% , 2944	(0) , 0.032	(0.010) , 0.49

USC Viterbi School of Engineering

Comparison of VN^3 and INE

- K = 1

NavTeq Entities	Density, Cardinality	VN^3 (cpu) , db sec	INE (cpu) , disk sec
Hospitals	0.04% , 46	(0) , 0.018	(0.30) , 12.4
Shopping	0.16% , 173	(0) , 0.020	(0.090) , 3.6
Parks	0.53% , 561	(0) , 0.021	(0.030) , 1.4
Schools	1.15% , 1230	(0) , 0.027	(0.015) , 0.6
Auto. Svc.	3.26% , 2093	(0) , 0.030	(0.013) , 0.67
Restaurants	5.80% , 2944	(0) , 0.032	(0.010) , 0.49

Almost instantly

Comparison of VN^3 and INE

- K = 1

NavTeq Entities	Density, Cardinality	VN^3 (cpu) , db sec	INE (cpu) , disk sec
Hospitals	0.04% , 46	(0) , 0.018	(0.30) , 12.4
Shopping	0.16% , 173	(0) , 0.020	(0.090) , 3.6
Parks	0.53% , 561	(0) , 0.021	(0.030) , 1.4
Schools	1.15% , 1230	(0) , 0.027	(0.015) , 0.6
Auto. Svc.	3.26% , 2093	(0) , 0.030	(0.013) , 0.67
Restaurants	5.80% , 2944	(0) , 0.032	(0.010) , 0.49

0 cpu time!

USC Viterbi School of Engineering

Comparison of VN^3 and INE

- K = 1

NavTeq Entities	Density, Cardinality	VN^3 (cpu) , db sec	INE (cpu) , disk sec
Hospitals	0.04% , 46	(0) , 0.018	(0.30) , 12.4
Shopping	0.16% , 173	(0) , 0.020	(0.090) , 3.6
Parks	0.53% , 561	(0) , 0.021	(0.030) , 1.4
Schools	1.15% , 1230	(0) , 0.027	(0.015) , 0.6
Auto. Svc.	3.26% , 2093	(0) , 0.030	(0.013) , 0.67
Restaurants	5.80% , 2944	(0) , 0.032	(0.010) , 0.49

Increases for sparse

Comparison of VN^3 and INE

- K = 5

Entities	Density, Cardinality	VN^3 (cpu) , db sec	INE (cpu) , disk sec
Hospitals	0.04% , 46	(1.5) , 6.5	(1.7) , 78.3
Shopping	0.16% , 173	(0.45) , 3.3	(0.5) , 21.1
Parks	0.53% , 561	(0.15) , 1.5	(0.2) , 8.2
Schools	1.15% , 1230	(0.06) , 0.75	(0.07) , 3.5
Auto. Svc.	3.26% , 2093	(0.01) , 0.47	(0.05) , 2.43
Restaurants	5.80% , 2944	(0.01) , 0.87	(0.03) , 1.5

VN^3 cpu time comparable to INE

USC Viterbi School of Engineering

Comparison of VN^3 and INE

- K = 5

Entities	Density, Cardinality	VN^3 (cpu) , db sec	INE (cpu) , disk sec
Hospitals	0.04% , 46	(1.5) , 6.5	(1.7) , 78.3
Shopping	0.16% , 173	(0.45) , 3.3	(0.5) , 21.1
Parks	0.53% , 561	(0.15) , 1.5	(0.2) , 8.2
Schools	1.15% , 1230	(0.06) , 0.75	(0.07) , 3.5
Auto. Svc.	3.26% , 2093	(0.01) , 0.47	(0.05) , 2.43
Restaurants	5.80% , 2944	(0.01) , 0.87	(0.03) , 1.5

Database time still dominant

USC Viterbi School of Engineering

Comparison of VN^3 and INE

- K = 5

Entities	Density, Cardinality	VN^3 (cpu) , db sec	INE (cpu) , disk sec
Hospitals	0.04% , 46	(1.5) , 6.5	(1.7) , 78.3
Shopping	0.16% , 173	(0.45) , 3.3	(0.5) , 21.1
Parks	0.53% , 561	(0.15) , 1.5	(0.2) , 8.2
Schools	1.15% , 1230	(0.06) , 0.75	(0.07) , 3.5
Auto. Svc.	3.26% , 2093	(0.01) , 0.47	(0.05) , 2.43
Restaurants	5.80% , 2944	(0.01) , 0.87	(0.03) , 1.5

Excluding K=1, VN3 is between 2 (high densities / larger K) to 14 (low densities / larger K) times faster than INE

VN^3 vs INE

		Query Processing Time (Sec)											
		K=1		K=5		K=10		K=25		K=50		K=100	
		VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE
Entities	Qty (density)	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk
Hospital	46 (0.0004)	(0) 0.018	(0.3) 12.4	(1.5) 6.5	(1.7) 78.3	(4.5) 14.0	(3.8) 165.1	(15.3) 35.1	(10.1) 430.2	- -	- -	- -	- -
Shopping Centers	173 (0.0016)	(0) 0.020	(0.09) 3.6	(0.45) 3.3	(0.5) 21.1	(1.3) 6.9	(1.1) 44.0	(3.4) 18.1	(3.1) 118.0	- -	- -	- -	- -
Parks	561 (0.0053)	(0) 0.021	(0.03) 1.4	(0.15) 1.5	(0.2) 8.2	(0.37) 2.8	(0.3) 15.3	(1.4) 6.4	(0.8) 36.4	(2.5) 13.3	(1.6) 71.1	- -	- -
Schools	1230 (0.0115)	(0) 0.027	(0.015) 0.6	(0.06) 0.75	(0.07) 3.5	(0.18) 1.46	(0.14) 6.6	(0.7) 3.9	(0.36) 15.6	(1.9) 7.5	(0.7) 32.2	- -	- -
Auto Services	2093 (0.0326)	(0) 0.030	(0.013) 0.57	(0.01) 0.65	(0.05) 2.43	(0.09) 1.4	(0.09) 4.3	(0.58) 2.95	(0.23) 10.0	(1.65) 6.68	(0.44) 19.4	(2.78) 13.1	(0.87) 38.00
Restaurants	2944 (0.0580)	(0) 0.032	(0.01) 0.49	(0.01) 0.57	(0.03) 1.34	(0.04) 1.48	(0.06) 2.7	(0.26) 2.8	(0.15) 6.8	(0.8) 6.1	(0.3) 13.3	(1.85) 12.8	(0.6) 26.0

VN^3 vs INE

VN^3 finds the 1st NN using R-Tree
INE needs to expand the network around q

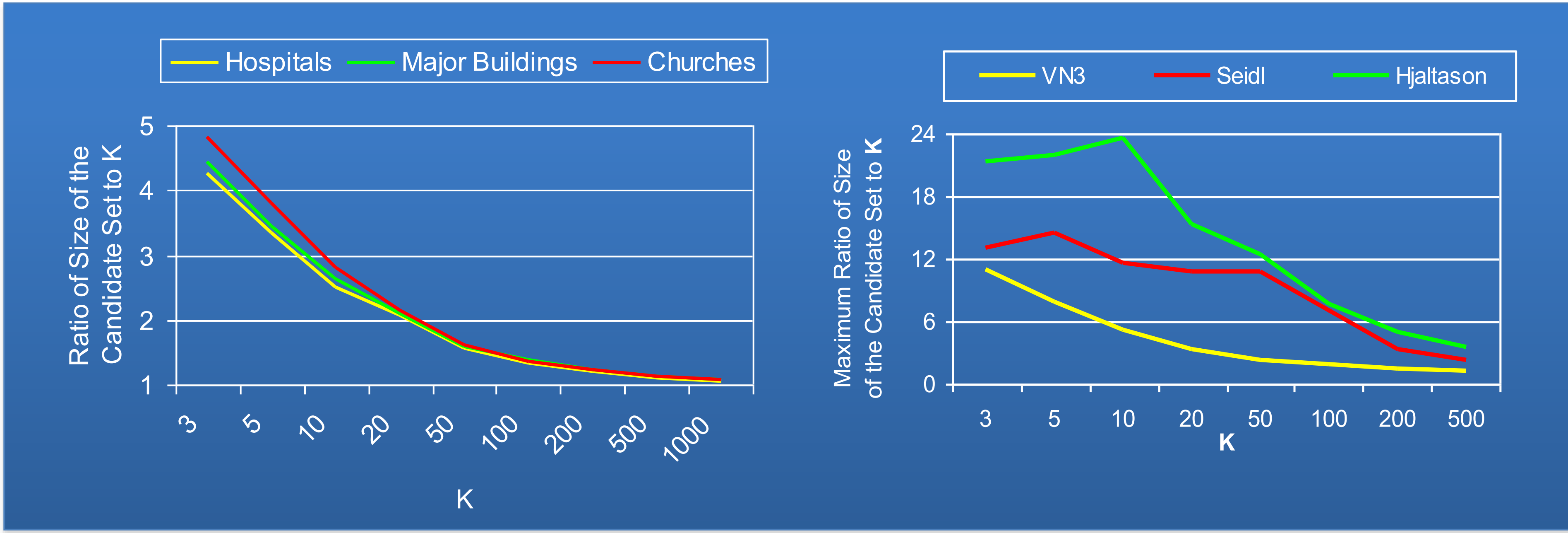
		Query Processing Time (Sec)											
		K=1		K=5		K=10		K=25		K=50		K=100	
		VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE
Entities	Qty (density)	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk
Hospital	46 (0.0004)	(0) 0.018	(0.3) 12.4	(1.5) 6.5	(1.7) 78.3	(4.5) 14.0	(3.8) 165.1	(15.3) 35.1	(10.1) 430.2	- -	- -	- -	- -
Shopping Centers	173 (0.0016)	(0) 0.020	(0.09) 3.6	(0.45) 3.3	(0.5) 21.1	(1.3) 6.9	(1.1) 44.0	(3.4) 18.1	(3.1) 118.0	- -	- -	- -	- -
Parks	561 (0.0053)	(0) 0.021	(0.03) 1.4	(0.15) 1.5	(0.2) 8.2	(0.37) 2.8	(0.3) 15.3	(1.4) 6.4	(0.8) 36.4	(2.5) 13.3	(1.6) 71.1	- -	- -
Schools	1230 (0.0115)	(0) 0.027	(0.015) 0.6	(0.06) 0.75	(0.07) 3.5	(0.18) 1.46	(0.14) 6.6	(0.7) 3.9	(0.36) 15.6	(1.9) 7.5	(0.7) 32.2	- -	- -
Auto Services	2093 (0.0326)	(0) 0.030	(0.013) 0.57	(0.01) 0.65	(0.05) 2.43	(0.09) 1.4	(0.09) 4.3	(0.58) 2.95	(0.23) 10.0	(1.65) 6.68	(0.44) 19.4	(2.78) 13.1	(0.87) 38.00
Restaurants	2944 (0.0580)	(0) 0.032	(0.01) 0.49	(0.01) 0.57	(0.03) 1.34	(0.04) 1.48	(0.06) 2.7	(0.26) 2.8	(0.15) 6.8	(0.8) 6.1	(0.3) 13.3	(1.85) 12.8	(0.6) 26.0

VN^3 vs INE

CPU Time:
INE uses a queue which is incrementally updated

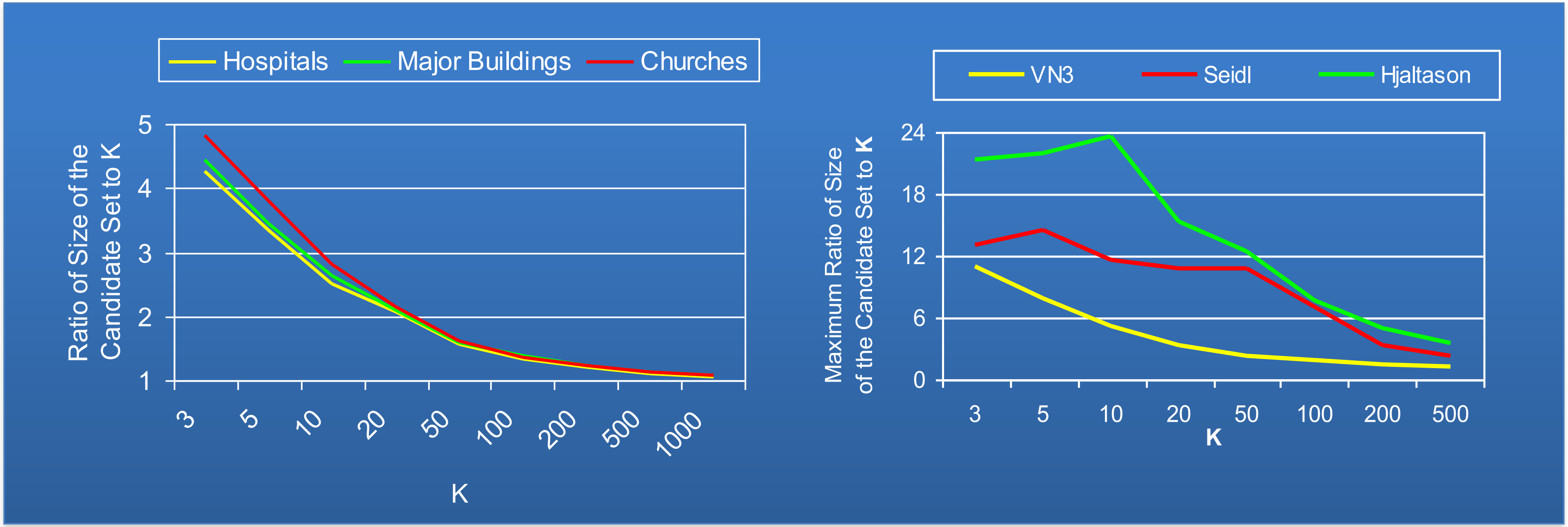
		Query Processing Time (Sec)											
		K=1		K=5		K=10		K=25		K=50		K=100	
		VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE	VN^3	INE
Entities	Qty (density)	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk	(cpu) disk
Hospital	46 (0.0004)	(0) 0.018	(0.3) 12.4	(1.5) 6.5	(1.7) 78.3	(4.5) 14.0	(3.8) 165.1	(15.3) 35.1	(10.1) 430.2	- -	- -	- -	- -
Shopping Centers	173 (0.0016)	(0) 0.020	(0.09) 3.6	(0.45) 3.3	(0.5) 21.1	(1.3) 6.9	(1.1) 44.0	(3.4) 18.1	(3.1) 118.0	- -	- -	- -	- -
Parks	561 (0.0053)	(0) 0.021	(0.03) 1.4	(0.15) 1.5	(0.2) 8.2	(0.37) 2.8	(0.3) 15.3	(1.4) 6.4	(0.8) 36.4	(2.5) 13.3	(1.6) 71.1	- -	- -
Schools	1230 (0.0115)	(0) 0.027	(0.015) 0.6	(0.06) 0.75	(0.07) 3.5	(0.18) 1.46	(0.14) 6.6	(0.7) 3.9	(0.36) 15.6	(1.9) 7.5	(0.7) 32.2	- -	- -
Auto Services	2093 (0.0326)	(0) 0.030	(0.013) 0.57	(0.01) 0.65	(0.05) 2.43	(0.09) 1.4	(0.09) 4.3	(0.58) 2.95	(0.23) 10.0	(1.65) 6.68	(0.44) 19.4	(2.78) 13.1	(0.87) 38.00
Restaurants	2944 (0.0580)	(0) 0.032	(0.01) 0.49	(0.01) 0.57	(0.03) 1.34	(0.04) 1.48	(0.06) 2.7	(0.26) 2.8	(0.15) 6.8	(0.8) 6.1	(0.3) 13.3	(1.85) 12.8	(0.6) 26.0

VN^3: Performance of Filter Step



- USGS Data, Euclidean space

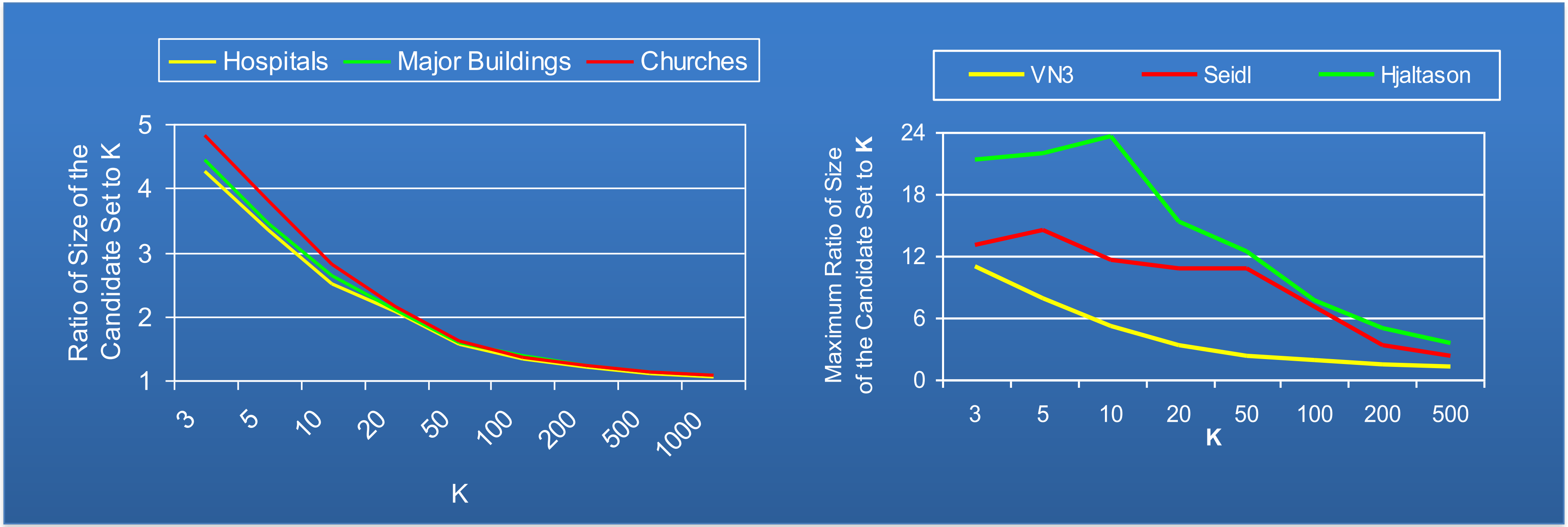
VN^3: Performance of Filter Step



- USGS Data, Euclidean space

- Ratio of size of candidate set to K decreases as K increases
  - Some of the new neighbors are already explored!
- VN^3's filter Behaves independently from density/distribution of points of interest
  - Size of the candidate set (O(k)) is independent of the density/distribution

VN^3: Performance of Filter Step



- USGS Data, Euclidean space
- Ratio of size of candidate set to K decreases as K increases
  - Some of the new neighbors are already explored!
- VN^3' s filter Behaves independently from density/distribution of points of interest
  - Size of the candidate set (O(k)) is independent of the density/distribution
- Better selectivity as compared to other approaches (up to 4 times)
- Very low variance (predictable behavior)

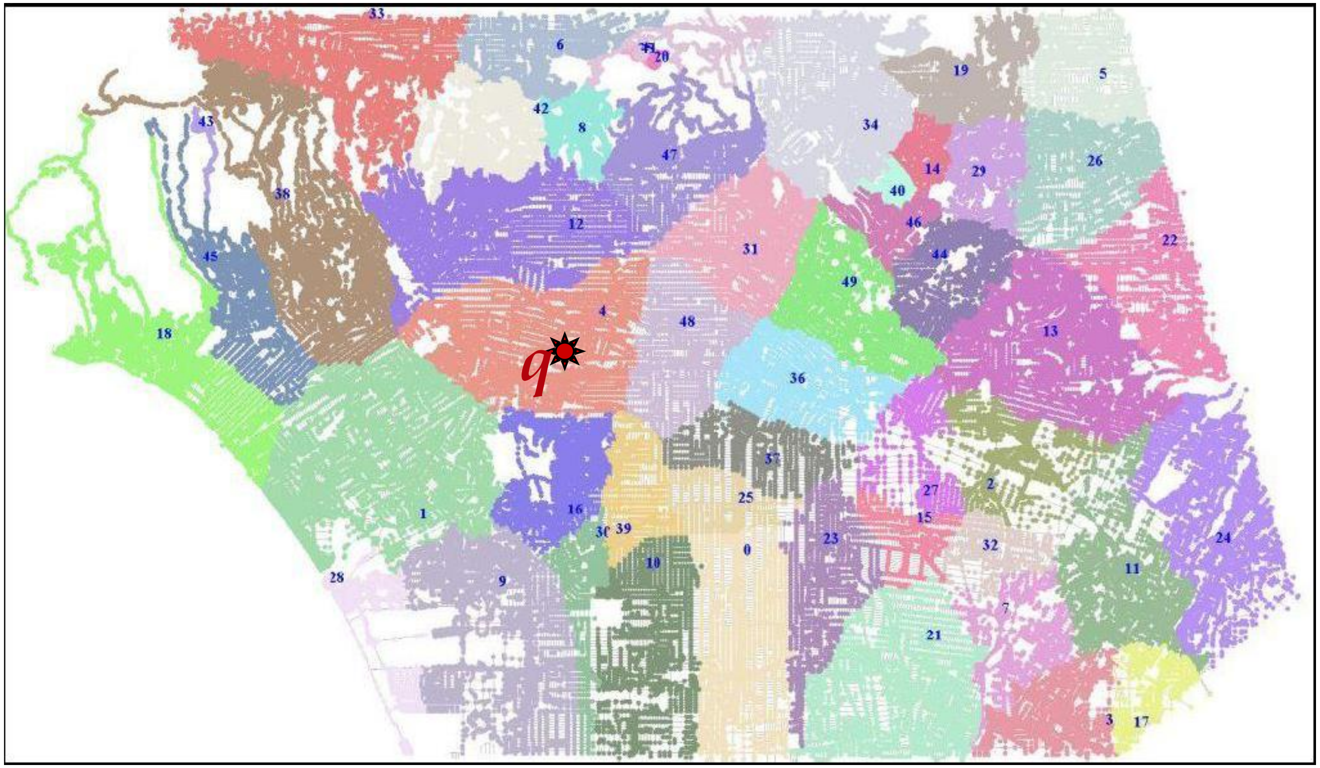
Conclusion

- A novel approach for KNN queries in SNDB
- Based on:
  - Pre-calculating the solution space (first order Voronoi diagrams)
  - Pre-computing some distances (from borders to points inside each polygon)
- Outperforms the only other solution for SNDB
- Independent from object distribution
- Outperforms the solutions for Euclidean space in "filtering"
- Although VN3 is built on complex properties to prove its correctness, but it is a straightforward approach to implement by utilizing simple data structures such as R-tree and lookup tables.

USC Viterbi
School of Engineering

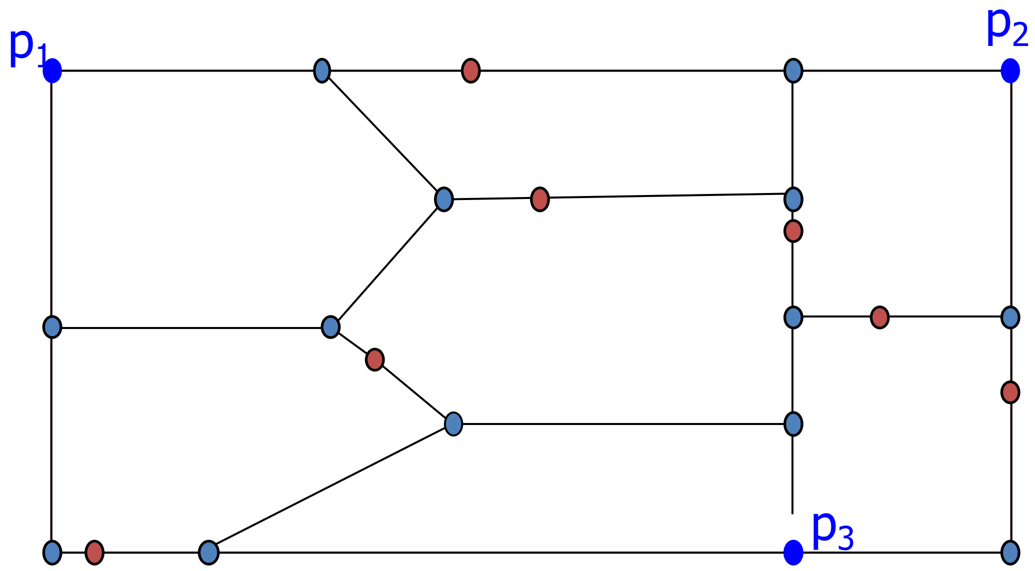
Problem Definition

- Given a NVD and query location q, how can we efficiently find the NVP that overlaps with q; **contain(q)**?



Indexing NVD

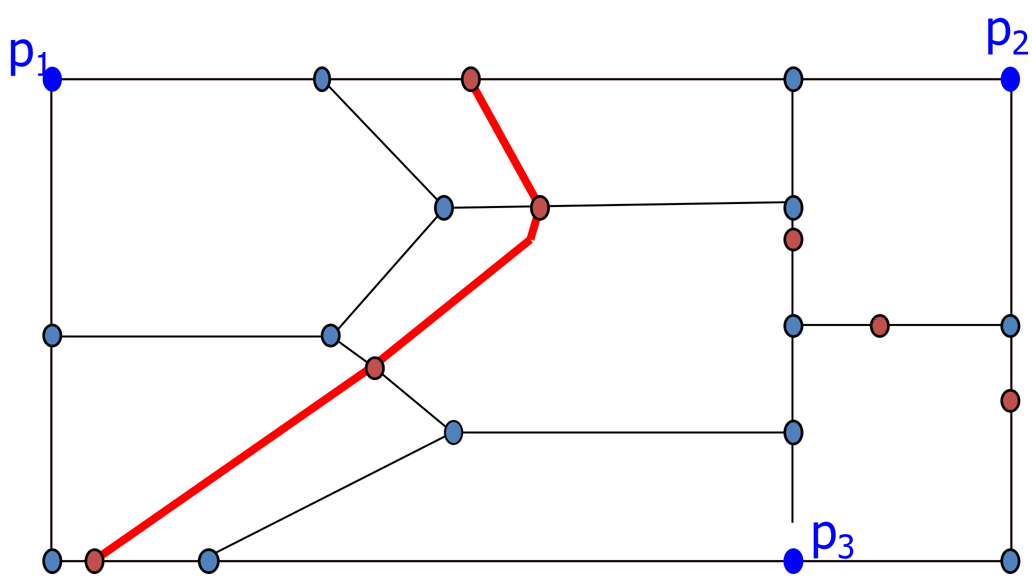
- Voronoi R-tree (VR-tree)
  - Generate NVPs by connecting border points (See Section 4.3)
  - Create minimum bounding rectangle (MBR) around NVPs and index MBRs with R-Tree



USC Viterbi School of Engineering

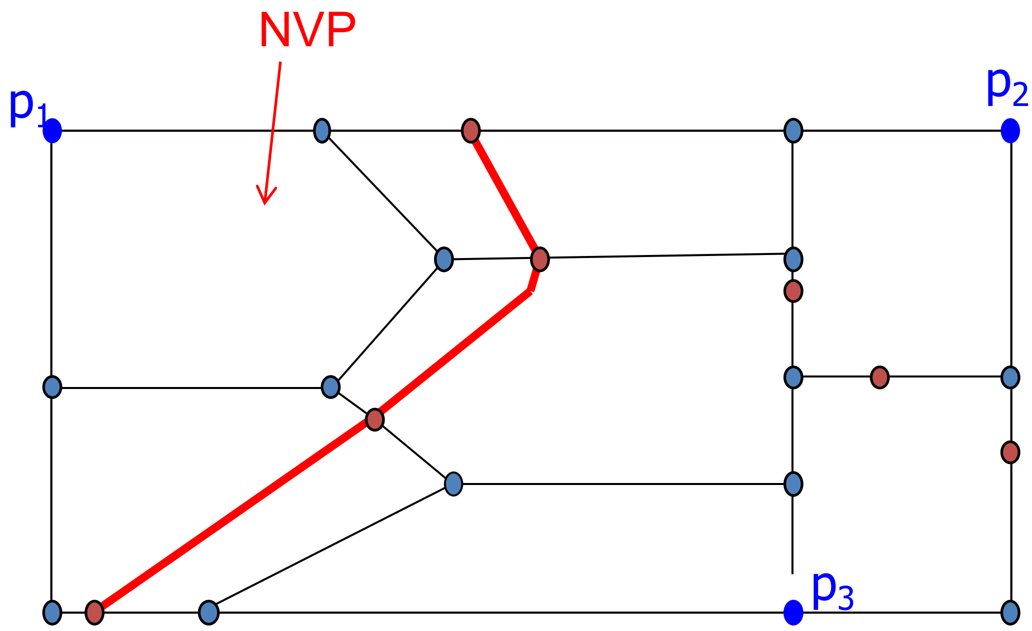
Indexing NVD

- Voronoi R-tree (VR-tree)
  - Generate NVPs by connecting border points (See Section 4.3)
  - Create minimum bounding rectangle (MBR) around NVPs and index MBRs with R-Tree



Indexing NVD

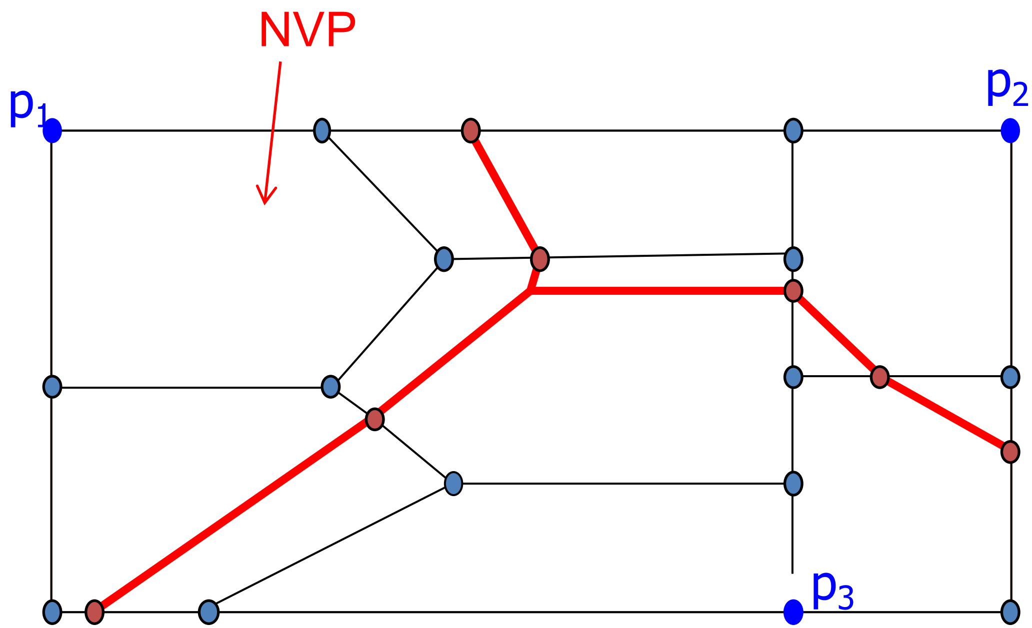
- Voronoi R-tree (VR-tree)
  - Generate NVPs by connecting border points (See Section 4.3)
  - Create minimum bounding rectangle (MBR) around NVPs and index MBRs with R-Tree



USC Viterbi School of Engineering

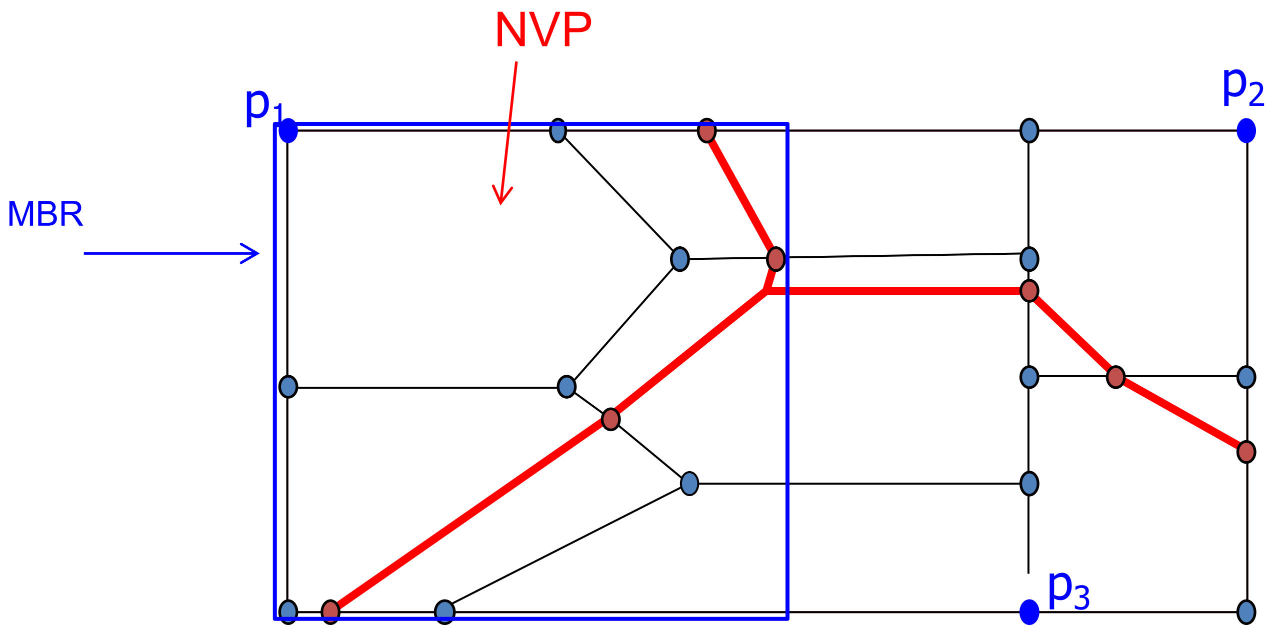
Indexing NVD

- Voronoi R-tree (VR-tree)
  - Generate NVPs by connecting border points (See Section 4.3)
  - Create minimum bounding rectangle (MBR) around NVPs and index MBRs with R-Tree



Indexing NVD

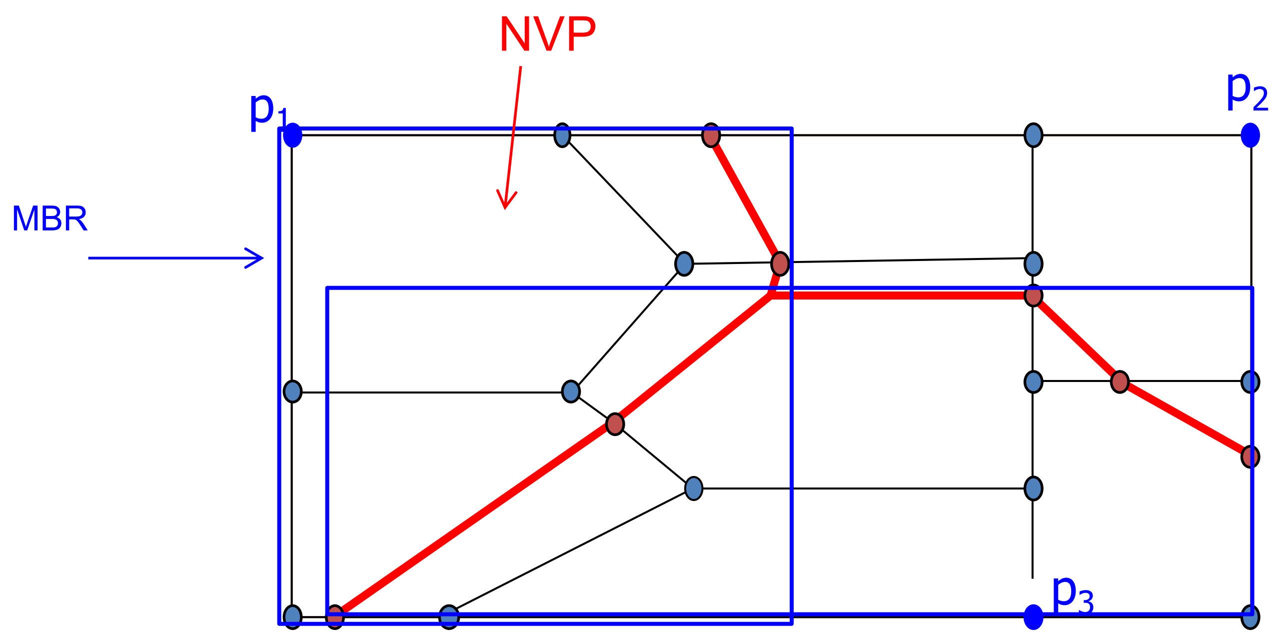
- Voronoi R-tree (VR-tree)
  - Generate NVPs by connecting border points (See Section 4.3)
  - Create minimum bounding rectangle (MBR) around NVPs and index MBRs with R-Tree



USC Viterbi School of Engineering

Indexing NVD

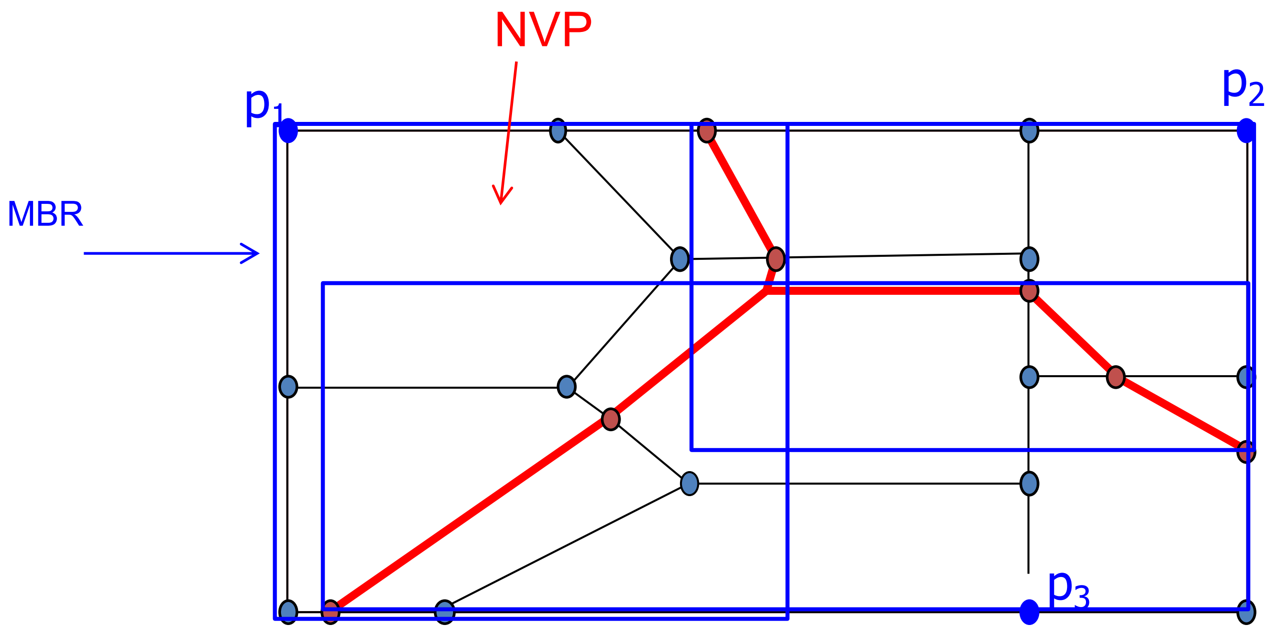
- Voronoi R-tree (VR-tree)
  - Generate NVPs by connecting border points (See Section 4.3)
  - Create minimum bounding rectangle (MBR) around NVPs and index MBRs with R-Tree



USC Viterbi School of Engineering

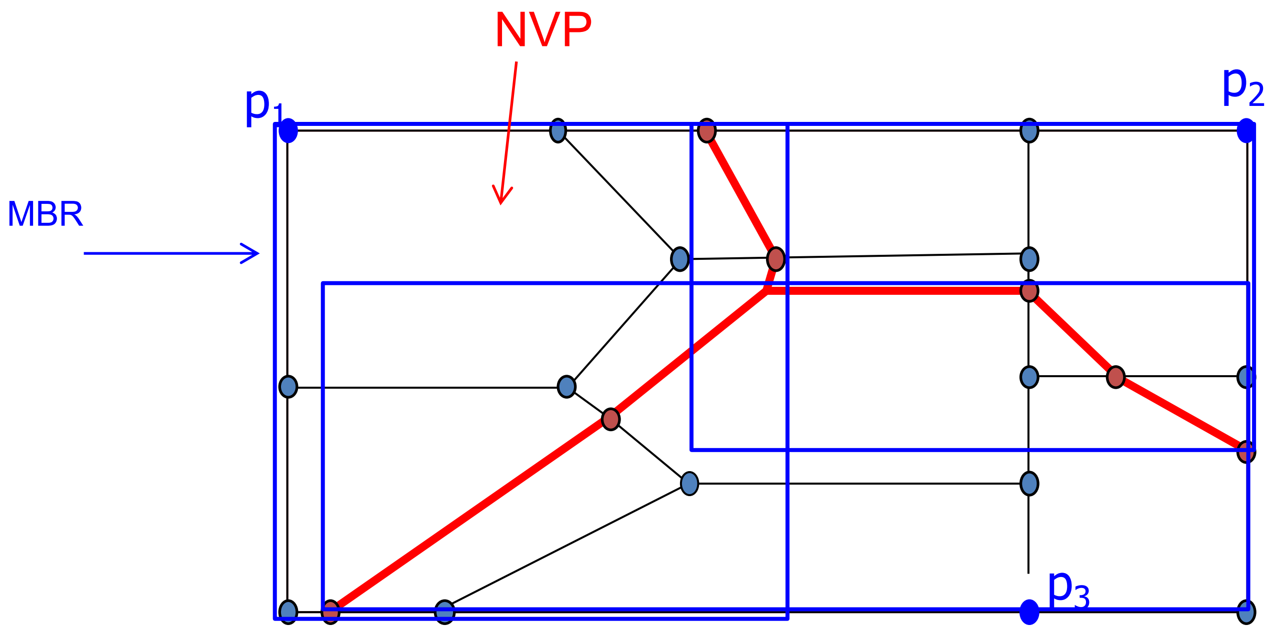
Indexing NVD

- Voronoi R-tree (VR-tree)
  - Generate NVPs by connecting border points (See Section 4.3)
  - Create minimum bounding rectangle (MBR) around NVPs and index MBRs with R-Tree



Indexing NVD

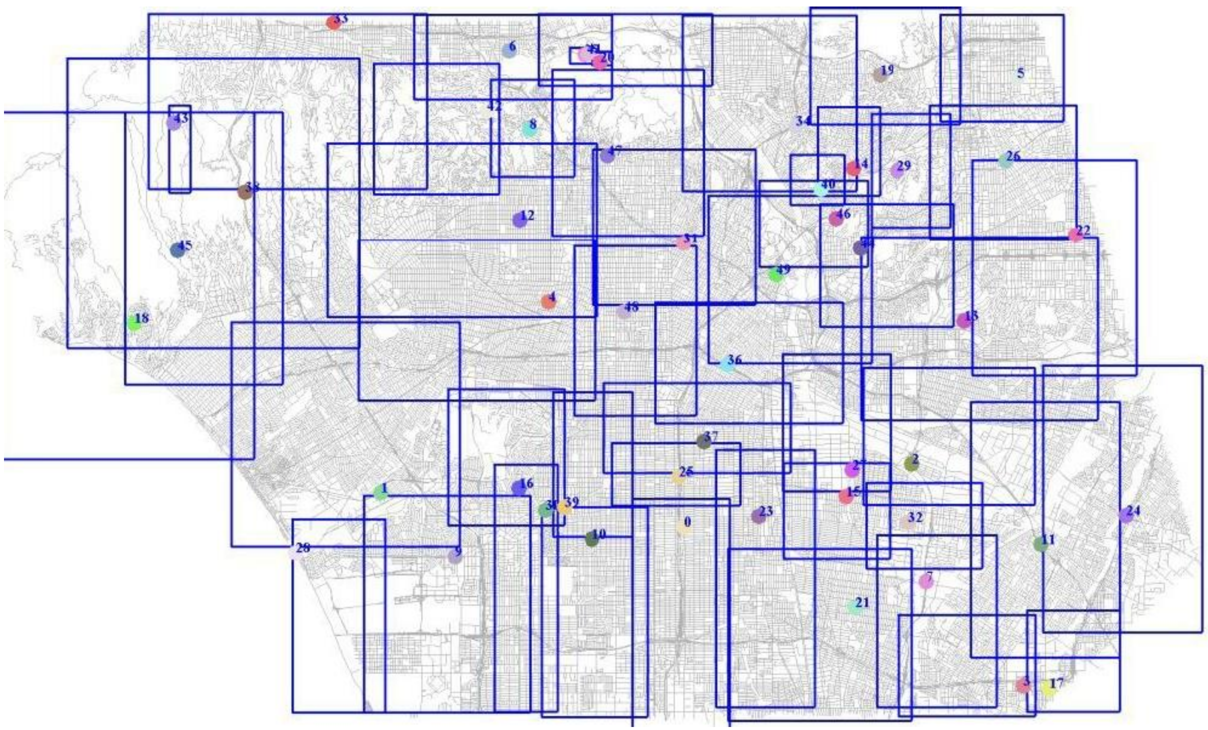
- Voronoi R-tree (VR-tree)
  - Generate NVPs by connecting border points (See Section 4.3)
  - Create minimum bounding rectangle (MBR) around NVPs and index MBRs with R-Tree



VR-tree [Shahabi'04, Taniar'07, Safar'06, Xuan'10]

Indexing NVD

- VR-tree – Problem 1
  - VR-tree splits the network space with hierarchically nested and **largely over-lapping** MBRs
  - **Non-disjoint** partitioning of the space



USC Viterbi School of Engineering

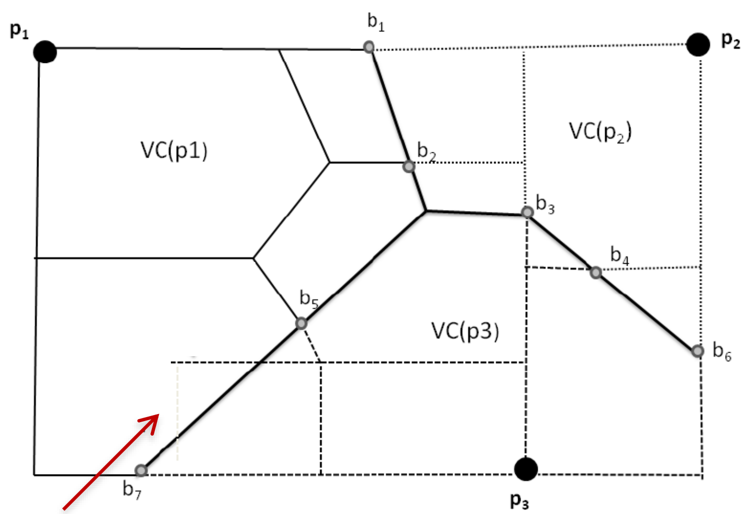
Indexing NVD

- VR-tree – Problem 1
  - VR-tree splits the network space with hierarchically nested and **largely over-lapping** MBRs
  - **Non-disjoint** partitioning of the space

USC Viterbi School of Engineering

Indexing NVD

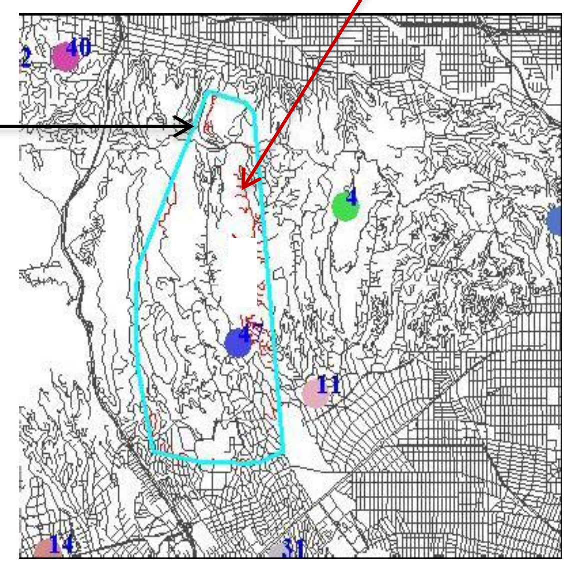
- VR-tree – Problem 2
  - Although the NVPs are computed based on the network distance metric, they are treated as polygons in Euclidean space
  - Misclassification of edges: False-negative edges



False-negative edges

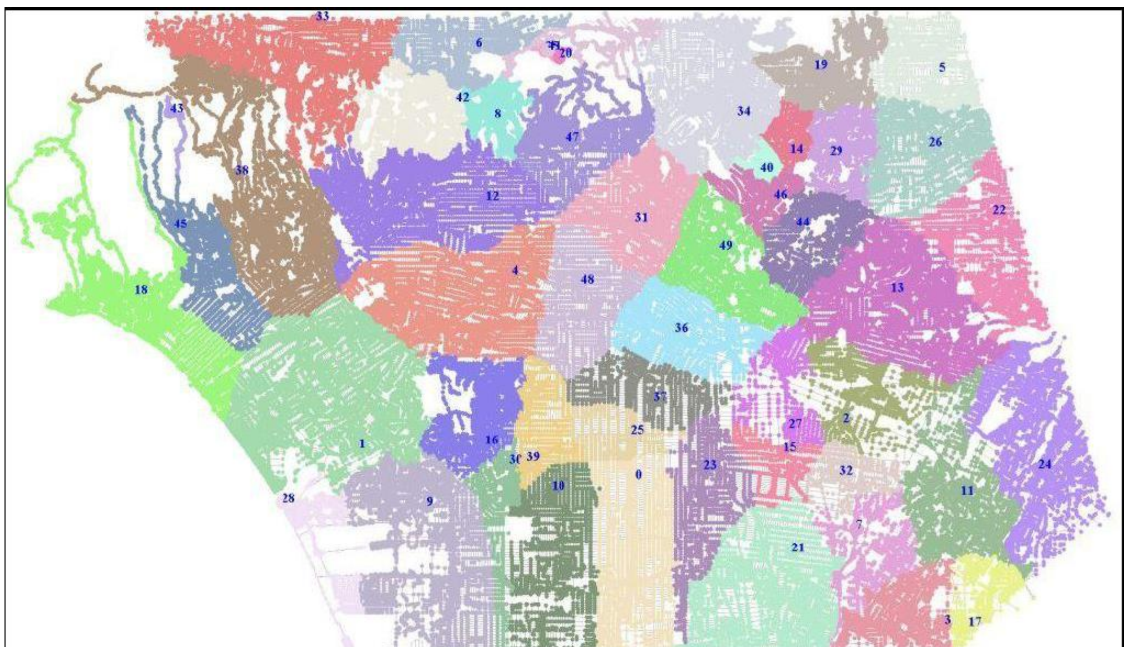
NVP

False-negative edges



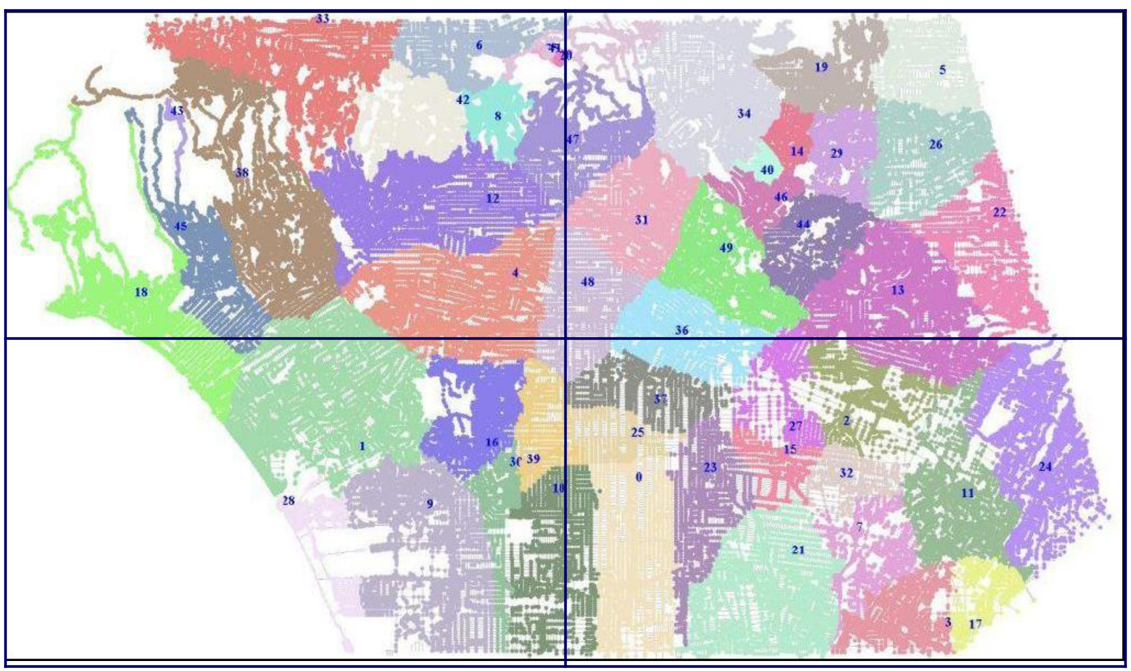
Indexing NVD

- VQ-tree
  - NVPs are
    - **mutually exclusive** : they do not have any overlapping areas
    - **collectively exhaustive**: every node/edges in the network is associated with at least one generator



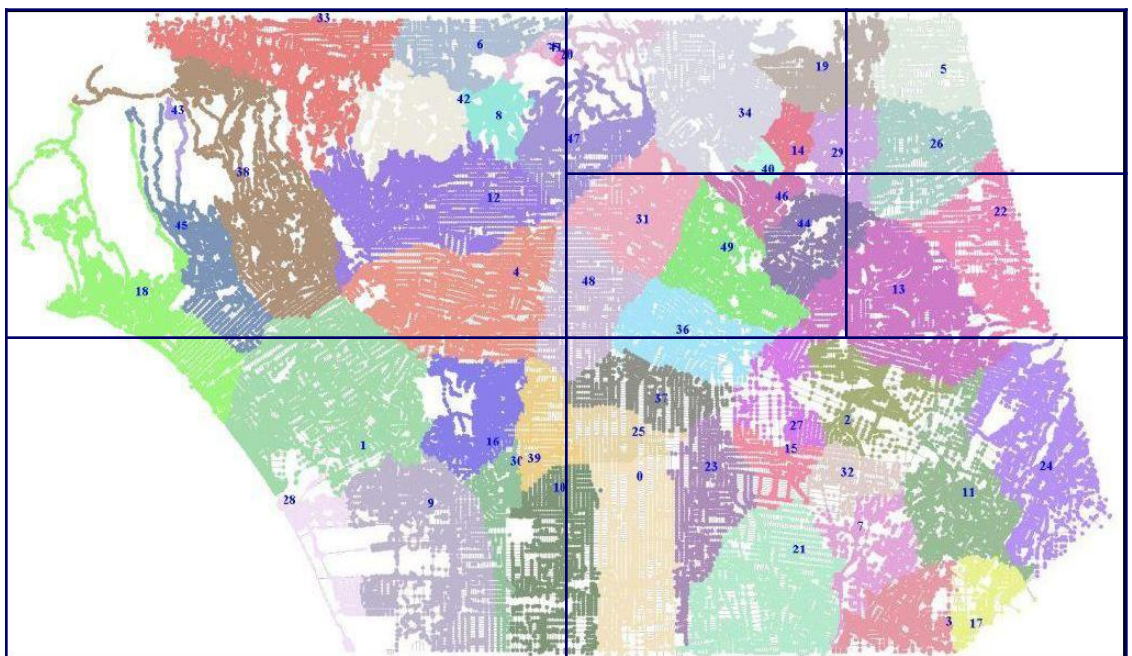
Indexing NVD

- VQ-tree
  - NVPs are
    - **mutually exclusive** : they do not have any overlapping areas
    - **collectively exhaustive**: every node/edges in the network is associated with at least one generator
  - Disjoint decomposition of the NVPs



Indexing NVD

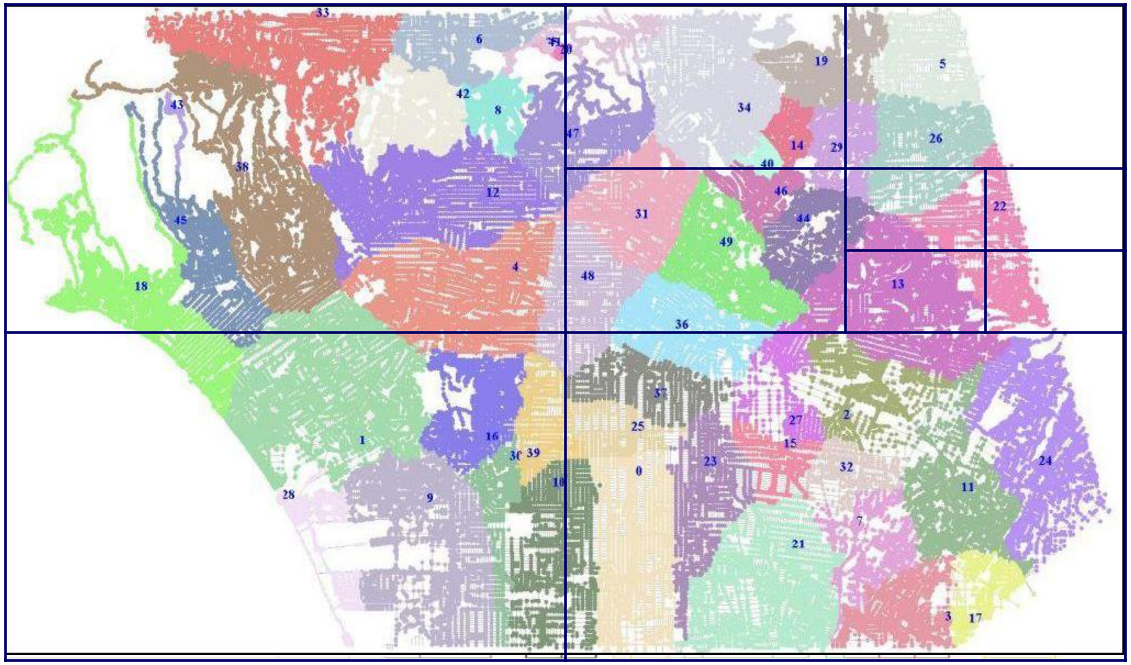
- VQ-tree
  - NVPs are
    - **mutually exclusive** : they do not have any overlapping areas
    - **collectively exhaustive**: every node/edges in the network is associated with at least one generator
  - Disjoint decomposition of the NVPs



USC Viterbi School of Engineering

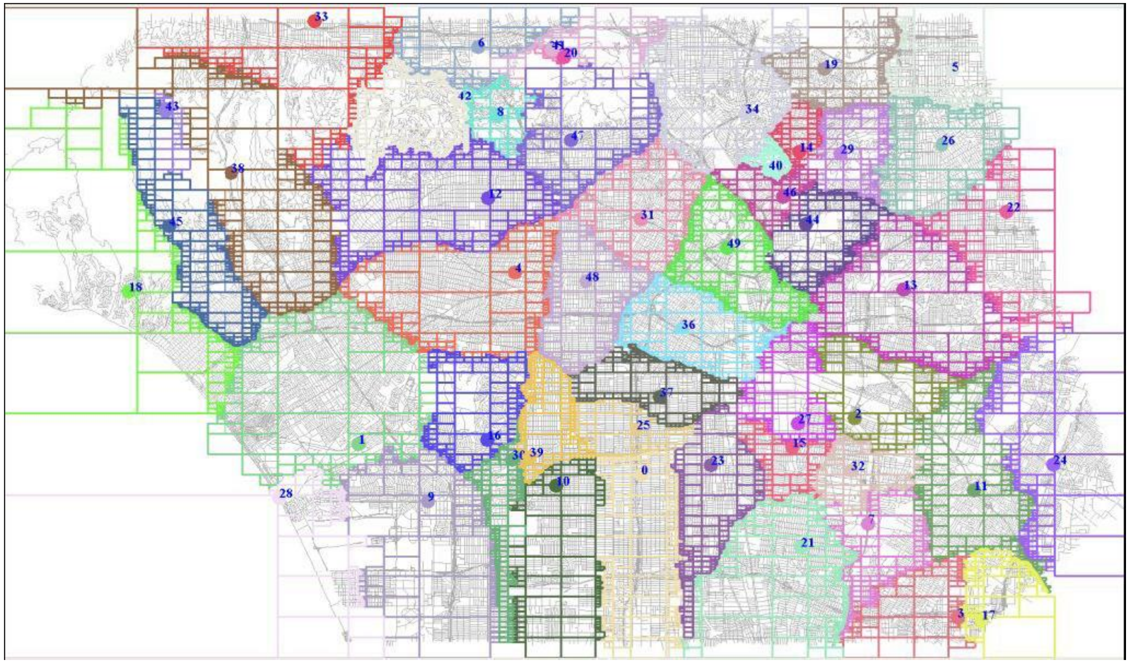
Indexing NVD

- VQ-tree
  - NVPs are
    - **mutually exclusive** : they do not have any overlapping areas
    - **collectively exhaustive**: every node/edges in the network is associated with at least one generator
  - Disjoint decomposition of the NVPs



Indexing NVD

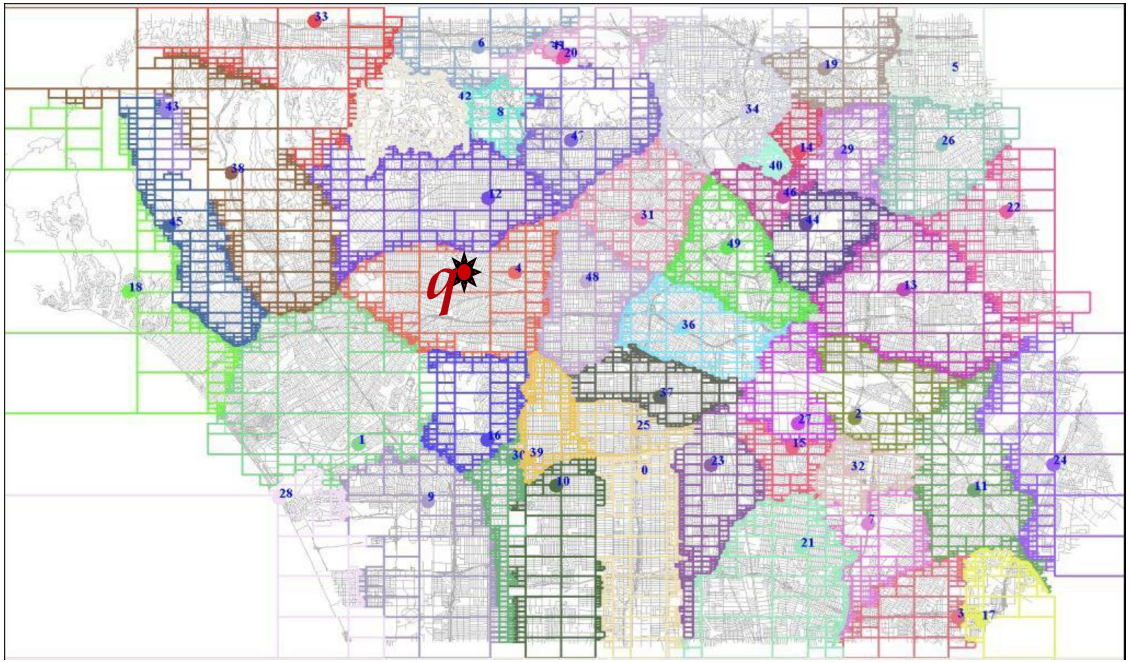
- VQ-tree
  - NVPs are
    - **mutually exclusive** : they do not have any overlapping areas
    - **collectively exhaustive**: every node/edges in the network is associated with at least one generator
  - Disjoint decomposition of the NVPs



USC Viterbi School of Engineering

Indexing NVD

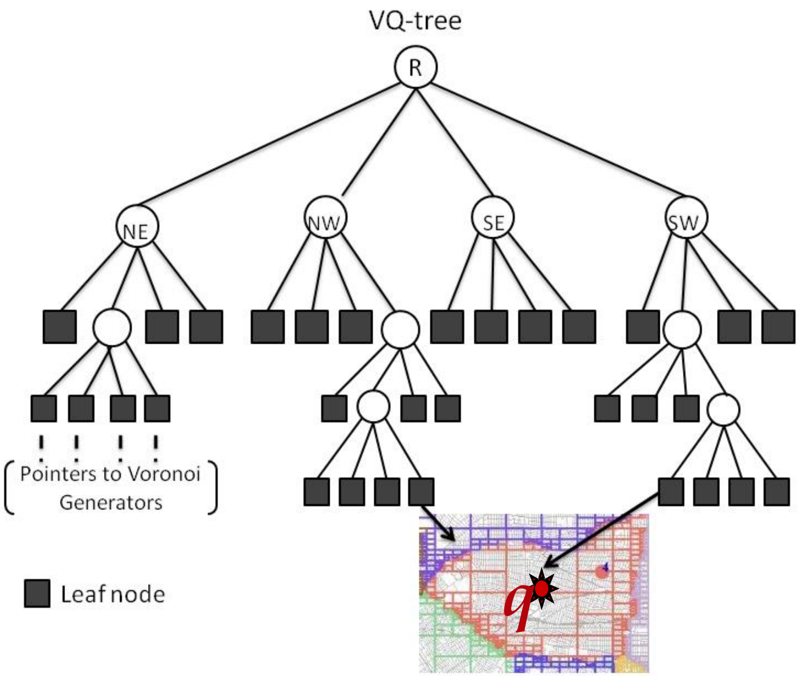
- VQ-tree
  - Each network NVP consists of disjoint quad-tree blocks



USC Viterbi School of Engineering

Indexing NVD

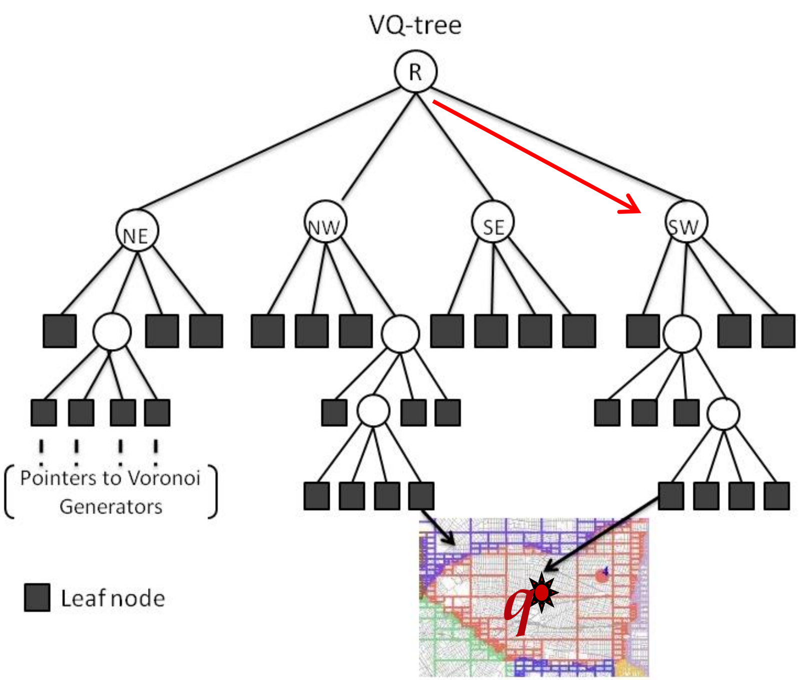
- VQ-tree
  - Each network NVP consists of disjoint quad-tree blocks
  - Leaf nodes does not keep any information about the network but store the region information (i.e., coordinates) of the quad-blocks



USC Viterbi School of Engineering

Indexing NVD

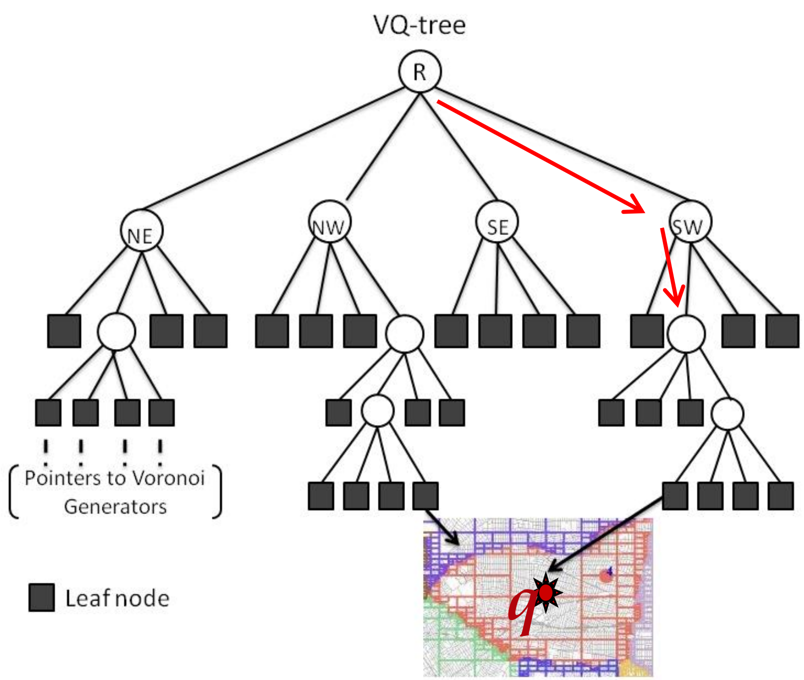
- VQ-tree
  - Each network NVP consists of disjoint quad-tree blocks
  - Leaf nodes does not keep any information about the network but store the region information (i.e., coordinates) of the quad-blocks



USC Viterbi School of Engineering

Indexing NVD

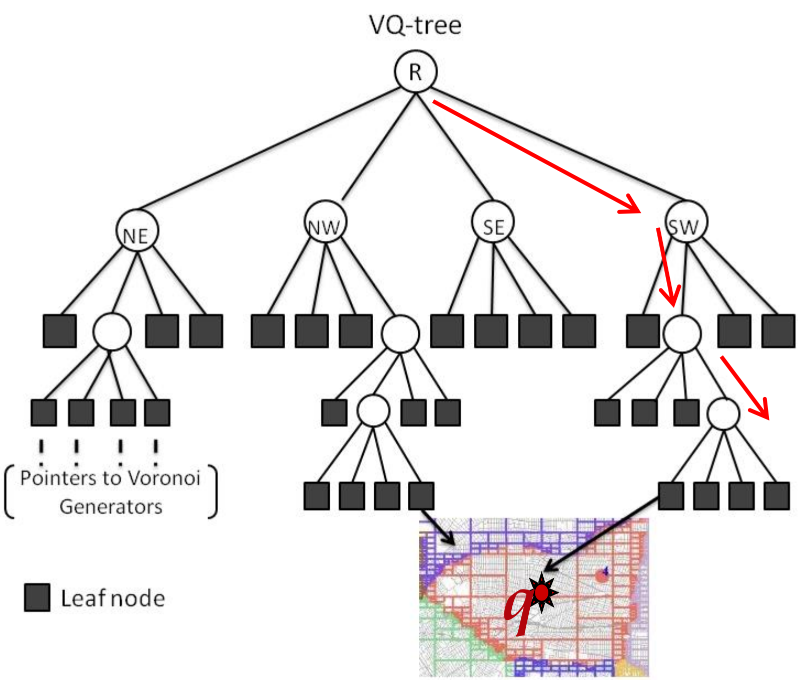
- VQ-tree
  - Each network NVP consists of disjoint quad-tree blocks
  - Leaf nodes does not keep any information about the network but store the region information (i.e., coordinates) of the quad-blocks



USC Viterbi
School of Engineering

Indexing NVD

- VQ-tree
  - Each network NVP consists of disjoint quad-tree blocks
  - Leaf nodes does not keep any information about the network but store the region information (i.e., coordinates) of the quad-blocks



USC Viterbi School of Engineering

Discussion

- What if the edge weights are changing?
- What if the both query and data objects are moving?
- What if you like to find the nearest hotel and gas station at the same time?

USC Viterbi
School of Engineering

References

- Kolahdouzan, M. R. & Shahabi, C. Voronoi-Based K Nearest Neighbor Search for Spatial Network Databases . VLDB, 2004, 840-851
- A presentation by Ugur Demiryurek in csci587 Fall’2010